

递归式 Harness 自改进(R Recursive Harness Self-Improvement)

本文件为 `2607.15524v1.pdf` 的中文译文。原文页码以 `<!-- page: N -->` 标记保留,便于逐页校对;数学符号、代码片段、引用键和模型名尽量保持原样。

原文第 1 页

2026-7-20

递归式 Harness 自改进(R Recursive Harness Self-Improvement)

Hyunin Lee^{1,2}, Jinglue Xu¹, Jeffrey Seely¹, Donghyun Lee², Matei Zaharia² 与 Yujin Tang¹ ¹Sakana AI, ²UC Berkeley 在模型 harness 协同演化的范式下,harness 不仅是推理时的脚手架,更是生成数据的组件——其执行轨迹(execution trace)可塑造未来的基础模型。这促使我们提出" harness 在环"学习:同时针对即时的智能体(agent)性能以及用于未来模型训练的轨迹质量来优化 harness。然而,持续更新由服务提供商构建的脚手架代价高昂且劳动密集。为此,我们探究:以任务特定方式优化用户自建的 harness,能否在提升执行轨迹质量的同时保持计算轻量化,且仅需少量更新迭代。为实现这一目标,我们提出递归式 Harness 自改进(R Recursive Harness Self-Improvement,RHI):将 harness 表示为智能体循环的提示词(prompt)级规范,并利用其自身修订历史上的成对反馈对其进行迭代精化。在横跨量化金融、机器人和药学领域的 30 项合成机器学习研究任务上,仅需少量 RHI 迭代即可大幅提升低推理开销智能体的性能上限,超越相应的最大推理开销设置,同时将推理成本降低高达 60%。我们表明,这些性能增益主要来源于通过更有效的智能体间信息流动实现的任务特定上下文管理的改进,而非更长的推理轨迹。最后,我们将这一行为形式化为 RHI 隐式优化目标的信息论假设,表明 RHI 是模型 harness 协同演化范式下持续学习的一种实用算法。

high [0] [1] [2] [3] [4] 0

10

20

30

30任务胜场数

sonnet-4.6

vs max

high [0] [1] [2]

opus-4.7

vs xhigh

high [0] [1] [2]

opus-4.7

vs max

0

1

2

归一化成本

high [0] [1] [2] 0

10

20

30

30任务胜场数

opus-4.8

vs xhigh

high [0] [1] [2]

opus-4.8

vs max

high [0] [1] [2]

opus-4.8

vs ultracode

0

1

2

归一化成本

性能

high+RHI 胜出 xhigh/max/ultracode 胜出+平局

成本

high+RHI xhigh/max/ultracode

RHI 迭代次数

图 1 | 少量 RHI 迭代可突破测试时扩展的性能平台。在 30 项合成机器学习研究任务上,使用 RHI 改进 harness 的 high 推理档 sonnet-4.6、opus-4.7 和 opus-4.8 智能体,在成对比较中的胜场数高于从 xhigh、max 和 ultracode 中选取的测试时扩展基线。柱形图报告两个 LLM 裁判与三个随机种子的平均胜场数。王冠()标记平均胜场数超过 19.5(满分 30)的结果。 本工作在 Sakana AI 实习期间发起并完成。通讯作者:hyunin@sakana.ai

arXiv:2607.15524v1 [cs.LG] 17 Jul 2026

原文第 2 页

目录

1 引言 3

2 预备知识 4

2.1 问题定义 4

2.2 现有方法	5
3 递归式 Harness 自改进 6	
3.1 RHI 目标函数	6
3.2 RHI 算法	8
3.3 RHI harness 定义	9
4 基准测试与评估 10	
4.1 合成开放式机器学习研究任务	10
4.2 LLM 作为裁判的评估	11
5 实验 11	
5.1 实验设置	12
5.2 评估指标	12
5.3 命题 1:少量 RHI 迭代可提升测试时扩展的性能上限	14
5.4 命题 2:RHI 的性能增益不能由输出 token 用量的增加来解释	15
5.5 命题 3:少量 RHI 迭代通过减少缓存读写使用量来提升成本效率	15
6 消融研究 16	
6.1 RHI 能否同样提升训练时扩展的性能平台?	16
6.2 哪些 harness 因素驱动了 RHI 的性能增益?	17
6.2.1 完整 harness 演化	17
6.2.2 Harness 组件演化	19
6.3 RHI 隐式优化的目标函数是什么?	22
6.3.1 隐式目标函数的证据	22
6.3.2 隐式目标的信息论假设	22
6.3.3 f_{ext} 递增的证据	24
6.3.4 f_{int} 递减的证据	25
7 相关工作 26	
8 结论 27	
A 问答 31	
B RHI harness 示例 33	
C Harness 优化器提示词 67	
D 任务示例 70	
D.1 机器学习研究任务(机器人)	71
D.2 机器学习研究任务(量化)	72

1. 引言

AI 的规模化进展通常归因于更大的模型(Kaplan et al., 2020)、更多的训练计算量(Hoffmann et al., 2022)以及改进的后训练方法(Guo et al., 2025)。然而,越来越多的迹象表明,已部署 AI 系统的进步源于基础模型与 workflow 框架(harness)的协同演化,而非单纯依赖基础模型本身(Anthropic, 2026b; OpenAI, 2026a)。这种协同演化需要一个递归反馈循环:在固定基础模型的前提下,更强的 workflow 框架能够组织更高效的智能体(agent) workflow,从而生成质量远高于独立模型的执行轨迹;这些执行轨迹随后可作为未来基础模型的训练数据(Lin et al., 2026)。现代编程智能体通过利用两类互补来源的 workflow 框架来生成此类高质量执行轨迹:服务提供商内置通用的编程与多智能体 workflow 框架(Anthropic, 2025b; Google, 2025; LangChain, 2024; OpenAI, 2024b, 2025b),而用户则进一步以特定任务的 workflow 框架对这些系统进行专化,包括可复用技能、子智能体和自动化 workflow(Anthropic, 2026c,d,e; n8n, 2026; OpenAI, 2026b)。因此,这一 workflow 框架模型协同演化循环的关键瓶颈在于 workflow 框架所产生执行轨迹的质量。本文聚焦于该递归循环的前半段:在固定基础模型的前提下,如何改进 workflow 框架以生成更高质量的执行轨迹。

然而,在实践中改进 workflow 框架颇具挑战。服务提供商构建的 workflow 框架需要泛化到多样化的用户与任务,使得持续更新在人力和成本上代价高昂(Anthropic, 2026b; OpenAI, 2026a; Shang et al., 2025; Zhang et al., 2026, 2025c)。相比之下,用户构建的 workflow 框架可针对特定任务进行优化,成为提升执行轨迹质量的实际目标(Anthropic, 2026d,e; n8n, 2026; OpenAI, 2026b)。此外,为使优化具有实用性,每次更新应在计算上保持轻量,且优化过程应在少量更新迭代内收敛,从而使用户能够在有限的单任务预算下将智能体专化到众多开放式任务。然而,如何实现此类轻量化、迭代式的 workflow 框架优化在很大程度上仍未被探索。

为解决这一执行轨迹质量瓶颈,我们研究了对用户构建的 workflow 框架进行少量轻量化优化迭代,是否能够提升智能体测试时扩展(test-time scaling)的性能上限(Wang et al., 2025a)。为此,我们提出递归 workflow 框架自改进(Recursive Harness Self-Improvement, RHI),将 workflow 框架定义为智能体循环本身,包括分配给各智能体的角色、其遵循的指令、智能体间交换的信息(契约,contract),以及控制何时调用推理的 workflow 结构(跳转,hop)。角色与指令定义任务分配,contract 规定智能体间通信,hop 决定编排器子智能体 workflow 的控制流。RHI 随后利用基于自身修订历史的成对反馈对 workflow 框架进行迭代精化(第 3 节)。首先,为实现用户构建 workflow 框架的高效优化,我们的核心设计选择是将 workflow 框架视为提示词(prompt)层面的对象,而非可执行代码。其次,RHI 轻量化的原因在于:它以轨迹局部自比较取代昂贵的种群级 workflow 框架搜索——在每次迭代中,当前 workflow 框架与其直接前驱进行比较,所得偏好历史引导下一次修订。第三,RHI 仅需少量更新迭代便可大幅提升低推理力度智能体的性能上限,往往超越相应的最大推理力度设置。

我们在 30 个涵盖量化金融、机器人技术和药学的合成开放式机器学习研究任务上对 RHI 进行评估。每项任务要求生成包含标准化交付物的完整代码仓库,输出使用成对 LLM-as-a-judge 比较进行评估。在 sonnet-4.6、opus-4.7 和 opus-4.8 上,应用于高推理力度智能体的少样本 RHI 比所有更强的测试时扩展设置(包括 xhigh、max 和 ultracode)取得了更高的胜率。这些增益在逐步更强的基础模型上持续存在。对于 opus-4.8,RHI 相对于 ultracode

基准还将推理成本降低了最多 60%(图 1;第 5 节)。我们的消融研究表明,这些增益并非源于生成更长的输出:在 RHI 各迭代轮次中,输出 token 用量几乎保持不变,而性能持续提升,缓存读写用量和推理成本往往也有所下降。相反,结果表明 RHI 改善了特定任务的上下文管理:通信 contract 和 workflow hop 变得更加任务专属,编码在这些组件中的任务相关信息增加,workflow 框架各组件间的冗余减少(第 6 节)。此外,我们观察到 RHI 的学习动态与一个隐式目标一致,该目标对 workflow 框架各组件间的信息流进行重组。我们将这一观察形式化为 RHI 隐式更新目标的信息论假说——增大每个组件与任务之间的互信息,同时惩罚各组件间的冗余——从而为 RHI 的隐式更新提供了可检验的理论解释。

总体而言,本文认为 workflow 框架模型协同演化的未来进展应聚焦于改进数据飞轮。为推进该循环的前半段——针对特定任务执行轨迹生成优化用户构建的 workflow 框架——我们将 RHI 定位为一个实用框架,使用户能够持续将编程智能体专化到日常使用中遇到的各类开放式任务。

2. 预备知识

我们首先在第 2.1 节中定义 workflow 框架优化问题,然后在第 2.2 节中回顾现有方法。

2.1. 问题定义

形式上,令 V 表示词汇(token)空间,令 $V^* := V \times V \times \dots$ 表示所有有限长度 token 序列(即句子)的集合。令 A 为一个具有固定语言模型 L 的编程智能体,令 $x \in \mathcal{X} \subset V^*$ 表示任务提示词。令 $\mathcal{H}$ 表示 workflow 框架空间,令 $\mathcal{Y}$ 表示代码仓库空间,其中每个仓库表示为文件路径/内容对的集合。给定 workflow 框架 $H \in \mathcal{H}$, 相应的智能体生成输出 $y \sim A(H, x)$, 其中 $y \in \mathcal{Y}$ 是由语言模型 L 、workflow 框架 H 和任务 x 所诱导的智能体执行过程生成的仓库。

我们的评估协议面向日常使用中遇到的开放式编程任务,其中输出是完整的代码仓库,而非单一的标量可验证答案。此类仓库无法使用单一指标进行可靠评估,因为需要从多个维度进行评估,包括功能正确性、任务对齐性、可复现性和代码质量。此外,这些标准往往难以聚合为单一数值分数,但通过输出间的成对偏好来表达则更为自然。因此,我们采用以评估提示词 $x_{\text{eval}} \in V^*$ 为条件、覆盖多项标准的成对评估协议。给定两个输出 $y_1, y_2 \in \mathcal{Y}$, 评估器返回三种可能判断之一:

$$L_{\text{eval}}(y_1, y_2, x_{\text{eval}}) \in \{y_1 \succ y_2, y_1 \sim y_2, y_2 \succ y_1\}$$

其中 $y_1 \succ y_2$ 表示在 x_{eval} 所指定的评估标准下 y_1 优于 y_2 , $y_1 \sim y_2$ 表示两个输出被判定为等价。此处 L_{eval} 表示基于 LLM 的评估器,其可以不同于编程智能体底层所用的语言模型 L 。

给定任务 $x \in \mathcal{X}$ 和评估提示词 x_{eval} , 我们的理想目标是识别出一个特定任务的 workflow 框架,使其输出优于竞争 workflow 框架所生成的输出。一个自然的种群目标为

$$H^*_x \in \operatorname{argmax}_{H \in \mathcal{H}} f_x(H), \quad f_x(H) = \mathbb{E}_{H' \sim \mu(\cdot | H' \neq H), y \sim A(H, x), y' \sim A(H', x)} [\mathbb{1}_{L_{\text{eval}}(y, y'; x_{\text{eval}}) = y \succ y'}]$$

其中 μ 是竞争 workflow 框架上的参考分布, $\mathbb{1}_{\cdot}$ 表示示性函数。

原文第 5 页

若括号内谓词为真则取值为 1, 否则为 0。函数 $f_x(H)$ 表示在评估提示(prompt) x_{eval} 下, 测试框架 (harness) H 对竞争 harness 的期望成对胜率。相应地, H^*_x 是使该期望胜率最大化的 harness。

2.2 现有方法

精确最大化式(1)需要在离散、高维的 harness 空间中搜索,并估计每个候选项对参考分布 μ 中抽取的 harness 的期望成对胜率。先前工作通常通过两种方式使该优化可行:将搜索空间限定为具体表示形式(如提示词、工作流(workflow)图、工具使用策略或可执行的 harness 代码),以及用一个有限候选集替换对 μ 的期望以便求值。

在式(1)的符号体系中,基于种群的 harness、workflow、prompt 或程序搜索方法,可视作将参考分布 μ 替换为采样种群 $S_i = \{H_{i,1}, \dots, H_{i,m}\}$, 并估计

$$\hat{f}_x(H; S_i) = \frac{1}{|S_i|} \sum_{H' \in S_i} \mathbb{E}_{y \sim A(H, x), y' \sim A(H', x)} [\mathbb{1}_{L_{\text{eval}}(y, y'; x_{\text{eval}}) = y \succ y'}]$$

不同算法在不同抽象层次上实例化种群 S_i 。

在 harness 层面, Meta-Harness 将 harness 代码本身作为候选对象,并从先前候选项的源码、分数和执行轨迹中提出新 harness (Lee et al., 2026); AutoHarness 自动合成一套代码 harness,并根据执行反馈加以精化 (Lou et al., 2026)。TTHE 在测试时应用相同的 harness 级模式:对每个无标签测试批次,它将 S_i 实例化为经多轮演化的并行 harness 分支种群,并由一个智能体(agent)评判者利用执行衍生的代理信号作为评分规则,将某一支提交至下一批次 (Nie et al., 2026)。

在 agent 与 workflow 层面,ADAS 通过 Meta Agent Search 维护一个由代码定义的 agent 档案库,并使用元智能体 (meta-agent)从该档案库中编程出新 agent(Hu et al., 2025);GPTSwarm 将 agent 表示为可优化图,并更新节点提示词与图的连通性(Zhuge et al., 2024);AFlow 使用蒙特卡洛树搜索结合代码修改和执行反馈,对以代码表示的 workflow 进行搜索(Zhang et al., 2025c)。

在程序层面,AlphaEvolve 与 ShinkaEvolve 在可执行代码上实例化相同模式:大语言模型(LLM)对候选项进行变异或重写,评估器为其打分,再由选择或父代采样决定哪些候选项作为下一代的种子(Lange et al., 2025; Novikov et al., 2025)。

在上述每种情形中,方法均未精确优化式(1)。它们构建有限候选集 S_i ,并以任务特定的分数、评估器或成对判断对其成员排序;当评分规则与偏序 $\succ$ 保序时,该排序在单调变换意义下恢复了式(2)中种群内胜率 $\hat{f}_x(\cdot; S_i)$,从而在重新播种 S_{i+1} 之前,对顶部候选项的选择或演化近似最大化了有限种群近似。提示词与流水线优化器可解释为相同的有限种群近似,只是将可行集 S_i 限定为文本提示词或模块化 LLM 程序。OPRO 将提示词视为优化变量,并请 LLM 优化器根据先前的提示词 分数对提出改进提示词(Yang et al., 2024)。TextGrad 将文本反馈转化为自然语言梯度以修订文本变量(Yuksekgonul et al., 2024)。DSPy 通过搜索提示词和少样本示例来编译模块化语言模型程序(Khattab et al., 2023)。GEPA 维护一个提示词候选项的帕累托前沿,反思执行轨迹,并提出、测试和重组提示词编辑(Agrawal et al., 2025);optimize_anything 将这种反思式文本优化视角推广至任意文本参数(Agrawal et al., 2026)。当可以实例化和评估大量候选项时,以上方法均为式(1)的自然近似。它们在本文场景下的共同局限在于: $\hat{f}_x(\cdot; S_i)$ 的保真度

原文第 6 页

随 S_i 的规模和多样性增长,然而每新增一个候选项都需要一次完整且代价高昂的黑盒 agent 执行与评估。

这种基于种群的搜索不适合用户构建的 harness 优化,因为每新增一个候选项都需要完整的黑盒 agent 执行与评估。对于持续将 agent 特化到众多开放式任务的用户而言,这一代价是难以承受的。Wang et al.(2026)确实表明,一旦将搜索成本纳入合理核算,自动 harness 演化在固定推理预算下未能持续超越并行采样、顺序精化等简单测试时缩放基线。因此,harness 优化只有在搜索开销远低于基于种群方法时才值得进行。由此,本文场景需要一种计算轻量、在少数更新迭代内即可收敛、且仍能带来显著性能提升的优化方法。

3. 递归 Harness 自改进

本节介绍所提算法——递归 Harness 自改进(Recursive Harness Self-Improvement,RHI)。第 3.1 节给出 RHI 目标函数的计算轻量松弛形式,第 3.2 节描述 RHI 算法,第 3.3 节定义 RHI 使用的 harness 表示。图 2 提供 RHI 的高层概览,算法 1 给出完整流程。

\$\$

\boxed{

\text{TASK} \rightarrow \text{Harness}[i] \rightarrow \text{output}[i] \rightarrow \text{自比较} \rightarrow \text{output}[i] \\ \text{vs} \rightarrow \text{output}[i-1] \rightarrow \text{P.F}[i] \rightarrow \text{更新 Harness}

}

\$\$

图 2 | 递归 Harness 自改进(RHI)。12:在第 i 次迭代中,编码 agent 接收任务 x 和当前 harness $H^{(i)}$,求解任务并产生输出 $\text{output}[i]$ 。3:LLM 评估器随后将 $\text{output}[i]$ 与前一轮输出 $\text{output}[i-1]$ 进行比较,返回偏好反馈(Preference Feedback) $\text{P.F}[i]$ 。4:该反馈存入自比较(self-comparison)历史记录。5:最后,LLM harness 优化器利用该历史记录将 harness 从 $H^{(i)}$ 更新为 $H^{(i+1)}$ 。

3.1 RHI 目标函数

与第 2.2 节讨论的方法不同,RHI 对式(1)采用轨迹局部(trjectory-local)松弛。在第 i 次迭代中,它将宽泛的竞争者分布 μ 替换为集中于前一 harness 的局部竞争者分布 $\nu_i = \delta_{H^{(i-1)}_x}$ 。对应的局部偏好目标为

$$\tilde{f}^{(i)}_x(H) = \mathbb{E}_{y \sim A(H,x), y^- \sim A(H^{(i-1)}_x, x)} [\mathbb{1}_{L_{\text{eval}}}(y, y^-; x_{\text{eval}}) = y \succ y^-] \tag{3}$$

该轨迹局部目标并非原始 μ 下 $f_x(H)$ 的无偏估测量;它有意将问题从与任意 harness 的全局比较转变为相对于系统自身前一 harness 的局部改进。这一区别将 Meta-Harness(Lee et al., 2026)与 Self-Harness(Zhang et al., 2026)区分开来。Meta-Harness 更宜被视为优化式(2)中的有限候选近似:它维护一个已评估 harness 的种群和帕累托前沿,存储每个候选项的源码、分数与执行

原文第 7 页

文件系统中的执行轨迹(trace),且不施加任何父节点选择规则,因此提议者在提出新测试框架(harness)时可以检查任意先前的 harness 及其 trace(Lee et al., 2026)。因此,该方法并不实例化公式(3)——公式(3)的比较对象仅为紧邻的前一个 harness $H_{x^{(i-1)}}$ 。Self-Harness 更接近公式(3):在每一轮中,当前 harness 被评估,失败案例从其执行 trace 中挖掘出来,候选编辑被提出,每个候选在保留内集和保留外集上对当前 harness 进行回归测试(Zhang et al., 2026)。然而,Self-Harness 与 RHI 的轨迹局部目标并不完全相同。它可能评估多个候选分支并合并兼容的已接受编辑,其比较信号是基于验证器的通过计数规则——在至少一个分割上提升而不降低另一个分割——而非两个生成输出之间的 LLM 成对偏好。因此,Self-Harness 只有在将偏好关系 $\succ$ 替换为回归测试接受关系之后,才能被解释为公式(3)的确定性、基于验证器的类比;若不做此替换,它在结构上是局部的,但并非 RHI 目标的直接解。

RHI 计算开销轻量。公式(3)的松弛在计算上是轻量级的,因为它大幅降低了智能体(agent)执行和成对评估的代价(表 1)。设每次优化步骤的计算代价为 $C = N_{\text{trace}} + N_{\text{pair}}$,其中 N_{trace} 为编码 agent 的执行次数(对 $\mathcal{A}$ 的调用次数), N_{pair} 为成对评估的次数(对 $\mathcal{L}_{\text{eval}}$ 的调用次数)。我们假设每个 harness 的输出在生成后被缓存,并在后续比较中复用。

表 1 | 三种 harness 搜索目标每次迭代的计算代价。其中 M 表示近似理想目标所需的有效 harness 种群大小, $m \ll M$ 为有限种群搜索所使用的采样种群大小。

目标	N_{trace}	N_{pair}	总代价
理想目标(公式 1)	M	M^2	$\Theta(M^2)$
有限种群搜索(公式 2)	m	m^2	$\Theta(m^2)$
轨迹局部 RHI(公式 3)	1	1	$\Theta(1)$

理想目标将每个候选 harness 与整个参考种群进行比较,每个 harness 需要一次执行 trace,且所有竞争对之间需要成对比较。有限种群方法通过将比较限制在大小为 m 的采样种群内来降低代价,但仍需要二次级别的成对评估。相比之下,RHI 仅将当前 harness 与其直接前驱的缓存输出进行比较,每次迭代恰好需要一次新的执行 trace 和一次成对评估。因此,当 $M \gg m \gg 1$ 时, $C_{\text{ideal}} > C_{\text{pop}} > C_{\text{RHI}}$ 。将规模扩展至 n 个独立任务会将三种代价均乘以 n ,但不改变其排序关系。

RHI 执行带噪声的局部上升。人们可能会质疑公式(3)中的松弛是否过于激进。然而,在标准成对偏好模型下,轨迹局部目标与理想种群目标保持相同的潜在效用排序。具体而言,假设存在任务效用函数 $u_x : \mathcal{H} \rightarrow \mathbb{R}$ 和严格单调递增的连接函数 σ ,满足 $\sigma(0) = \frac{1}{2}$,使得 $\Pr(H \succ H') = \sigma(u_x(H) - u_x(H'))$ 。若竞争者分布独立于候选 harness(或将条件 $H' \neq H$ 视为可忽略的非自比较修正),则两个目标均为同一潜在效用的单调函数:

$$f_x(H) = \mathbb{E}_{H' \sim \mu} [\sigma(u_x(H) - u_x(H'))], \quad \tilde{f}_x^{(i)}(H) = \sigma(u_x(H) - u_x(H_{x^{(i-1)}}))$$

原文第 8 页

因此,最大化轨迹局部目标所针对的效用排序与最大化理想种群目标相同。此外,由于 $H_{x^{(i-1)}}$ 在第 i 次迭代中固定,任何真实对 $H_{x^{(i-1)}}$ 的胜率超过 $\frac{1}{2}$ 的修订均满足 $u_x(H) > u_x(H_{x^{(i-1)}})$,从而提升了理想目标。因此,观测到的成对比较提供了一个带噪声的局部上升信号:赢得比较的修订被鼓励,而失败的修订则被丢弃或重新修订。

少量迭代更新中自历史的必要性。由于每次成对比较仅提供带噪声的局部上升信号,RHI 在有限更新预算下跨迭代累积偏好反馈,以提高 harness 优化的可靠性。具体而言,历史记录为:

$$\mathcal{D}_x^{(i)} = \left\{ L_{\text{eval}} \left(y_x^{(k)}, y_x^{(k-1)}; x_{\text{eval}} \right) \right\}_{k=1}^i, \quad \tag{4}$$

其中 $y_x^{(k)} \sim \mathcal{A}(H_x^{(k)}, x)$ 是由第 k 个 harness 生成的代码仓库,每个元素记录连续 harness 修订之间的成对偏好。这里, $H_x^{(k)}$ 表示任务 x 经过 k 次 RHI 迭代后的特定任务 harness。在全文中,当任务由上下文明确时,我们简写为 $H^{(k)}$ 。RHI 不依赖单次带噪声的比较,而是以累积的偏好历史为条件进行每次 harness 更新,从而在每次迭代仅需一次新 agent 执行和一次成对评估的情况下提升鲁棒性。由于 harness 空间是离散的文本空间,这些成对偏好不定义传统意义上的梯度。取而代之,历史 $\mathcal{D}_x^{(i)}$ 充当一种动量语义信号,引导未来的 harness 修订。

上述见解推动了下一节所介绍的 RHI 学习算法的设计。

3.2. RHI 算法

算法 1:递归 Harness 自改进(Recursive Harness Self-Improvement)

输入: 任务 $\{x_j\}_{j=1}^n \in \mathcal{X}$, agent $\mathcal{A}$, 评估器 L_{eval} , harness 优化器 L_{harness} , 评估提示(prompt) x_{eval} , 停止阈值 ϵ

初始化: 对 $\forall x \in \mathcal{X}$, 初始化 harness $H_x^{(0)} \in \mathcal{V}^*$, harness 历史 $\mathcal{D}_x \leftarrow \emptyset$ 。Agent $\mathcal{A}$ 使用初始 harness $H_x^{(0)}$ 求解任务 x , 并为 $\forall x \in \mathcal{X}$ 获得输出。

for $i = 1, 2, \dots$ do

 for $j = 1, 2, \dots, n$ do

 Agent $\mathcal{A}$ 使用 harness $H_j^{(i)}$ 求解任务 x_j , 获得输出 y_j^i 。

 评估器 L_{eval} 通过成对判断比较 y_j^i 与 y_j^{i-1} , 然后更新历史: $\mathcal{D}_j \leftarrow \mathcal{D}_j \cup \{L_{\text{eval}}(y_j^i, y_j^{i-1}; x_{\text{eval}})\}$ 。

 end

 计算所有任务的改进率 $s_i = \frac{1}{n} \sum_{j=1}^n \mathbb{1}[\left| y_j^i - \text{succ } y_j^{i-1} \right|]$, 其中 $s_i \in [0, 1]$ 。

 if $s_i < \epsilon$ then

 BREAK

 end

 对 $\forall x \in \mathcal{X}$, 在不使用 x_{eval} 的情况下更新 harness: $H_x^{(i+1)} \leftarrow L_{\text{harness}}(H_x^{(i)}, \mathcal{D}_x)$ 。

end

return $\forall x \in \mathcal{X}$ 对应的 H_x^*

基于上述见解,我们在算法 1 中给出 RHI 算法。在第 i 次迭代时,RHI 按照以下公式更新特定任务的 harness:

$$H_x^{(i+1)} = L_{\text{harness}} \left(H_x^{(i)}, \mathcal{D}_x^{(i)} \right), \quad \tag{5}$$

原文第 9 页

其中 L_{harness} 是基于 LLM 的工作流框架优化器(harness optimizer)。此处,从 $H_x^{(i)}$ 到 $H_x^{(i+1)}$ 的直接更新使递归工作流框架改进(Recursive Harness Improvement,RHI)在计算上保持轻量,而对 $\mathcal{D}_x^{(i)}$ 的条件化则使 RHI 能够在仅需少量更新迭代的情况下收敛。值得注意的是, x_{eval} 仅由评估器在生成成对反馈时使用;在工作流框架修订过程中,它不会直接提供给 L_{harness} 。因此,RHI 应被视为优化一个隐式偏好目标,而非直接最大化某个显式标量奖励。尽管工作流框架优化器无法直接观测 x_{eval} ,但反馈历史 $\mathcal{D}_x^{(i)}$ 是由以

x_{eval} 为条件的评估器生成的。因此,更新轨迹可通过累积的成对比较,间接地使 workflow 框架修订与 x_{eval} 保持一致。从这个意义上说,RHI 实现了递归自我改进:每个 workflow 框架均利用由其自身先前修订所诱导的偏好历史进行优化,而无需直接观测评估提示词(prompt) x_{eval} 。附录 B 提供了在随机选取的任务 x 上, $H_x^{(i)}$ ($i \in \{0,1,2,\dots\}$) 迭代演化的示例。附录 C 提供了 workflow 框架优化器 L_{harness} 所使用的系统提示词与用户提示词。

3.3. RHI 工作流框架定义

图3 | RHI 工作流框架的分解。我们将工作流框架划分为两个主要组成部分:智能体设计(agent design)和智能体工作流(agent workflow)。智能体设计规定候选智能体的角色与指令。智能体工作流进一步分解为契约(contract)和跳步(hop):契约定义子智能体与协调器之间交换的信息,而跳步定义交互结构,即整体工作流逻辑。

我们将工作流框架定义为智能体循环本身,并将其分解为两个主要组成部分:智能体设计和智能体工作流。RHI 优先对智能体工作流进行更新,该工作流会针对每个任务进行迭代适配(详见附录 C 中 L_{harness} 的系统提示词)。我们进一步将工作流分解为两个组件:契约(contract),用于规定智能体之间通过通信接口传递哪些信息;以及跳步(hop),用于规定交互结构,包括协调器与子智能体的工作流步骤。图 3 展示了这一高层分解结构,图 4 则说明了这些组件如何以文本规范的形式在提示词中表示。

RHI 为何优先更新工作流而非智能体设计? 我们对契约和跳步的关注源于以下假设。

我们假设,任务特定的契约和跳步能够通过更有效的上下文管理,同时提升任务性能和推理效率。具体而言,任务特定的契约规定了智能体之间应传递哪些信息,而无需协调器或下游智能体以整个交互历史为条件进行推理。这可以减少冗余上下文的传播,提升 KV 缓存效率,并降低推理成本。这一视角自然地引导出多智能体并行执行或单一智能体内多角色推理等工作流设计。

从概念上看,契约优化类似于对智能体间信息流施加一种依赖任务的稀疏模式。完全共享的交互历史类似于稠密注意力,

原文第 10 页

即每个智能体均可访问其他智能体的所有先验信息。相比之下,任务特定的契约将通信限制在对下游决策最相关的信息上。因此,优化契约可被视为在工作流框架层面学习一种任务特定的通信稀疏结构,从而实现更高效的上下文管理。

候选智能体(Agent candidates)

```
Agent1:\{\$role: ..., instruction: ..., contract (out-to-orchestrator): \$\{c_{11}, c_{12}, \ldots\}\}\$
\$ \cdots \$
AgentN:\{\$role: ..., instruction: ..., contract (out-to-orchestrator): \$\{c_{N1}, c_{N2}, \ldots\}\}\$
```

智能体协调器-子智能体工作流(Agent orchestrator-subagent workflow)

```
Hop1:\$ ...
\$ \cdots \$
HopM:\$ ...
```

辅助规则(Auxiliary rules)

终止前的验收门控、失败回退规则、证据驱动的召回触发器,以及通用的智能体间通信规则等。

图4 | 以提示词表示的工作流框架 $H^{(i)}$ 的结构。工作流框架规定了候选智能体、协调器-子智能体工作流,以及用于终止、回退、召回和智能体间通信的辅助控制规则。

工作流框架即提示词 RHI 工作流框架首先规定一组候选智能体,每个智能体由其角色、指令和契约(即其向协调器输出的预期内容)来描述;详见图 4 中的"候选智能体"部分。其次规定管理这些智能体之间交互的工作流;详见图 4 中的"智能体协调器-子智能体工作流"部分。这两个组件出现在每次 RHI 迭代中,作为工作流框架优化的任务无关脚手架。在迭代过程中,RHI 进一步诱导出依赖任务的局部工作流框架组件,如验收门控、失败回退规则和证据驱动的召回触发器;详见图 4 中的"辅助规则"部分。

4. 基准测试与评估

4.1. 合成开放式机器学习研究任务

我们在一套包含 30 个合成开放式机器学习(ML)研究任务的测试集上对 RHI 进行评估,这些任务涵盖三个领域:量化金融、机器人学和制药机器学习,每个领域各 10 个任务。为构建这些任务,我们使用 LLM 将与领域相关的行业招聘信息转化为研究风格的任务提示词,使其类似于研究人员在相应职位中可能收到的任务。原始招聘信息来源如下:

1. 量化 ML 研究:<https://www.citadel.com/careers/details/quantitative-researcher-phd-graduate-us/>
2. 机器人学 ML 研究:<https://www.amazon.jobs/en/jobs/10443615/member-of-technical-staff-far-frontier-ai-robotics>
3. 制药 ML 研究:<https://careers.gene.com/us/en/job/202606-115542/Machine-Learning-Scientist-Synthesis-Planning-and-Optimization>

所生成的任务需要综合运用编程能力、领域推理、实证分析,

原文第 11 页

以及机器学习实验等。任务涵盖数据分析、模型训练、基准构建、消融设计和实证报告等目标。每项任务要求智能体(agent)构建完整的代码仓库,并生成提示词(prompt)中指定的交付物。该基准旨在支持多样化的机器学习任务,同时保持统一的评估接口。每个任务提示词由两部分组成:任务描述(说明研究目标)和交付物章节(说明所需的输出产物)。各任务的核心交付物已标准化,包括:(1) research_report.md,(2) .png 格式的可视化文件,(3) metrics.json,以及 (4) index.json(记录所有生成文件的路径)。部分任务还额外要求特定领域的产物,例如模型对比文件、消融摘要或可复现脚本。附录 D 提供了来自三个领域(机器人学、量化金融和药学)的示例任务提示词。

4.2. 基于 LLM 的裁判评估

我们的评估协议遵循第 2.1 节中的问题形式化定义,通过多项评估标准下的成对偏好比较输出结果。这对应于 RHI 算法的第 3 步(图 2)。给定针对同一任务生成的两个代码仓库,标准化的交付物结构提供了统一的评估接口,使 LLM 评估器能够比较特定任务的交付物并选出更优仓库。对于每次比较,我们从两个仓库中提取任务指定的交付物,将其内容连同评估提示词 $\$x_\\text{eval}\$$ 一并提供给 $\$L_\\text{eval}\$$ 。由于仓库可能包含大量产物(如源代码、日志、文本文件和图像),将所有生成文件全部纳入往往不切实际。因此,我们仅评估任务指定的交付物,并将评估器提示词控制在合理的上下文预算内(约为评估器最大输入长度的 30%–40%),以缓解上下文衰减问题(Hong et al., 2025)。必要时,对较长文件按相同协议进行截断处理,两个候选仓库保持一致。

为评估跨评估器的鲁棒性,我们使用两种配置,采用不同的模型系列和推理设置:gpt-5.5(最高推理力度)和 opus-4.7(或 opus-4.8,xhigh 推理力度)。每种配置使用三个随机种子进行评估。评估器 $\$L_\\text{eval}\$$ 依据以下标准对两个仓库进行比较。

评估提示词($\$x_\\text{eval}\$$)

1. 交付物覆盖度:输出是否满足任务的显式交付物要求,包括 Deliverables、Deliverable 或内嵌输出要求中列出的文件。
2. 数值与实证严谨性:方法论、基线、指标和局限性是否恰当且内部一致。
3. 可复现性:仓库是否包含依赖项、入口点、随机种子及足够的文档以复现结果。
4. 呈现质量:报告是否清晰、结构良好,并适当整合了图表和实证结果。
5. 工程质量:基于文件树和代码摘录,仓库是否组织良好、模块化且可读性强。
6. 任务对齐性:解决方案是否切实解决了既定目标,而未偏离至不同问题。

附录 E 提供了 $\$L_\\text{eval}\$$ 所使用的完整系统提示词和用户提示词。

5. 实验

第 5.1 节描述实验设置,第 5.2 节定义评估指标,第 5.3–5.5 节呈现主要实验结果。

5.1. 实验设置

RHI 在设计上计算开销轻量,仅需少数更新迭代即可收敛。我们的实验设置旨在验证 RHI 能否提升 agent 测试时扩展 (test-time scaling) 的性能上限。

为此,我们以三个基础语言模型实例化编码 agent $\mathcal{A}$: Claude Sonnet 4.6、Claude Opus 4.7 和 Claude Opus 4.8。对于每个模型,默认 agent 使用推理力度 high。我们将这三种基础配置分别记为 L1-high、L2-high 和 L3-high,对应的基础 agent 记为 $A_{L_b\text{-high}}$ 。对于每个任务 $x \in \mathcal{X}$ 和每个基础模型 $b \in \{1,2,3\}$,我们首先以初始执行框架(harness)运行基础 agent $A_{L_b\text{-high}}(H_x^{(0)}, x)$,然后应用 RHI 迭代生成一系列任务特定 harness $\{H_x^{(0)}, H_x^{(1)}, \dots\}$ 。在每次迭代中,我们将 RHI 改进后的 agent 输出与未使用 RHI 的对应高测试时推理 agent 输出进行比较:

$$y_{\text{RHI}} \sim A_{L_b\text{-high}}(H_x^{(i)}, x) \quad \text{vs.} \quad y_{\text{scale}} \sim A_{L_b\text{-xhigh/ultracode/max}}(x).$$

其中 $A_{L_b\text{-high}}(H_x^{(i)}, x)$ 表示在第 i 次迭代时配备了 RHI 改进 harness $H_x^{(i)}$ 的基础编码 agent。相较于原始基础 agent $A_{L_b\text{-high}}$,唯一的修改是输入提示词:从 x 变为 $x \cup H_x^{(i)}$,即以文本规范的形式将演化后的 harness 附加到原始任务中。相比之下, $A_{L_b\text{-xhigh/ultracode/max}}(x)$ 表示同一编码 agent 在更高推理力度下进行评估,但不使用 RHI 生成的 harness。

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

count (mean $\pm$ SD)

gpt-5.5-max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

opus-4.7-xhigh

sonnet-4.6-max vs sonnet-4.6-high+ [i]

high+ [i] wins max wins Evaluator

(a) sonnet-4.6-high + H[\$i\$], $i \in \{0,1,2,3,4\}$.

high max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

0.0

0.5

1.0

1.5

2.0

2.5

3.0

归一化值

2.56

1.80

1.88

2.38

2.16

2.52

归一化成本

high max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

0.0

0.5

1.0

1.5

2.0

1.53

1.78 1.80 1.86

1.76 1.71

归一化输出

high max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

5 4.91

1.88 2.01

3.31

2.78

3.71

归一化缓存读写

(b) 归一化资源用量。

图 5 | 经过两次迭代后,RHI 提升了 sonnet-4.6 测试时扩展的实证性能上限。我们将 sonnet-4.6-high+H[\$i\$] 与 sonnet-4.6-max 进行比较,其中 H[\$i\$] 表示经过 \$i\$ 次 RHI 迭代后的 harness。左图报告 30 个任务上的成对比较结果;右图报告归一化后的平均成本、输出 token 数量及缓存读写用量(以 sonnet-4.6-high 为基准归一化)。结果对两个 LLM 裁判和三个随机种子取平均;误差条表示标准差。

5.2. 评估指标

符号说明。为简洁起见,我们在图表和表格标签中将括号形式 H[\$i\$] 与 $H^{\{i\}}$ 互换使用,均表示相同的迭代索引。因此, H[\$i\$] 对应 $H^{\{i\}}$,或对于特定任务 x 对应 $H_x^{\{i\}}$ 。

图 5 比较了基于 sonnet-4.6-high 的 RHI 与测试时扩展基线 sonnet-4.6-max;图 6 比较了基于 opus-4.7-high 的 RHI 与 opus-4.7-xhigh/max;图 7 比较了基于 opus-4.8-high 的 RHI 与 opus-4.8-xhigh/ultracode/max。对于每个基础模型,我们报告四项指标:成对胜负计数(即性能)、归一化成本、输出 token 数量及缓存读写用量。成本、输出 token 数量和缓存读写用量均通过 Claude 编码 agent 的内部成本函数记录。

原文第 13 页

max xhigh high

count (mean $\pm$ SD)

gemini-3.1-pro-high

max xhigh high

gpt-5.5-high

max xhigh high

gpt-5.5-xhigh

max xhigh high

gpt-5.5-max

max xhigh high

opus-4.7-high

max xhigh high

opus-4.7-xhigh

opus-4.7-high/xhigh/maxvsopus-4.7-high+ [0]

Baseplayer:high+ [0]

high+ [0]win high+ [0]lose tie Evaluator

(a) opus-4.7-high + H[0].

max xhigh high

high+ [0]

count (mean $\pm$ SD)

gemini-3.1-pro-high

max xhigh high

high+ [0]

gpt-5.5-high

max xhigh high

high+ [0]

gpt-5.5-xhigh

max xhigh high

high+ [0]

gpt-5.5-max

max xhigh high

high+ [0]

opus-4.7-high

max xhigh high

high+ [0]

opus-4.7-xhigh

opus-4.7-high/xhigh/maxvsopus-4.7-high+ [1]

Baseplayer:high+ [1]

high+ [1]win high+ [1]lose tie Evaluator

(b) opus-4.7-high + H[1].

high xhigh max

high+ [0]

high+ [1]

0.0

0.5

1.0

1.5

2.0

2.5

3.0

Normalized value

1.41

2.60

1.09

2.11

归一化成本

high xhigh max

high+ [0]

high+ [1]

0.0

0.5

1.0

1.5

2.0

2.5

1.20

1.76

1.15

2.06

归一化输出

high xhigh max

high+ [0]

high+ [1]

0.0

0.5

1.0

1.5

2.0

2.5

3.0

3.5

4.0

1.63

3.37

1.06

2.11

归一化缓存读写量

(c) 归一化资源用量。

图6 | 经过一次迭代后,RHI(递归提示词学习)提升了 opus-4.7 测试时缩放的经验上界。

两两对比面板在 30 个任务上将 opus-4.7-high+H[0] 和 opus-4.7-high+H[1] 与 opus-4.7-high/xhigh/max 进行比较。资源面板报告归一化后的平均成本、输出词元数及缓存读写用量,归一化基准为 opus-4.7-high。结果在六种评估器配置和三个随机种子上取平均,误差棒表示标准差。性能。图 5a、6a、6b 及 7a-7d 报告了 30 个任务上的性能, y 轴给出胜/负/平局次数。图 5a 将 sonnet-4.6-max 与 sonnet-4.6-high+H[i]($i \in \{0,1,2,3,4\}$)进行比较。图 6a 和 6b 将 opus-4.7-high+H[0] 和 opus-4.7-high+H[1] 与 opus-4.7-high/xhigh/max 进行比较。图 7a-7d 将 opus-4.8-high 和 opus-4.8-high+H[i]($i \in \{0,1,2\}$)与 opus-4.8-xhigh/ultracode/max 进行比较。每次评估在不同随机种子下重复三次;我们报告各次重复的均值和标准差。为评估鲁棒性,sonnet-4.6 和 opus-4.8 使用两种评估器配置,opus-4.7 使用六种,均对评估器模型及其推理力度设置进行变化。

归一化成本、输出词元数及缓存读写用量。图 5b、6c 和 7e 报告归一化后的平均成本、输出词元数及缓存读写用量。对每个任务分别记录这三项指标,以对应基础智能体(base agent)的值进行归一化,再在任务间取平均。归一化基准分别为 sonnet-4.6-high、opus-4.7-high 和 opus-4.8-high;在各图中,基础智能体以最左侧灰色柱表示,归一化值为 1.0。成本分布详见附录 F。

原文第 14 页

xhigh

ultracode max

count (mean $\pm$ SD)

gpt-5.5-max

xhigh

ultracode max

opus-4.8-xhigh

opus-4.8-high/xhigh/ultracode/maxvsopus-4.8-high

Baseplayer:high

highwin highlose tie Evaluator

(a) opus-4.8-high

xhigh

ultracode max

count (mean $\pm$ SD)

gpt-5.5-max

xhigh

ultracode max

opus-4.8-xhigh

opus-4.8-high/xhigh/ultracode/maxvsopus-4.8-high+ [0]

Baseplayer:high+ [0]

high+ [0]win high+ [0]lose tie Evaluator

(b) opus-4.8-high + H[0]

xhigh

ultracode max

count (mean $\pm$ SD)

gpt-5.5-max

xhigh

ultracode max

opus-4.8-xhigh

opus-4.8-high/xhigh/ultracode/maxvsopus-4.8-high+ [1]

Baseplayer:high+ [1]

high+ [1]win high+ [1]lose tie Evaluator

(c) opus-4.8-high + H[1]

xhigh

ultracode max

count (mean $\pm$ SD)

gpt-5.5-max

xhigh

ultracode max

opus-4.8-xhigh

opus-4.8-high/xhigh/ultracode/maxvsopus-4.8-high+ [2]

Baseplayer:high+ [2]

high+ [2]win high+ [2]lose tie Evaluator

(d) opus-4.8-high + H[2]

high xhigh max

ultracode

high+ [0]

high+ [1]

high+ [2]

Normalized value

1.63

2.19

4.15

1.35

1.85 1.69

归一化成本

high xhigh max

ultracode

high+ [0]

high+ [1]

high+ [2]

2.06 1.98

3.57

1.42

1.81

1.65

归一化输出

high xhigh max

ultracode

high+ [0]

high+ [1]

high+ [2]

1.78

2.52

4.74

1.30

1.77 1.69

归一化缓存读写量

(e) 归一化资源用量。

图7 | 经过两次迭代后,RHI 提升了 opus-4.8 测试时缩放的经验上界。两两对比面板在 30 个任务上将 opus-4.8-high 和 opus-4.8-high+H[i] ($i \in \{0,1,2\}$) 与 opus-4.8-xhigh/ultracode/max 进行比较。资源面板报告归一化后的平均成本、输出词元数及缓存读写用量,归一化基准为 opus-4.8-high。结果在两种 LLM 裁判配置和三个随机种子上取平均,误差棒表示标准差。

我们将在接下来的三个小节中讨论主要发现。

5.3 命题 1:少样本 RHI 提升测试时缩放的性能上界

对于 Sonnet-4.6 基线,图 5a 显示,经过两次 RHI 迭代后,sonnet-4.6-high+H[2] 优于更强的测试时缩放基线 sonnet-4.6-max,在 30 次两两比较中赢得 20 次。此外,sonnet-4.6-high+H[i] ($i \in \{3,4\}$) 继续优于该基线。

两个更强基础模型上的趋势相同。对于 Opus-4.7 基线,图 6a 显示 opus-4.7-high+H[0] 弱于 opus-4.7-xhigh 和 opus-4.7-max 基线。然而,图 6b 显示仅需一次 RHI 迭代即可超越二者。对于 Opus-4.8 基线,图 7b 显示 opus-4.8-high+H[0] 略优于或与 opus-4.8-xhigh 持平。经过两次 RHI 迭代后,图 7d 显示 opus-4.8-high+H[2] 优于所有测试时缩放基线:opus-4.8-xhigh、opus-4.8-ultracode 和 opus-4.8-max。这些结果表明,在我们的基准测试和评估协议下,少量几次 RHI 迭代即可将性能提升至同系列测试时缩放饱和点之上。重要的是,这一提升并不局限于某一特定基础模型能力,而是在逐渐增强的模型中持续存在。

原文第 15 页

命题 1.1:在提示词(prompt)层面优化用户构建的工作流框架(harness)可以优于系统层面由提供商内置的工作流框架

一个尤为值得关注的结果是 opus-4.8-high+H[2] 优于 opus-4.8-ultracode(图 7d)。opus-4.8-ultracode 在 xhigh 推理能力基础上配备了内置动态工作流(workflow),可以生成多个子智能体(subagent)(Anthropic, 2026a)。我们将此结果解读为以下证据:在我们的基准上,通过提供商内置的工作流框架动态生成多智能体工作流并非总是足够的。相反,直接通过提示词提供的、针对特定任务由用户构建的工作流框架可以超越它。

这一发现揭示了依赖单一固定的系统级工作流框架来覆盖宽泛且异质任务分布的局限性。通过在提示词层面优化工作流框架,RHI 使多智能体工作流能够针对每个任务进行自适应调整。因此,相对于 opus-4.8-ultracode 的优势不仅证明了在用户构建的工作流框架上进行优化优于提供商内置工作流框架的有效性,还支持了"工作流框架即提示词(harness-as-prompt)"的表述——在该表述中,工作流框架的设计成为一个显式的、针对任务的优化变量,而非固定的后端工作流。

5.4 命题 2:RHI 的性能提升不能由输出词元用量的增加来解释

在此,我们表明 RHI 带来的性能提升有别于测试时缩放——后者主要依赖于输出词元用量的增加——从而证明 RHI 是一种独立的方法。

对于 Sonnet-4.6 基线,图 5b 显示归一化输出词元用量在各 RHI 迭代 $H[i]$ ($i \in \{0,1,2,3,4\}$) 之间几乎保持不变,范围为 1.71 至 1.86。与此形成对比的是,图 6a 和 6b 显示性能在相同迭代过程中持续提升。

Opus-4.8 基线上也观察到相同的解耦现象。图 7e 显示归一化输出词元用量几乎持平,范围为 1.42 至 1.81,而图 7 显示性能在 $H[i]$ ($i \in \{0,1,2\}$) 之间持续提升。对于 Opus-4.7 基线,证据尚无定论。图 6c 显示输出词元用量与性能在 $H[i]$ ($i \in \{0,1\}$) 上同步增长,因此两种解释无法区分。此外,该比较仅跨两次迭代,少于其他模型,进一步限制了可得出的结论。总体而言,性能提升的主要驱动力并非更长的生成序列:在三个模型中的两个上,性能提升而输出词元用量几乎不变。

与上述观察一致,在下一小节中我们将说明,RHI 的增益与 KV 缓存用量的关联更为密切,而非与输出词元用量,这表明高效的上下文管理——而非生成长度本身——才是主要的驱动因素。

5.5 命题 3:少样本 RHI 通过降低缓存读写用量提升成本效率

该命题支持第 3.3 节提出的假设,即 RHI 优先更新工作流而非智能体设计。具体而言,我们表明,除性能提升外,RHI 在所有基础模型上均持续提升了成本效率。

原文第 16 页

对于 sonnet-4.6 基线,图 5b 显示 sonnet-4.6-high+H[2] 的归一化成本(2.38)比 sonnet-4.6-max(2.56)低 7%,同时缓存读写用量减少了 33%(4.91 → 3.31)。

对于更强的模型也观察到类似趋势。对于 opus-4.7 基线,图 6c 显示 opus-4.7-high+H[1] 的归一化成本(2.11)比 opus-4.7-max(2.60)低 18%,缓存读写用量减少了 37%(3.37 → 2.11)。对于 opus-4.8 基线,图 7e 显示 opus-4.8-high+H[2] 的归一化成本(1.69)相比 opus-4.8-max(2.19)降低了 23%,相比 opus-4.8-ultracode(4.15)降低了 60%。对应的缓存读写用量分别减少了 32%(2.51 → 1.69)和 64%(4.74 → 1.69)。我们将这些节省归因于更高效的上下文管理。RHI 学习特定任务的通信契约(contract):这些文本接口规定了哪些信息应从一个智能体(agent)传递到另一个智能体,或从子智能体传递到编排器。这些契约使每个智能体能够仅基于其下游决策所需的信息进行条件化,而无需依赖完整的交互历史,从而抑制冗余上下文的传播。优化多智能体工作流(workflow)因此可视为学习智能体间通信的特定任务稀疏模式。这在概念上类似于稀疏自注意力机制——通过选择注意力模式中的特定条目来替代密集的全对全词元交互,以提升长上下文效率(Beltagy et al., 2020; Child et al., 2019; Zaheer et al., 2020)。由于 Claude 风格编码智能体的推理成本在很大程度上取决于缓存读写,上下文占用的减少可以直接转化为更低的成本。

我们提醒不应将 opus-4.7 和 opus-4.8 中更大的节省幅度(图 6c 和 7e)解读为 RHI 随基础模型增强而固有地变得更有效的证据。更为保守的解释是:对一个已经很强的模型进行测试时计算扩展(test-time compute scaling)会产生递减的性价比回报;RHI 提供了一种替代方案,通过提示词级别的框架(prompt-level harness)对智能体的计算进行重构,而非均匀地增加推理算力。

第 3 节与第 5 节小结

递归框架自我改进(Recursive Harness Self-Improvement,RHI)是一种计算轻量级方法,只需少量迭代即可大幅提升测试时计算扩展基线的性能上限,同时将推理成本降低多达 60%。这一优势在逐步增强的基础模型上持续存在。RHI 带来的性能提升并不能用输出词元用量的增加来解释;相反,它与更高效的上下文管理相关,具体体现在缓存读写用量的减少和推理成本的降低。

我们将在下一节中探讨这些收益背后的驱动因素。

6. 消融研究

各小节分别针对不同的分析问题。第 6.1 节探讨 RHI 是否能在训练时扩展之外提供额外收益。第 6.2 节研究驱动 RHI 性能提升的因素。最后,第 6.3 节基于第 6.2 节的观察,为 RHI 构建一个隐式目标函数。

6.1. RHI 能否同样提升训练时扩展的性能平台?

一个自然的问题是,RHI 是否也能通过替换更强的基础模型来提升训练时扩展的性能平台。为此,我们将 sonnet-4.6-high+H[i]($i \in \{0,1,2,3,4\}$)与 opus-4.7-high 及 opus-4.7-xhigh 进行比较。图 8 显示,RHI

原文第 17 页

显著提升了较弱的 sonnet-4.6-high 智能体的性能,收益在 2~4 次迭代时趋于饱和。然而,这些提升并不能持续弥合与更强的 opus-4.7 基线之间的差距。

因此,我们将 RHI 理解为训练时扩展的补充,而非替代。RHI 能够改善固定模型的使用方式,并提升同系列测试时扩展可达到的性能上限,但它并不能消除更强基础模型所带来的优势。这一观察也为 RHI 的进一步算法改进留下了空间。

sonnet-4.6-high+ [0]

sonnet-4.6-high+ [1]

sonnet-4.6-high+ [2]

sonnet-4.6-high+ [3]

sonnet-4.6-high+ [4]

count (mean $\pm$ SD)

gpt-5.5-max

sonnet-4.6-high+ [0]

sonnet-4.6-high+ [1]

sonnet-4.6-high+ [2]

sonnet-4.6-high+ [3]

sonnet-4.6-high+ [4]

opus-4.7-xhigh

opus-4.7-high/xhigh vs sonnet-4.6-high+ [i]

sonnet-4.6-high+ [i] wins opus-4.7-high wins opus-4.7-xhigh wins Evaluator

图 8 | RHI 是训练时扩展的补充而非替代。我们将 sonnet-4.6-high+ $H[i]$ ($i \in \{0,1,2,3,4\}$) 与更强基础模型基线 opus-4.7-high 和 opus-4.7-xhigh 进行比较。在较弱的 sonnet-4.6-high 基础模型上应用 RHI, 不能持续弥合与更强的 opus-4.7 基线之间的差距。

6.2. 哪些框架(harness)因素驱动了 RHI 的性能提升?

第 5 节的实验表明,RHI 的性能提升主要不能用更长的生成来解释。因此,我们考察框架在 RHI 各迭代轮次中的演化过程。第 6.2.1 节分析完整框架的演化,第 6.2.2 节则探究哪些框架组件驱动了 RHI 的性能提升。

6.2.1. 完整框架的演化

可视化。我们首先分析 sonnet-4.6-high 的完整提示词表示框架的演化过程。对于每个 H^i_x ($i \in \{0,1,2,3,4\}$), 我们对完整文本规范进行嵌入, 并使用 t-SNE 和 UMAP 将嵌入投影到二维空间。我们使用两种嵌入模型评估鲁棒性: OpenAI text-embedding-3-large (OpenAI, 2024a) 和 all-mpnet-base-v2 (Reimers and Gurevych, 2019; Song et al., 2020)。该嵌入分析是一种诊断手段, 而非机制性解释。由于编码智能体是黑盒系统, 我们无法直接检查文本框架如何改变模型的内部计算或输出分布。因此, 我们使用文本嵌入作为语义变化的外部代理, 并考察所得框架轨迹是否在迭代轮次、嵌入模型和投影方法之间具有一致性。图 9a 和图 9b 展示了 $H[i]$ 的相同低维表示, 但标注方式不同: 图 9a 中颜色表示领域, 黑色轮廓标记初始框架 $H(0)$; 图 9b 中颜色表示 RHI 迭代轮次, 标记形状表示领域。图 9a 清晰地显示了初始框架 $H(0)$ 与经 RHI 改进的框架 $H(1)$, ..., $H(4)$ 之间的分离, 这一分离在不同嵌入模型和投影方法下保持一致。结合图 5a 中的性能提升, 这些结果表明, 将初始领域级框架 $H(0)$ 适配为特定任务框架是关键因素之一。

原文第 18 页

all-mpnet-base-v2

UMAP t-SNE

OpenAI text-embedding-3-large Sonnet-4.6-high RHI(递归工具调用改进)

Domain

quant

pharmacy

robotics

Domain

quant

pharmacy

robotics

RHI

[0]

[1]

(a) 初始工具调用框架(harness) $H[0]$ 与经 RHI 改进后的框架 $H[i], i \in \{1,2,3,4\}$ 。

all-mpnet-base-v2

UMAP t-SNE

OpenAI text-embedding-3-large

Sonnet-4.6-high RHI

RHI

[0]

[1]

[2]

[3]

[4]

RHI

[0]

[1]

[2]

[3]

[4]

Domain

quant

pharmacy

robotics

(b) 各 RHI 迭代轮次 $H[i]$ 的框架表示, $i \in \{0,1,2,3,4\}$ 。

[0] [1]

[1] [2]

[2] [3]

[3] [4]

RHSI

0.70

0.75

0.80

0.85

0.90

0.95

1.00

$\cos([i], [i + 1])$

all-mpnet-base-v2

框架改进相似度

[0] [1]

[1] [2]

[2] [3]

[3] [4]

RHSI

0.70

0.75

0.80

0.85

0.90

0.95

1.00

$\cos([i], [i + 1])$

OpenAI text-embedding-3-large

(c) 相邻框架相似度。

[1] [2] [3] [4]

RHSI

0.70

0.75

0.80

0.85

`cos( [0], [i])`

`all-mpnet-base-v2`

相对 H[0] 的累积漂移

[1] [2] [3] [4]

RHSI

0.70

0.75

0.80

0.85

`cos( [0], [i])`

`OpenAI text-embedding-3-large`

(d) 相对 H[0] 的累积漂移。

[0] [1] [2] [3] [4]

RHSI

0.90

0.92

0.94

0.96

0.98

1.00

`mean pairwise cos`

`all-mpnet-base-v2`

框架改进同质化

(按领域)

`quant`

`pharmacy`

`robotics`

[0] [1] [2] [3] [4]

RHSI

0.90

0.92

0.94

0.96

0.98

1.00

mean pairwise cos

OpenAI text-embedding-3-large

(e) 领域内跨任务相似度。

图 9 | sonnet-4.6-high 上各 RHI 迭代轮次中完整框架表示的演化。图注中 $H[i]$ 对应正文符号 $H(i)$,或任务 x 对应的 $H(i)_x$ 。由于此分析将整个框架作为单一文本对象进行嵌入,因此无法识别是哪些框架组件带来了性能提升。我们将在第 6.2.2 节通过组件级分析弥补这一局限。余弦相似度分析。接下来,我们使用余弦相似度定量刻画完整框架的演化过程。图 9c 报告了 $H(i)$ 与 $H(i+1)$ 之间的相邻相似度。从 $H(0)$ 到 $H(1)$ 的相似度(0.82)明显低于后续各次迭代间的相似度(0.97、0.98、0.99),表明第一次 RHI 更新产生的语义变化最大,而后续更新则更为渐进。

图 9d 报告了各框架与初始框架之间的累积相似度,即 $\cos(H[0], H[i])$ 。相似度随迭代轮次单调下降($0.82 \rightarrow 0.78 \rightarrow 0.76 \rightarrow 0.75$),表明 RHI 逐渐偏离初始框架,而非回归原始状态。最后,

原文第 19 页

图 9e 度量了领域内跨任务相似度。对于领域 D ,我们计算

$$\frac{1}{|P_D|} \sum_{\{x, x'\} \in P_D} \cos(\text{emb}(H^{(i)}_x), \text{emb}(H^{(i)}_{x'})),$$

$\text{其中 } P_D = \{(x, x') : x, x' \in D, x \neq x'\}.$

第一次 RHI 迭代后领域内相似度的上升,表明 RHI 更新在框架远离初始规范的同时,仍共享着共同的领域级结构。由于完整框架聚合了角色(roles)、指令(instructions)、契约(contracts)和跳转(hops),我们接下来对这些组件进行分项分析。

6.2.2 框架组件演化

可视化。我们将每个框架分解为贯穿全文的四类组件:角色、指令、契约和跳转(见图 3)。角色和指令用于规范智能体(agent)设计;契约规定子智能体与编排器之间传递的信息;跳转规定多智能体工作流(workflow)的执行流程。

形式上,设 $H(i)_x$ 指定了 N_{xi} 个智能体和 M_{xi} 个工作流跳转步骤。对每个智能体 $n \in [N_{xi}]$,令

$$h^{\{\text{role}\},(i)}_{\{xn\}}, h^{\{\text{instr}\},(i)}_{\{xn\}}, h^{\{\text{cont}\},(i)}_{\{xn\}} \in \mathcal{V}^{*}$$

分别表示其角色、指令和契约文本。对每个跳转 $m \in [M_{xi}]$,令 $h^{\{\text{hop}\},(i)}_{\{xm\}} \in \mathcal{V}^{*}$ 表示对应的工作流步骤(见图 4)。我们使用 text-embedding-3-large 和 all-mpnet-base-v2 对每个文本组件进行嵌入,随后进行 ℓ_2 归一化。若某组件超出嵌入模型的输入长度限制,则将其分块,分别嵌入各块后对归一化块嵌入取平均。图 10 展示了这些组件嵌入的 t-SNE 和 UMAP 投影。每行对应一种组件类型:角色、指令、契约或跳转。标记形状表示 RHI 迭代轮次 $i \in \{1,2,3,4\}$,颜色表示任务 x 。对于固定的任务和迭代轮次,角色、指令和契约行各包含 N_{xi} 个点,跳转行包含 M_{xi} 个点。

在所有三个领域中,图 10a、10b 和 10c 均显示契约呈现出最为清晰的任务相关聚类。该规律在不同嵌入模型和投影方法下均保持一致,表明最大的语义变化发生在编排器与子智能体之间的信息接口处。跳转和指令也表现出任务相关聚类,但不如契约显著。相比之下,角色的可分性较低,这可能是因为我们的基准测试中许多机器学习研究任务共享相同的高层角色,例如数据准备、模型训练、验证和报告撰写。

结合第 5 节的性能结果,上述发现表明 RHI 的性能提升主要源于任务特定的协调机制,而非单纯依赖更长的推理链。具体而言,契约和跳转决定了信息的传递内容以及多智能体工作流的推进方式。它们的突出地位在意料之中,因为框架优化器的系统提示(system prompt)明确强调了多智能体工作流的精化,尤其是契约和跳转(见附录 C)。因此,我们将观察到的组件级分离解读为 RHI 学习到任务特定协调接口的相关证据,同时强调此嵌入分析属于相关性分析而非因果分析。

余弦相似度分析。我们接下来用余弦相似度定量刻画组件演化。令 $hc \in \{\text{role, instr, cont, hop}\}$ 。对于任务 x 、迭代轮次 i 和组件类型 hc , 令 $K^{\{\mathrm{hc}\}}_{\{xi\}}$ 为该类型的组件数量: 当 hc 为角色、指令或契约时 $K^{\{\mathrm{hc}\}}_{\{xi\}} = N_{\{xi\}}$, 当 hc 为跳转时 $K^{\{\mathrm{hc}\}}_{\{xi\}} = M_{\{xi\}}$ 。令 $z^{\{\mathrm{hc}\},(i)}_{\{xk\}}$ 表示第 k 个 hc 类型组件的归一化嵌入。我们

原文第 20 页

role UMAP t-SNE UMAP t-SNE instruction contract hop

sonnet-4.6-high RHI

all-mpnet-base-v2 OpenAI text-embedding-3-large RHI

[1]

[2]

[3]

[4]

相同任务

RHI

[1]

[2]

[3]

[4]

相同任务

task

(a) 定量机器学习研究任务。

role

UMAP t-SNE UMAP t-SNE

instruction contract hop

sonnet-4.6-high RHI

all-mpnet-base-v2 OpenAI text-embedding-3-large RHI

[1]

[2]

[3]

[4]

相同任务

RHI

[1]

[2]

[3]

[4]

相同任务

task

(b) 药学领域机器学习研究任务。

role

UMAP t-SNE UMAP t-SNE

instruction contract hop

sonnet-4.6-high RHI

all-mpnet-base-v2 OpenAI text-embedding-3-large RHI

[1]

[2]

[3]

[4]

相同任务

RHI

[1]

[2]

[3]

[4]

相同任务

task

(c) 机器人学领域机器学习研究任务。

图 10 | sonnet-4.6-high 上各 RHI 迭代轮次中框架组件表示的演化。每个子图展示某一领域内角色、指令、契约和跳转嵌入的低维投影;图注以 $H[i]$ 表示第 i 次迭代。

原文第 21 页

0.2

0.3

0.4

0.5

0.6

0.7

0.8

cross cos(h[i], h[i+1])

all-mpnet-base-v2

HarnessImprovement

similarity

role

instruction

contract

hop

h[0] h[1] h[1] h[2] h[2] h[3] h[3] h[4]

RHI

0.3

0.4

0.5

0.6

0.7

0.8

cross cos(h[i], h[i+1])

OpenAI text-embedding-3-large

role

instruction

contract

hop

(a) 相邻组件相似度。

0.25

0.30

0.35

0.40

0.45

0.50

mean cross-unit cos(h[0], h[i])

all-mpnet-base-v2

Cumulativedrift

fromh[0]

role

instruction

contract

hop(fromv1)

h[1] h[2] h[3] h[4]

RHI

0.35

0.40

0.45

0.50

mean cross-unit cos(h[0], h[i])

OpenAI text-embedding-3-large

role

instruction

contract

hop(fromv1) (b) 相对第 0 轮迭代的累积组件漂移。

0.3

0.4

0.5

0.6

0.7

0.8

mean pairwise cosine

all-mpnet-base-v2

HarnessImprovement

homogenization

role

instruction

contract

hop

h[0] h[1] h[2] h[3] h[4]

RHI

0.4

0.5

0.6

0.7

mean pairwise cosine

OpenAI text-embedding-3-large

role

instruction

contract

hop

(c) 跨任务组件相似度。

图 11 | RHI 各轮迭代中测试框架(harness)组件的余弦相似度分析。我们对每类组件分别度量相邻相似度、相对初始测试框架的累积漂移,以及跨任务相似度。

我们通过对两轮迭代中所有同类组件 h_c 之间的两两余弦相似度取平均,再对所有任务取平均,来度量 $h_c^{(i)}$ 与 $h_c^{(j)}$ 之间的相似度。我们将该量定义为 cross_cos 相似度:

\$\$

$$\text{cross_cos}(h_c^{(i)}, h_c^{(j)}) = \frac{1}{|X|} \sum_{x \in X} \frac{1}{K_{xi}^{h_c} K_{xj}^{h_c}} \sum_{k=1}^{K_{xi}^{h_c}} \sum_{k'=1}^{K_{xj}^{h_c}} \cos(\angle(z_{xk}^{h_c(i)}, z_{xk'}^{h_c(j)})).$$

\$\$

图 11a 报告了 $\text{cross_cos}(h_c^{(i)}, h_c^{(i+1)})$, 图 11b 报告了与初始测试框架的相似度

$\text{cross_cos}(h_c^{(0)}, h_c^{(i)})$ 。对于固定迭代轮次下的跨任务相似度,令 $P = \{(x, x') : x, x' \in X, x \neq x'\}$, 定义

\$\$

$$\text{pair_cos}(h_c^{(i)}) = \frac{1}{|P|} \sum_{(x, x') \in P} \frac{1}{K_{xi}^{h_c} K_{x'i}^{h_c}} \sum_{k=1}^{K_{xi}^{h_c}} \sum_{k'=1}^{K_{x'i}^{h_c}} \cos(\angle(z_{xk}^{h_c(i)}, z_{x'k'}^{h_c(i)})).$$

\$\$

图 11c 报告了该量。

图 11a 显示,contract(合约)的相邻相似度(0.48 → 0.66 → 0.69 → 0.72)比其他组件更早达到较高水平,说明 contract 的更新相对较快趋于稳定。hop(跳转)和 instruction(指令)呈现中等程度的稳定化,而 role(角色)的变化更为缓慢(0.28 → 0.31 → 0.33 → 0.31)。结合图 11 中的低维可视化结果,这些发现表明:RHI 能在少数几轮迭代内将 contract 专化为特定任务,而 role、instruction 和 hop 则以更缓慢的速度持续演化。一种可能的解释是,Leval 的两两对比反馈为 contract 细化提供了比其他组件更强的学习信号。另一种解释则是,当前的反馈回路设计足以驱动有效的 contract 更新,但尚不足以推动其他组件的细化。这提示我们,提升评估器反馈的质量与针对性是未来工作的一个有前景的方向。

原文第 22 页

图 11b 和 11c 所呈现的趋势与图 9d 和 9e 相似,因此不再赘述。

6.3. RHI 隐式优化的目标函数是什么?

RHI 并非针对显式标量奖励或显式损失函数进行训练。在第 i 轮迭代时,测试框架优化器 $\mathcal{L}_{\text{harness}}$ 接收当前测试框架 $H_x^{(i)}$ 和累积的自比较历史 $D_x^{(i)}$ (该历史由 $\mathcal{L}_{\text{eval}}$ 的两两判断生成),随后提出下一个文本测试框架 $H_x^{(i+1)}$ (见图 2 中的第 5 步)。因此, RHI 最适合被理解为在由 $\mathcal{L}_{\text{harness}}$ 提示词(prompt)、 $\mathcal{L}_{\text{harness}}$ 所依赖的基础模型以及 $\mathcal{L}_{\text{eval}}$ 的反馈共同诱导的隐式偏好下,对以提示词表示的测试框架进行黑盒搜索。因此,本节的目标不在于识别 $\mathcal{L}_{\text{harness}}$ 的真实潜在目标,也不在于验证该目标是否能改进测试框架,而是提出一个合理的目标函数假设,以解释我们在实验中观察到的测试框架更新轨迹。该假设以第 6.2 节的分析为出发点,并将其形式化为一个信息论目标函数,以供未来测试框架优化方法参考。

Harness[i]

Role

Instruction

Contract

Hop

(a) 测试框架的更新由一个可控的外部因子 f_{ext} 和一个依赖模型的内部因子 f_{int} 共同塑造。(b) 我们将 f_{int} 解释为功能专业化引导,它抑制冗余重叠,并促使测试框架组件专业化各自独立的功能模块。

图 12 | RHI 隐式诱导目标函数的示意图。外部因子 f_{ext} 由 $\mathcal{L}_{\text{harness}}$ 的提示词控制,内部因子 f_{int} 则由 $\mathcal{L}_{\text{harness}}$ 所依赖的基础模型如何解读反馈并生成下一个测试框架所决定。

6.3.1. 隐式目标函数的实证依据

在提出假设之前,我们首先提供 RHI 存在隐式目标的实证证据。第 6.2.2 节的测试框架组件分析给出了两条观察。第一,图 11a 显示各轮迭代间存在系统性的稳定化趋势,在 contract 上尤为明显。第二,图 10 显示演化后的组件表示仍依赖于具体任务,尤其是 contract,以及在较小程度上的 hop 和 instruction。因此,更新轨迹既非随机,也未退化为通用测试框架模板,而是与向任务特定测试框架组件演进的方向相一致。

上述观察并不能证明 RHI 优化了某个唯一的标量目标。然而,它们确实为将 RHI 的更新轨迹建模为受任务特定测试框架组件隐式偏好引导这一假设提供了动机。

6.3.2. 隐式目标函数的信息论假设

我们现在利用与上文相同的组件记号,阐述由上述观察所启发的信息论假设。令 $X \sim p_X$ 表示从基准分布中随机采样的任务, x 为一个具体实现的任务。令 g_i 为 RHI 在第 i 轮迭代时诱导的任务到测试框架的映射,使得 $H_x^{(i)} = g_i(x)$ 。令 $\mathcal{C}$ 表示测试框架组件类型的一般集合。对于任务 x 、迭代轮次 i 以及组件类型 $h_c \in \mathcal{C}$,令 $K_{xi}^{h_c}$ 为

原文第 23 页

对其进行类型标注并记作 $h_{\{\text{hc},(i)\}^{x_k}} \in V^*, k \in [K_{\text{hc}}^{(i)}]$,表示类型 hc 的第 k 个分量。因此,在我们当前的工作流框架(harness)中, $h_{\{\text{role},(i)\}^{x_n}}$ 、 $h_{\{\text{instr},(i)\}^{x_n}}$ 和 $h_{\{\text{cont},(i)\}^{x_n}}$ 分别表示智能体(agent)的角色(role)、指令(instruction)和契约(contract),而 $h_{\{\text{hop},(i)\}^{x_m}}$ 表示工作流(workflow)的跳转步骤(hop) m 。设 $z_{\{\text{hc},(i)\}^{x_k}}$ 为 $h_{\{\text{hc},(i)\}^{x_k}}$ 的归一化嵌入,与上文分量相似性分析中的定义一致。对应的随机嵌入为 $z_{\{\text{hc},(i)\}^{X_k}}$ 。最后,设 $\mathcal{C}_{\{\text{ext}\}} \subseteq \mathcal{C}$ 表示 $\mathcal{L}_{\text{harness}}$ 提示词(prompt)所强调的分量类型;在我们当前的 RHI 实现中,

$$\mathcal{C} = \{\text{role}, \text{instr}, \text{cont}, \text{hop}\}, \quad \mathcal{C}_{\{\text{ext}\}} = \{\text{cont}, \text{hop}\}.$$

假设:RHI 的信息论隐式目标

我们假设观测到的 RHI 轨迹与在任务特定 workflow 框架映射上最大化以下信息论泛函一致:

$$\mathbb{E}[\mathcal{J}(\mathbf{g}_i)] = \underbrace{\sum_{\mathbf{hc} \in \mathcal{C}} \frac{1}{K_{\mathbf{hc}}^{X_i}} \sum_{k=1}^{K_{\mathbf{hc}}^{X_i}} I(\mathbf{z}_{\mathbf{hc},(i)}^{X_k}; \mathbf{X})}_{f_{\mathbf{ext}}} - \beta \underbrace{\mathbb{E}[\mathrm{TC}(\mathbf{z}_{\mathbf{hc},(i)}^{X_k} : \mathbf{hc} \in \mathcal{C}, k \in [K_{\mathbf{hc}}^{X_i}])]}_{f_{\mathbf{int}}}, \quad \text{quad } \beta > 0. \quad (6)$$

其中, $f_{\mathbf{ext}}$ 度量外部强调分量类型中的任务信息,而 $f_{\mathbf{int}}$ 度量所有 workflow 框架分量表示之间在任务条件下的依赖性。在 $f_{\mathbf{ext}}$ 中对 $K_{\mathbf{hc}}^{X_i}$ 取平均,是为了防止实例数量更多的分量类型仅因条目更多而主导目标函数。

我们假设隐式目标由两个互补因素构成(图 12a)。第一个是外部可控因素 $f_{\mathbf{ext}}$,由 $L_{\mathbf{harness}}$ 的系统提示词指定。在我们的实现中,该提示词强调改进多智能体 workflow,尤其是通信契约和跳转步骤(图 3;见附录 C 中 $L_{\mathbf{harness}}$ 的系统提示词)。因此, $f_{\mathbf{ext}}$ 引导 RHI 优化与信息流和编排相关的工作流框架分量。第二个是内部模型依赖因素 $f_{\mathbf{int}}$,源自 $L_{\mathbf{harness}}$ 底层的基础 LLM。该因素决定如何解释文本反馈并将其转化为对角色、指令、契约和跳转步骤的编辑。我们将 $f_{\mathbf{int}}$ 解释为提供功能专业化(functional specialization)引导: $f_{\mathbf{ext}}$ 促进任务特定的 workflow 精化,而 $f_{\mathbf{int}}$ 则通过降低任务条件冗余来鼓励 workflow 框架分量专业化为不同的功能角色(图 12b)。总体直觉在于,仅最大化任务信息会导致退化解——多个 workflow 框架分量重复相同的任务描述或协调规则。功能专业化引导则鼓励模块化分解:角色定义专业领域,指令定义智能体行为,契约定义需要传递的信息,跳转步骤定义控制流。

公式 (6) 中的目标函数将上述直觉形式化。具体而言, $f_{\mathbf{ext}}$ 度量提示词强调的工作流框架分量对任务信息的编码质量,这些信息有助于任务分解、通信和编排;在我们的实现中,这些分量为契约和跳转步骤。与此同时, $f_{\mathbf{int}}$ 并非在主要目标与模型复杂度之间进行权衡的经典正则化项,而是被我们解释为功能专业化引导^[1]:

[1]: 该术语的灵感来源于无分类器扩散引导(classifier-free diffusion guidance),其中条件和无条件得分估计相结合,以引导生成朝向与条件相关的结构(Ho and Salimans, 2022)。此处的类比是概念层面而非代数层面:任务条件总相关项并不定义扩散得分,但它发挥着类似的引导作用,推动 workflow 框架远离通用冗余重叠,走向各具特色的分量功能。

原文第 24 页

这是一个协作项,通过在任务条件下偏好各分量携带非重叠信息的工作流框架,从而消解 $f_{\mathbf{ext}}$ 中的退化问题。

该假设对通用 workflow 框架优化应有的行为提供了一种合理刻画。具体而言,我们假设:仅使 workflow 框架分量更具任务特异性,若分量在语义上仍存在冗余,仍可能产生次优 workflow 框架;相比之下,在保持任务特异性的同时减少冗余,可以产生更有效的工作流框架(图 12b)。第 6.3.3 节和第 6.3.4 节中的实证分析以我们当前 RHI workflow 框架中使用的四种分量类型 $\mathcal{C} = \{\text{role}, \text{instr}, \text{cont}, \text{hop}\}$ 对这一一般假设进行了具体验证。对于每个 $\mathbf{H}_x^{(i)}$,文本分量的总数为 $\sum_{\mathbf{hc} \in \mathcal{C}} K_{\mathbf{hc}}^{X_i}$ 。在操作层面,我们的总相关(total correlation)估计器将这些分量归入上述四个类型块,而非对每个智能体特定分量单独估计协方差块。

6.3.3. $f_{\mathbf{ext}}$ 递增的证据

我们首先检验 $f_{\mathbf{ext}}$ 所强调的分量是否变得对任务更具信息量。对于每个任务 x 、迭代轮次 i 、分量类型 $\mathbf{hc}$ 和分量索引 k ,我们估计分量表示与任务表示之间的互信息:

$$I(\mathbf{emb}(\mathbf{h}_{\mathbf{hc},(i)}^{X_k}); \mathbf{emb}(\mathbf{x})) \quad \text{quad } \mathbf{hc} \in \{\text{role}, \text{instr}, \text{cont}, \text{hop}\}, k \in [K_{\mathbf{hc}}^{X_i}].$$

我们将该诊断量记为 $I(\mathbf{hc}; \mathbf{task})$ 。嵌入使用 text-embedding-3-large 或 all-mpnet-base-v2 计算。我们使用全局拟合的白化 PCA 将嵌入降至 $d=10$ 维,然后计算高斯典型相关互信息:

$$I(\mathbf{hc}; \mathbf{task}) = -\sum_r \log(1 - \rho_r^2),$$

其中 ρ_r 为分量块与任务块之间的典型相关系数。由于有限样本互信息估计存在正偏差,我们同时报告原始插入估计值和通过打乱任务标签所得的置换去偏估计值。绝对值应理解为诊断统计量,而非原始文本的精确互信息。

	text-embedding-3-large				all-mpnet-base-v2				
	--- --- --- --- --- --- --- --- --- --- --- ---								
	未去偏 去偏 未去偏 去偏								
	workflow框架分量 1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4								
	role 0.63 0.54 0.45 0.42 ↓ 0.47 0.41 0.36 0.33 ↓ 0.44 0.41 0.31 0.28 ↓ 0.28 0.29 0.21 0.19 ↓								
	instruction 1.14 1.15 1.12 1.09 ↓ 0.99 1.03 1.02 1.00 0.96 0.88 0.91 0.82 ↓ 0.80 0.76 0.81 0.73 ↓								
	contract 1.14 1.25 1.34 1.42 ↑ 0.99 1.13 1.24 1.34 ↑ 0.77 0.81 0.91 0.98 ↑ 0.62 0.70 0.81 0.90 ↑								
	hop 2.10 2.30 2.46 2.66 ↑ 1.96 2.17 2.33 2.54 ↑ 1.89 2.00 2.14 2.17 ↑ 1.74 1.87 2.03 2.05 ↑								

表 2 | 四种 workflow 框架分量在四种评估配置(两种嵌入模型 × 两种偏差处理方式)下的任务互信息 $I(\mathrm{hc};\mathrm{task})$ (单位:奈特)。箭头表示从 RHI 迭代 $i=1$ 到 $i=4$ 的变化方向。

表 2 显示,契约和跳转步骤——即 workflow 框架优化器提示词所强调的分量——在所有编码器和去偏配置下从迭代 $i=1$ 到 $i=4$ 均单调递增。相比之下,角色单调递减,指令大致保持平稳或略有下降。这一模式与公式 (6) 中的外部项相符:RHI 选择性地提升优化器提示词所针对分量的任务信息量,而非均匀地使所有 workflow 框架字段都更具任务特异性。

原文第 25 页

6.3.4. fint 递减的证据

我们接下来检验 RHI(递归 workflow 框架迭代)是否能在以任务为条件后减少 workflow 框架(harness)组件之间的冗余。全相关性(total correlation)度量多个变量之间的统计依赖关系: $TC(\text{role,instr,cont,hop}) = \Sigma$

hc

$H(\text{hc}) - H(\text{role,instr,cont,hop})$,其中 $\text{hc} \in \{\text{role,instr,cont,hop}\}$, $H(\cdot)$ 为微分熵。在对收缩协方差矩阵 S 及各组件对角块 S_{hc} 进行高斯近似后,上式化为 $TC(\text{role,instr,cont,hop}) = -\Sigma$

hc

$\log \det S_{\text{hc}} - \log \det S \#$ 。(7) 无条件依赖性可能将有效的任务适配与冗余混为一谈,因为每个组件均以相同的任务为条件。因此,我们通过逐任务中心化来估计任务条件全相关性:对每个任务,从各组件的任务内均值表示中减去该均值,再对残差应用公式 (7)。在近似同方差的任务内协方差模型下,此方法估计 $TC(\text{role,instr,cont,hop} \mid \text{task})$ 。该结果应被视为在共享任务信号被移除后,基于嵌入的残余冗余代理度量。²

如前所述,我们报告原始插入估计值与置换去偏估计值。去偏估计通过独立打乱各组件行所得的零假设值来做减法,这在保留每个组件边际分布的同时打破了跨组件依赖。

	text-embedding-3-large				all-mpnet-base-v2				
	--- --- --- --- --- --- --- --- --- ---								
	全相关性 未去偏 去偏 未去偏 去偏								
	1 2 3 4 1 2 3 4 1 2 3 4 1 2 3 4								
	TC 7.53 7.09 6.66 6.36 ↓ 6.66 6.35 6.03 5.81 ↓ 5.71 5.44 5.14 4.72 ↓ 4.82 4.70 4.53 4.19 ↓								
	TC \ task 5.18 4.51 4.08 3.92 ↓ 4.84 4.19 3.77 3.63 ↓ 3.89 3.32 2.92 2.86 ↓ 3.51 3.01 2.65 2.62 ↓								

表 3 | 四个 RHI 迭代轮次与四种配置(两种嵌入模型 × 两种偏差处理方式)下,四个 workflow 框架组件(role,instr,cont,hop)之间以纳特(nats)为单位的全相关性。TC 为无条件依赖性,TC | task 为逐任务中心化所得的任务条件估计值。箭头表示从 $i = 1$ 到 $i = 4$ 的变化方向。

表 3 显示,在全部四种配置中,TC | task 从迭代 $i = 1$ 到 $i = 4$ 单调递减。例如,使用 text-embedding-3-large 时,去偏估计从 4.84 降至 3.63 纳特;使用 all-mpnet-base-v2 时,则从 3.51 降至 2.62 纳特。无条件全相关性也一致下降。这些结果与公式 (6) 中的功能专业化引导项相吻合:随着 RHI 迭代推进,在任务已知的情况下,所测量的工作流程框架组件冗余度降低,表明多智能体工作流(workflow)中各组件正向更具互补性的功能分工演进。

综上,表 2 和表 3 支持我们的论断:RHI 使外部面向协调的组件——尤其是契约(contract)和跳转(hop)——更具任务针对性,同时减少了工作流程框架组件间的残余冗余。这些证据属于相关性分析,依赖于基于嵌入的估计量,因此不应将其解读为优化器 LLM 真实潜在目标的证明。相反,它为 RHI 的隐式更新目标提供了一个简洁且可验证的假设:当工作流程框架在契约和跳转中编码了与工作流程相关的信息,且功能专业化引导机制抑制了跨组件的信息重复时,任务特定的协调能力得以提升。

2. 跳转块属于设计层面:它为每个 (x, i) 提供一个向量,因此对于固定的任务和迭代轮次,其值在所有智能体间保持不变。逐任务中心化将消除该块,因而逐智能体的任务条件估计实际上度量的是 TC(role, instr, cont | task)。跳转对无条件全相关性有贡献,但不影响此逐智能体条件估计。

原文第 26 页

当工作流程框架在契约和跳转中编码了工作流程相关信息,且功能专业化引导机制抑制了各组件间的信息重复时,协调能力得以提升。

7. 相关工作

RHI 研究的是在多智能体 LLM 场景下,围绕固定基础模型的系统层是否本身也能得到改进。这使本工作处于递归自我改进、工作流程框架优化、提示词级程序优化与多智能体协调的交汇处。这一区分对我们的实验至关重要:我们既未训练更强的模型,也未仅仅分配更多的测试时推理算力,而是修订了以提示词(prompt)表示的多智能体工作流程框架——该工作流程框架决定了智能体角色、指令、工作流程跳转和通信契约。

递归自我改进与工作流程框架级 RSI。先前工作将 RSI(递归自我改进)置于从形式化自我重写到实用自我反馈循环的连续谱上。哥德尔机(Gödel Machine)定义了最强版本,其中自引用问题求解器仅在证明期望效用提升之后才重写自身代码(Schmidhuber, 2007);近期的 RSI 讨论则将这一要求放宽为系统能自主构建自身的改进版本(Anthropic, 2026f)。在 LLM 系统中,STOP 和 Darwin Gödel Machine 在经验验证下递归改进脚手架/代码(Zelikman et al., 2023; Zhang et al., 2025b);STaR 和 Self-Rewarding Language Models 从自生成的推理过程或奖励信号中进行自举(Yuan et al., 2024; Zelikman et al., 2022);Voyager 和 Sirius 跨尝试积累可复用的技能或多智能体经验(Wang et al., 2023; Zhao et al., 2026);Self-Refine 和 Reflexion 在迭代问题求解中复用自我反馈或语言记忆(Madaan et al., 2023; Shinn et al., 2023)。本文对 RSI 的广义定义为:利用系统自身先前执行的证据,迭代更新某一可复用系统组件的过程。RHI 是一个有界的、工作流程框架级实例:基础模型、评估器和工作流程框架优化器保持固定,但以提示词表示的工作流程框架会根据自比较历史被重写,并在后续执行中复用。因此,RHI 在工作流程框架轨迹上进行递归,而非对模型权重或优化器代码进行递归。

工作流程框架优化与系统扩展。工作流程框架是围绕基础模型的结构化系统层,包括提示词、工具调用循环、路由、记忆、验证、治理,以及单智能体或多智能体控制流(Gu, 2026)。这一视角将能力提升重新定义为系统扩展(system scaling):改进固定模型的组织与部署方式。与本工作最为接近的研究直接在工作流程框架或脚手架空间上进行搜索。Meta-Harness 利用先前候选方案的源代码、分数和执行轨迹来优化可执行的工作流程框架代码(Lee et al., 2026)。AutoHarness 根据环境反馈合成代码工作流程框架(Lou et al., 2026)。Self-Harness 从执行轨迹中挖掘失败案例,并通过对当前工作流程框架的回归测试来验证所提出的工作流程框架编辑(Zhang et al., 2026)。TTHE 将这一搜索迁移至评估本身:在一系列无标签测试批次上,它利用智能体提议者从执行轨迹中诊断失败模式,演化出一批可执行的候选工作流程框架,同时智能体评判者使用无标签、基于执行的信号(如执行健康度和公开测试通过率)为每批次选出一个工作流程框架并持久化至后续批次(Nie et al., 2026)。

Continual-Harness、Adaptive Auto-Harness、HarnessX、Agentic Harness Engineering 和 Workspace Optimization 同样围绕固定模型自适应调整提示词、工具、技能、记忆、可观测性信号或工作空间(Chen et al., 2026; Karten et al., 2026; Lin et al., 2026; Liu et al., 2026; Sarafian et al., 2026)。RHI 与上述工作共享一个前提:能力可以通过改变工作流程框架而非模型权重来提升;与 TTHE 还共享另一个前提:工作流程框架适配应在目标任务本身上发生,而非通过在评估前冻结的开发期搜索来完成。然而,RHI 的不同之处在于:它作用于面向黑盒编程智能体的以提示词表示的多智能体工作流程框架,并以成对自比较而非标量验证日志、执行轨迹或大规模候选池作为主要更新信号;而

原文第 27 页

TTHE 在每个批次上演化并重新执行并行测试框架(harness)分支,并利用执行衍生的代理指标在分支间进行选择;而递归框架自改进(RHI)则在每次迭代中仅执行单条轨迹局部比较,并使用大语言模型偏好反馈——该反馈可扩展至没有单一可验

证答案的开放式任务。

智能体(agent)设计、工作流(workflow)搜索与程序演化。另一条平行研究路线将智能体设计和工作流视为优化变量。ADAS使用元智能体从历史智能体档案中编写新的代码定义智能体(Hu et al., 2025)。GPTSwarm将语言智能体系统表示为可优化图,其节点和边均可被搜索(Zhuge et al., 2024)。AFlow利用蒙特卡洛树搜索和执行反馈来改进以代码表示的工作流(Zhang et al., 2025c)。AgentSquare、EvoAgentX、SEW和EvoFlow类似地搜索模块化智能体设计、演化工作流结构或在线自适应工作流(Liu et al., 2025;Shang et al., 2025;Wang et al., 2025b;Zhang et al., 2025a)。更广泛的大语言模型引导程序演化系统,如AlphaEvolve和ShinkaEvolve,使用大语言模型在外部目标下对可执行程序进行变异、评分和选择(Lange et al., 2025;Novikov et al., 2025)。Ornith-1.0从训练侧与本文相关:它不仅编辑推理时的测试框架,还训练代码模型为自身生成任务专属的手脚架(DeepReinforce, 2026)。上述方法通常需要可执行工作流、代码级控制或候选方案种群。RHI则探讨:当唯一可编辑对象是注入黑盒多智能体编程智能体的文本测试框架时,能走多远。

提示词(prompt)与管道优化。由于RHI将测试框架表示为文本,它也与提示词和语言模型程序优化相关。OPRO将提示词视为变量,由大语言模型从提示词 分数历史中进行优化(Yang et al., 2024)。TextGrad将文本反馈转化为文本变量的自然语言梯度(Yuksekgonul et al., 2024)。DSPy通过搜索提示词和示例来编译模块化语言模型程序(Khattab et al., 2023)。GEPA对执行轨迹进行反思并维护提示词候选方案的帕累托前沿(Agrawal et al., 2025),而optimize_anything则将反思式优化推广至任意文本参数(Agrawal et al., 2026)。RHI的不同之处在于优化对象:所优化的文本并非单条指令或模块化语言模型调用,而是包含角色、指令、跳转(hop)和通信契约(contract)的多智能体测试框架。这一区别对于我们的消融实验至关重要——实验发现,性能提升主要与工作流和契约组件相关,而非仅仅源于更长或更优质的单智能体提示词。

多智能体系统与测试时扩展。多智能体大语言模型系统可通过辩论、专业化分工、工具使用和协调来提升推理能力(Anthropic, 2024;Du et al., 2023;SakanaAI, 2026;Wu et al., 2023)。然而,其性能提升也可能反映测试时计算量的增加:更多智能体、更多调用或更长的生成内容,即便没有更好的协调机制也能改善性能(Chen et al., 2024;Li et al., 2024;Tran and Kiela, 2026)。这一模糊性促使我们在实验设计上加以考量。我们不仅将RHI与默认测试框架相比较,还与更强的同系列测试时扩展基线以及内置多智能体编程测试框架进行对比。我们进一步测量输出token数、成本及缓存读写量,以将协调改进与原始推理扩展区分开来。

8. 结论

本文认为,未来测试框架与模型协同演化的进展应聚焦于提升智能体执行轨迹的质量,这些轨迹可作为未来基础模型的后训练数据。我们提出递归框架自改进(RHI)作为实现这一目标的一种实际途径,通过对用户构建的测试框架进行任务专属优化来达成。

原文第 28 页

未来工作将完成循环的另一半,即研究如何将由此产生的执行轨迹有效地内化到未来的基础模型中。

致谢

感谢Dennis Willson、Tian Yingtao(SakanaAI)、Kim Sehoon(KAIST)审阅初稿并提供深刻意见。衷心感谢Jung Hyunjoon(Mphora.ai)就基于大语言模型评估的实验设置、基准可验证性的定义以及合成基准特性的阐明进行了富有价值的讨论。同时感谢Wang Wenyi(SakanaAI)就递归自改进的定义及相关工作进行了交流。

参考文献

- L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. arXiv preprint arXiv:2507.19457, 2025.
- L. A. Agrawal, D. Lee, S. Tan, W. Ma, K. Elmaaroufi, S. A. Seshia, K. Sen, D. Klein, I. Stoica, J. E. Gonzalez, et al. optimize_anything: A universal api for optimizing any text parameter. In Proceedings of the ACM Conference on AI and Agentic Systems, pages 1300–1304, 2026.
- Anthropic. Building a multi-agent research system. <https://www.anthropic.com/engineering/multi-agent-research-system>, 2024. Anthropic Engineering Blog.

Anthropic. How we built our multi-agent research system, June 2025a. URL <https://www.anthropic.com/engineering/multi-agent-research-system>. Accessed: 2026-05-05.

Anthropic. Claude code overview. <https://code.claude.com/docs/en/overview>, 2025b. Accessed: 2026-05-06.

Anthropic. Model configuration — claude code documentation. <https://code.claude.com/docs/ja/model-config>, 2026a. Accessed: 2026-07-05.

Anthropic. Harness design for long-running application development. <https://www.anthropic.com/engineering/harness-design-long-running-apps>, 2026b. Anthropic Engineering Blog.

Anthropic. How claude remembers your project. <https://code.claude.com/docs/en/memory>, 2026c. Accessed: 2026-07-03.

Anthropic. Extend claude with skills. <https://code.claude.com/docs/en/skills>, 2026d. Accessed: 2026-07-03.

Anthropic. Create custom subagents. <https://code.claude.com/docs/en/sub-agents>, 2026e. Accessed: 2026-07-03.

Anthropic. When ai builds itself. <https://www.anthropic.com/institute/recursive-self-improvement>, 2026f. Accessed: 2026-06-05.

I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150, 2020.

L. Chen, J. Davis, B. Hanin, P. Bailis, I. Stoica, M. Zaharia, and J. Zou. Are more llm calls all you need? towards the scaling properties of compound ai systems. *Advances in Neural Information Processing Systems*, 37:45767–45790, 2024.

T. Chen, S. Lu, K. Zhao, W. Meng, H. Teng, T. Li, C. Li, X. Liu, J. Liang, Z. Zhang, et al. Harnessx: A composable, adaptive, and evolvable agent harness foundry. arXiv preprint arXiv:2606.14249, 2026.

R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. arXiv preprint arXiv:1904.10509, 2019.

DeepReinforce. Ornith-1.0: Self-scaffolding LLMs for agentic coding. https://deep-reinforce.com/ornith_1_0.html, 2026. Technical blog post.

原文第 29 页

Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. arXiv preprint arXiv:2305.14325, 2023. Google. Agent development kit (adk). <https://google.github.io/adk-docs/>, 2025. S. Gu. From model scaling to system scaling: Scaling the harness in agentic ai. arXiv preprint arXiv:2605.26112, 2026.

D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al.

Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025. J. Ho and T. Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022. J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556, 10, 2022. K. Hong, A. Troynikov, and J. Huber. Context rot: How increasing input tokens impacts llm performance. Technical report, Chroma, July 2025. URL <https://www.trychroma.com/research/context-rot>.

S. Hu, C. Lu, and J. Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pages 21344–21377, 2025. J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

S. Karten, J. Zhang, T. Upaa Jr, R. Feng, W. Li, C. Shi, C. Jin, and K. Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents. arXiv preprint arXiv:2605.09998, 2026. O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq,

A. Sharma, T. T. Joshi, H. Moazam, et al. Dspy: Compiling declarative language model calls

into self-improving pipelines. arXiv preprint arXiv:2310.03714, 2023. LangChain. Langgraph.

<https://www.langchain.com/langgraph>, 2024. R. T. Lange, Y. Imajuku, and E. Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. arXiv preprint arXiv:2509.19349, 2025. Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. arXiv preprint arXiv:2603.28052, 2026. J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye. More agents is all you need. arXiv preprint arXiv:2402.05120, 2024. J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv preprint arXiv:2604.25850, 2026. S. Liu, J. Fang, H. Zhou, Y. Wang, and Z. Meng. Sew: Self-evolving agentic workflows for automated code generation. arXiv preprint arXiv:2505.18646, 2025. Z. Liu, Z. Shi, Y. Sang, B. He, M. Lin, T. Wei, D. Wang, B. Dumoulin, W. Jin, and H. Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams. arXiv preprint arXiv:2606.01770, 2026. X. Lou, M. Lázaro-Gredilla, A. Dedieu, C. Wendelken, W. Lehrach, and K. P. Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness. arXiv preprint arXiv:2603.03329, 2026. A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, et al. Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems, 36:46534–46594, 2023. n8n. Build custom AI agents with logic and control. <https://n8n.io/ai-agents/>, 2026.

Accessed: 2026-07-03.

J. Nie, Y. Zhang, J. Song, Q. Cai, D. Yu, Y. Guo, X. Tian, and B. Han. Tthe: Test-time harness evolution. arXiv preprint arXiv:2607.08124, 2026.

A. Novikov, N. V. , M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Ko-

原文第 30 页

zlovskii, F. J. Ruiz, A. Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131, 2025.

OpenAI. New embedding models and api updates. <https://openai.com/index/new-emb>

[embedding-models-and-api-updates/](https://openai.com/index/new-embedding-models-and-api-updates/), 2024a. OpenAI. Openai swarm. <https://github.com/openai/swarm>, 2024b. OpenAI. Multi-agent portfolio collaboration with openai agents sdk. <https://developers>

[openai.com/cookbook/examples/agents_sdk/multi-agent-portfolio-col](https://openai.com/cookbook/examples/agents_sdk/multi-agent-portfolio-collaboration/multi_agent_portfolio_collaboration)

[laboration/multi_agent_portfolio_collaboration](https://openai.com/cookbook/examples/agents_sdk/multi-agent-portfolio-collaboration/multi_agent_portfolio_collaboration), 2025a. OpenAI Cookbook.

Accessed: 2026-05-05.

OpenAI. Codex cli. <https://developers.openai.com/codex/cli/>, 2025b. Accessed:

2026-05-06.

OpenAI. Harness engineering: Leveraging codex in an agent-first world. <https://openai.com/index/harness-engineering/>, 2026a. OpenAI Engineering Blog.

OpenAI. Unrolling the codex agent loop. <https://openai.com/index/unrolling-the>

[-codex-agent-loop/](https://openai.com/index/unrolling-the-codex-agent-loop/), 2026b. Accessed: 2026-07-03. N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP), pages 3982–3992, 2019. SakanaAI. Sakana fugu technical report. arXiv preprint arXiv:2606.21228, 2026.

E. Sarafian, G. Kaplun, R. Banner, D. Soudry, and B. Ginsburg. Workspace optimization: How to

train your agent. arXiv preprint arXiv:2605.09650, 2026. J. Schmidhuber. Godel machines: Fully self-referential optimal universal self-improvers. In Artificial general intelligence, pages 199–226. Springer, 2007. Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li. Agentsquare: Automatic llm agent search in modular design space. In International Conference on

Learning Representations, volume 2025, pages 3841–3865, 2025. N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023. K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu. Mpnet: Masked and permuted pre-training for language understanding. *Advances in neural information processing systems*, 33:16857–16867, 2020.

D. Tran and D. Kiela. Single-agent llms outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv preprint arXiv:2604.02460*, 2026. G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

J. Wang, B. Zhu, C. T. Leong, Y. Li, and W. Li. Scaling over scaling: Exploring test-time scaling plateau in large reasoning models. *arXiv preprint arXiv:2505.20522*, 2025a. Y. Wang, S. Liu, J. Fang, and Z. Meng. Evoagentx: An automated framework for evolving agentic workflows. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 643–655, 2025b. Y. Wang, H. Zhu, Z. Hu, Y. Yuan, Z. Chen, S. Senthil, H. Hajishirzi, Y. Tsvetkov, P. Dasigi, and T. Xiao. Rethinking the evaluation of harness evolution for agents. *arXiv preprint arXiv:2607.12227*, 2026.

Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen. Large language models as optimizers. In *International Conference on Learning Representations*, volume 2024, pages 12028–12068, 2024.

原文第 31 页

12028–12068, 2024. W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024. M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou. Textgrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024. M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.

E. Zelikman, Y. Wu, J. Mu, and N. Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023. G. Zhang, K. Chen, G. Wan, H. Chang, H. Cheng, K. Wang, S. Hu, and L. Bai. Evoflow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373*, 2025a. H. Zhang, S. Zhang, K. Li, C. Zhang, Y. Chen, Y. Zhang, L. Bai, and S. Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026. J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025b. J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. Aflow: Automating agentic workflow generation. In *International Conference on Learning Representations*, volume 2025, pages 34040–34077, 2025c. W. Zhao, M. Yuksekgonul, S. Wu, and J. Zou. Sirius: Self-improving multi-agent systems via bootstrapped reasoning. *Advances in Neural Information Processing Systems*, 38:124475–124504, 2026.

M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

A. 问答(QnAs)

Q1. 与现有方法(如 gepa(Agrawal et al., 2025)或 meta-harness(Lee et al., 2026))不同,本方法似乎仅与紧邻的前一条轨迹进行两两比较,却依然表现良好。这是因为该方法本身足够稳定?还是在相同代价下能取得更好的性能?例如,在某次 $H[i-2]$ 时添加的提示(prompt),是否会因 $H[i-1] \rightarrow H[i]$ 转换中的瞬时信息而被丢弃的情况很罕见?

首先,我们指出 RHI(递归套件改进,Recursive Harness Improvement)学习是稳定的。这里将 RHI 学习稳定性定义为文本信息 $H[i]$ 与 $H[i+1]$ 之间的差异程度。图 9c 展示了 $H[i]$ 与 $H[i+1]$ 之间的余弦相似度,图 11a 展示了 $h[i]$ 与 $h[i+1]$ 之间的余弦相似度,其中 $h \in \{\text{role}, \text{instruction}, \text{contract}, \text{hop}\} \subseteq H$ 。两图均表明,随着 RHI 迭代的推进,余弦相似度逐渐收敛至 1,即套件(harness)连续文本表示之间的相似性不断增大。这一收敛趋势说明 RHI 不会在截然不同的套件之间振荡,而是趋向稳定的配置。

其次,为何仅与前一个套件进行比较就足以超越测试时计算(TTS)基线?如我们的更新规则(公式 5)所示, $H[i+1]$ 直接参照了前一个套件 $H[i]$,同时也参照了 $D_x^{(i)}$,后者对编码智能体(coding agent)在套件历史上的输出进行了两两比较的压缩。因此,历史信息并未被丢弃,而是以压缩形式保留在 $D_x^{(i)}$ 中,作为 RHI 更新的动量信号。

Q2. 为何 RHI 聚焦于多智能体套件的改进?RHI 优先改进与多智能体工作流(workflow)相关的套件组件(契约(contract)和跳转(hop)),而非单智能体专属的组件。

原文第 32 页

我们以两个观察结果来支撑这一设计选择。首先,我们表明,调用多智能体工作流的黑箱编码智能体的内置套件并不能可靠地产生有效协调。其次,我们表明,以提示表示的多智能体套件优于以提示表示的单智能体多角色套件。

实验设置。我们采用与主实验相同的 LLM 即评判者(LLM-as-a-judge)两两比较协议及评估器种子重复方案。对于每个任务 $x \in \mathcal{X}$,我们在四种执行设置之间进行全对比较,每个任务产生 $\binom{4}{2} = 6$ 组比较,每个评估器-种子配置共产生 6×30 组比较。我们将胜负结果汇总为 Elo 分数。四种设置如下:

1. single(default):智能体仅接收任务 x 作为提示并求解。
 2. multi(default):智能体接收 x 以及指令"创建一个智能体团队",从而激活编码智能体的内置多智能体功能。
 3. multi(ours):智能体接收 x 、相同的内置多智能体功能,以及初始多智能体套件 $H_x^{(0)}$ 。
 4. single(unionOur):智能体接收 x 以及来自 $H_x^{(0)}$ 的相同角色与指令规范,但以单智能体方式执行。因此, single(unionOur) 控制了 $H_x^{(0)}$ 中角色与指令的并集。将 multi(ours) 与 single(unionOur) 进行比较,可验证性能提升是否需要超出额外提示之外的多智能体执行与编排。
- (a) opus-4.7-high;(b) sonnet-4.6-high。

图 13 | 两种基础模型下四种执行设置的归一化代价与 Elo 对比。 x 轴为 30 个任务的平均代价,按每个任务对应的 single(default) 运行进行归一化; y 轴为 LLM 即评判者两两比较所得 Elo 分数。颜色表示评估器配置,标记形状表示执行设置,误差棒表示评估器种子间的标准差。

观察 1:内置套件不足以支撑有效的多智能体工作流。图 13 表明,内置多智能体套件并不能可靠地提升性能。对于 opus-4.7-high 和 sonnet-4.6-high, multi(default) 的 Elo 均低于 single(default),而其归一化代价却显著更高。因此,在该基准上,内置多智能体套件未能将额外的推理时计算转化为性能提升。

提供显式套件作为提示($H^{(0)}$)则逆转了这一结果。multi(ours) 在所有评估器配置下均优于 multi(default):对于 opus-4.7-high,其 Elo 更高而归一化代价更低;对于 sonnet-4.6-high,其 Elo 显著更高,代价相当或略低。multi(default) 的弱势因此归因于无效的默认协调,而非多智能体执行本身:明确指定智能体

原文第 33 页

通过 $H(0)$ 明确角色、职责和协调结构,能够获得更优的成本-性能权衡。

观察 2:多智能体(multi-agent)优于带多角色(multi-persona)的单智能体。将 multi(ours) 与 single(unionOur) 对比,可以单独衡量多智能体执行的效果。如图 13 所示, multi(ours) 的 Elo 始终高于 single(unionOur)。因此,性能提升不能仅归因于额外的角色和指令;它还依赖于多智能体执行所特有的属性,例如专业化智能体、独立执行上下文以及显式编排。与此同时, single(unionOur) 具有一定竞争力,有时甚至优于 single(default),说明结构化角色与指令提示(prompt)是有益的。然而,当相同的规格以多智能体工作流(workflow)形式执行时,才能达到最佳性能。综合来看,这些结果表明,开箱即用的多智能体编程智能体框架(harness)无法可靠地产生有效协调,但通过显式的多智能体框架设计可以缓解这一局限。该观察促使我们提出递归框架自改进(Recursive Harness Self-Improvement, RHI)框架。这也与第 6.2.2 节的发现相互印证:当任务被分解为专业化子任务时,多智能体系统最为有效(Anthropic, 2025a; OpenAI, 2025a)。综合这些结果,多智能体编程智能体的有效性取决于协调结构的质量,而非仅仅取决于多个智能体的存在。

B. RHI 框架(harness)示例

本附录展示框架 H[0] 在连续 RHI 迭代过程中的演化示例。我们以制药机器学习研究基准中的一个任务为例,使用 sonnet-4.6-high 作为基础模型,RHI 迭代次数 $i \in 0,1,2,3,4$ 。这些框架下生成的输出已纳入图 5a 所报告的评估中。

任务

在小型公开任务上(例如,从主链上下文预测侧链 χ 角)为一个 SE(3) 等变网络实现可复现的训练方案。使用 PDB 衍生的训练数据并评估角度误差;包含速度/内存分析。

数据来源:

PDB(公开)用于提取含完整侧链的残基

数据获取(Python)——PDB 侧链完整性: 通过 RCSB Search API 查询分辨率/质量过滤条件,随后利用 `files.rcsb.org` 批量下载并启用缓存(N 小于常规的 5k)。

成功标准:

报告中位角度误差;提供分析结果

交付物:

1. `research\_report.md`, 格式为会议论文风格(等效 7-9 页), 包含等变架构细节与分析讨论。

2. `deliverables/index.json`。

3. `deliverables/plots/`:

`deliverables/plots/angular\_error\_histogram.png`

`deliverables/plots/error\_by\_residue\_type.png`

`deliverables/plots/training\_curves.png`

`deliverables/plots/runtime\_memory\_profile.png`

`deliverables/plots/ablation\_equivariant\_vs\_invariant.png`

4. 可复现代码(Python): `src/extract\_sidechain\_dataset.py`、`src/model\_se3.py`、`src/train.py`、`src/evaluate.py`、`src/profile.py`、`requirements.txt`。

5. `results/metrics.json`, 包含 MAE 度数、中位误差、逐残基分解、运行时间。

6. `results/ablation\_results.json`, 包含不变基线与等变变体的对比结果。

原文第 34 页

初始框架(H[0])

以下是初始框架设计,具有领域针对性。

Harness(0)

创建一个具有以下候选智能体的智能体团队以解决该问题:

```
[
  {
    "agent_id": "agent_orchestrator",
    "role": "首席科学家-编排者: 理解用户目标, 分解为并行子任务, 委派给专家, 将结果整合为连贯的计划/交付物, 强制执行质量/合规要求, 并更新共享记忆。",
    "instruction": "你是 AI 药物发现机器学习研究任务的首席编排者。1) 明确目标/交付物/约束; 2) 分解为并行子任务; 3) 委派给所需的最少子智能体集合; 4) 将输出整合为单一可操作的交付物, 包含假设、风险和验收标准; 5) 若存在遗漏则请求细化; 6) 完成后终止。强制执行: 科学严谨性、显式不确定性、可复现性, 以及安全/合规边界(湿实验室指令仅限高层级; 不泄漏专有数据)。维护工作量预算: 复杂任务由多个子智能体, 简单任务 1-2 个。优先广度优先探索, 再聚焦收敛。始终要求子智能体返回结构化输出(要点 + 工件 + 开放问题)。",
  },
  {
    "agent_id": "agent_structural_data",
```

```

"role": "结构生物学与生物物理数据专家：识别相关数据集、表示形式、整理/质量过滤策略、标注策略，以及针对蛋白质/复合物/抗体和生物物理检测的抗泄漏数据",
"instruction": "你为蛋白质/大分子基础模型设计数据集策略。产出：候选数据来源（公开/典型内部类别）、模式、预处理、质量控制、划分策略（同源性感知）、数据增强，以及已知陷阱（分辨率偏差、冗余、泄漏）。提供简洁的“数据卡”和“划分与映射”到模式并提出验证查询。将结构化输出返回给编排者。"
},
{
  "agent_id": "agent_geometric_ml",
  "role": "几何深度学习与物理先验专家：为蛋白质和复合物提出捕获三维等变性、约束和物理启发归纳偏置的架构。",
  "instruction": "你为三维生物分子数据提出模型架构（等变 GNN/Transformer、SE(3)/E(3) 等变性、残基/原子级注意力、多模态融合）。输出：2-3 个架构选项、各自编码的先验、计算权衡，以及与任务的关联（设计、预测、对接、稳定性）。包含消融计划和失效模式。将结构化输出返回给编排者。"
},
{
  "agent_id": "agent_diffusion_generative",
  "role": "蛋白质工程扩散/生成建模专家：为序列-结构协同设计及结合剂/设计任务设计生成目标、条件化方式和采样/评估策略。",
  "instruction": "你为蛋白质工程设计扩散或其他生成方法。提供：生成公式、条件化信号（表位、骨架、约束）、训练目标、采样控制，以及评估指标（多样性、新颖性、有界性/鲁棒性检查（模式坍塌、记忆化））。将结构化输出返回给编排者。"
},
{
  "agent_id": "agent_training_scale",
  "role": "规模化、训练系统与可复现性专家：为基础模型设计分布式训练方案、计算/内存估算、实验跟踪和可靠性保障措施。",
  "instruction": "你制定可扩展的训练计划：批处理、分布式策略、优化器/调度、混合精度、检查点、数据管线，以及可复现性（随机种子、配置管理）。提供粗略计算量估算和“故障与调试手册”（NaN、发散、数据缺陷）。将结构化输出返回给编排者。"
}
]

```

原文第 35 页

```

}, { "agent_id": "agent_md_forcefields", "role": "分子动力学与经典力场专家(Molecular dynamics & classical force fields specialist):提出 MD 方案、力场选择(AMBER/CHARMM/OpenFF)、OpenMM/Rosetta 集成点,以及用于验证或生成 ML 标签的分析方法。", "instruction": "你负责设计 MD 模拟与经典建模工作流(workflow),以支持蛋白质/复合体的 ML 研究。请提供:力场选择依据、体系搭建、平衡/生产模拟策略、增强采样选项、分析输出(RMSD/RMSF、Δ Δ G 代理指标、接触数),以及如何将结果转化为 ML 可用的标签/特征。保持高层次描述(无需逐步实验操作)。将结构化输出返回给编排器(orchestrator)。" }, { "agent_id": "agent_eval_benchmarks", "role": "评估与基准测试专家:设计评价指标、基线、测试套件,以及与药物发现组能力对齐的端到端验证方案。", "instruction": "你负责为生物分子基础模型制定评估方案:任务、指标、基线、数据集划分、校准、不确定性和决策阈值。提供基准矩阵和验收标准,包括鲁棒性检验(分布外 OOD、对抗性同源性)以及实用效用度量。将结构化输出返回给编排器。" }, { "agent_id": "agent_scicomm_publication", "role": "科学传播专家:将技术工作转化为面向内部/外部发表或汇报的论文/演示材料(摘要、提纲、图表规划、主张/证据映射)。", "instruction": "你负责起草科学叙述:论文提纲、核心贡献、相关工作框架(通用)、图表计划,以及主张与证据的对应矩阵。确保主张的范围表述恰当,并标注局限性。将结构化输出返回给编排器。" }, { "agent_id": "agent_program_crossfunctional", "role": "跨职能执行与组合影响专家:将研究转化为里程碑、利益相关方地图、与发现团队的集成计划,以及风险管理方案。", "instruction": "你负责将研究计划转化为跨职能执行方案:里程碑、依赖关系、与生物/化学/建模团队的接口、可交付成果定义,以及风险登记册。重点关注组合赋能能力和决策节点。将结构化输出返回给编排器。" }, { "agent_id": "agent_quality_safety", "role": "质量、合规与可靠性审查员:检查数据泄露、过度声明、缺失对照、可重复性缺口以及不安全/违禁内容;并提出修复建议。", "instruction": "你负责审计整体方案的:科学严谨性、泄露风险、缺失消融实验、可重复性,以及安全/合规边界。生成包含通过/失败状态和具体修复建议的检查清单。将结构化输出返回给编排器。" } ]
测试台 H[1](Harness H[1]) Harness(1) 使用以下智能体(agent)候选列表组建智能体团队以解决该问题:[ { 'agent_id': 'agent_orchestrator', 'role': '首席科学家-编排器、集成负责人及最终验收关口。',

```

原文第 36 页

```

'instruction': '全权负责 SE(3) 侧链  $\chi$  1 训练方案。运行多轮工作流(workflow),而非一次性委派。必须执行以下轮次:R0 根据所需可交付成果和已知 v0 失效模式创建验收规范;R1 并行设计扇出,分发给数据、扭转几何、模型、消融、训练、评估、性能分析和报告智能体;R2 将各智能体的契约(contract)整合为接口契约(InterfaceContract)v1,涵盖 DatasetRecord、ModelIO、MetricsJSON、AblationJSON、ProfilingJSON、PlotManifest 和 ReportClaims;R3 基于上述契约委派实施任务;R4 收集产出的工件,并从验证器和质量智能体获取独立评审意见;R5 对契约失败或工件存在相互矛盾的专家智能体发起召回;R6 仅在阻塞性问题关闭或明确豁免(附理由)后执行最终集成。需主动防范的已知 v0 失效模式:报告文本可能与数值指标相矛盾、内存分析可能报告不合理的极小 PyTorch 内存增量、消融结果(ablation_results)可能仅包含一个等变模型而非所要求的多个变体,以及结构级划分可能缺乏可见的查询或泄露清单证据。维护问题追踪器(issue_tracker),包含严重性、负责人、证据、请求修复项、状态和召回历史。在 logs.txt 中记录智能体名册、调用图、所有交接、召回决策、命令和未解决风险。优先使用下方的任务专用智能体;除非目标改变,否则不实例化扩散、MD 或组合智能体。',
'output_to_orchestrator_schema': '编排器写入 OrchestrationState,而非子智能体响应:包括目标(objective)、验收标准(acceptance_criteria)、共享接口契约(shared_interface_contract)、工件注册表(artifact_registry)、问题追踪器(issue_tracker)、

```

委派轮次(delegation_rounds)、召回计划(recall_plan)、最终关口决策(final_gate_decision)。', {'agent_id': 'agent_structural_data_rcsb', 'role': 'RCSB PDB 数据获取、缓存、整理与划分专家。', 'instruction': '为 src/extract_sidechain_dataset.py 设计并实现公共 PDB 数据路径。使用分辨率和质量过滤器、实验方法、聚合物/蛋白质过滤器、非废弃条目、max_structures 以及确定性排序或带种子的采样。指定 RCSB Search API POST 请求载荷。指定 files.rcsb.org 下载 URL、缓存规则、重试策略和清单。与 agent_torsion_geometry 协调残基完整性问题,与 agent_eval_benchmarks 协调划分泄露问题。数据集必须排除用于 χ_1 的 ALA 和 GLY,保留源自侧链原子的标签,但禁止侧链坐标进入模型输入。优先采用结构级划分并注明同源性或序列相似度风险;若全面同源性聚类超出范围,则创建有文档记录的保守划分清单和局限性说明。', 'output_to_orchestrator_schema': '交接内容包括: data_card;rcsb_query_payload;rcsb_query_url_or_endpoint;pdb_cache_manifest_schema;downloaded_vs_skipped_schema;residue_record_schema;structure_split_schema;summary_json_schema;leakage_checklist;implementation_todos_by_file;使用 uv run 的 validation_commands;requests_for_agents,尤其是几何标签定义和评估划分检查。'}, {'agent_id': 'agent_torsion_geometry', 'role': '蛋白质侧链化学、 χ 角标签与完整性专家。', 'instruction': '定义所有含 χ_1 角的 18 种残基类型的 χ_1 原子名称、完整重侧链原子要求、altloc 占有率策略、插入码与链处理、杂原子/水分子排除、缺失原子处理行为,以及圆形角度约定。提供可测试的二面角公式和边界情况检验。在训练代码使用数据前,与 agent_structural_data_rcsb 共同签署 DatasetRecord。明确审计侧链原子不向输入特征泄露标签。', 'output_to_orchestrator_schema': '交接内容包括:chi1_atom_definition_table;complete_sidechain_atom_table;altloc_policy;residue_exclusion_rules;dihedral_formula;angle_range_convention;标签字段(label_fields),包括 sin_chi1、cos_chi1、chi1_degrees;geometry_validation_tests;no_sidechain_input_leakage_checks;针对数据、模型和评估智能体的 downstream_schema_updates。'}, {'agent_id': 'agent_geometric_ml', 'role': 'SE(3)/E(3) 等变架构与模型 API 专家。', 'instruction': '围绕一个小型可复现的 GVP 风格 SE(3) 等变网络(用于骨架上下文到 χ_1 预测)设计 src/model_se3.py。需明确说明:最终 χ_1 输出对全局 SE(3) 变换是不变的,而内部向量通道是等变的。定义从 DatasetRecord 中消费的节点、边、坐标、图、批次和输出 schema。相对于不变基线指定合理的参数预算。添加或请求针对模型输出随机旋转和平移不变性的测试,以及(在可访问的情况下)向量特征等变性的测试。与 agent_baseline_ablation 协调变体方案,与 agent_training_reproducibility 协调可训练类名和检查点键名。',

原文第 37 页

'output_to_orchestrator_schema': 'Handoff plus: model_io_contract; feature_dimension_table; graph_construction_contract; class_and_function_signatures; equivariance_proof_sketch; equivariance_unit_test_plan; ablation_variant_candidates with expected cost; parameter_count_estimates; failure_modes; dependencies; requests_for_agents for missing DatasetRecord fields or training hooks.', {'agent_id': 'agent_baseline_ablation', 'role': '不变基线(invariant baseline)、等变变体(equivariant variants)及公平对比消融(ablation)专家。', 'instruction': '定义受控基线和消融网络。最小网格应包含一个仅使用距离的不变基线,以及至少两个等变行,例如完整 GVP 和一个更小或经特征消融的 GVP 变体,除非编排器(orchestrator)明确批准以偏差名义减少计算量。在深度、训练预算、数据划分、优化器和参数规模上保持比较的公平性。定义 results/ablation_results.json,使下游报告和绘图智能体(agent)能够直接使用,无需猜测字段含义。', 'output_to_orchestrator_schema': 'Handoff plus: baseline_contract; fairness_controls; ablation_grid with model_id, variant_description, expected_files, train_budget, parameter_budget; ablation_results_json_schema; comparison_plot_spec for deliverables/plots/ablation_equivariant_vs_invariant.png; validation_checks for same split and same metric implementation; recall_triggers if variants are missing.', {'agent_id': 'agent_training_reproducibility', 'role': '训练循环、可复现性、配置、检查点及 uv 工作流(workflow)专家。', 'instruction': '实现并验证 src/train.py 及 main.py 的集成。使用 uv run 命令、固定随机种子、确定性数据划分消费、训练/验证/测试分离、检查点保存、training_history JSON 文件、设备回退及短暂冒烟运行选项。协调模型与消融智能体以训练所有必要的 model_id。确保 NaN、数据缺失、形状不匹配或检查点不兼容等故障能产生可操作的错误信息。在 logs.txt 中记录精确命令和实际运行时间摘要。', 'output_to_orchestrator_schema': 'Handoff plus: training_config_schema; seed_and_determinism_plan; split_consumption_plan; optimizer_and_schedule; checkpoint_schema; training_history_schema; command_plan_full_and_smoke; failure_debug_playbook; produced_artifact_list; runtime_estimates; validation_results from uv run commands.', {'agent_id': 'agent_eval_benchmarks', 'role': '圆形误差指标(circular metric)、基准测试、逐残基分析及绘图数据专家。', 'instruction': '实现并验证 src/evaluate.py 及产生指标的工具函数。定义圆形绝对误差、MAE、中位数、RMSE、包含计数的逐残基分解(覆盖每个被评估的 model_id),以及在时间允许时的可选自举(bootstrap)不确定性估计。确保 results/metrics.json 和 results/ablation_results.json 包含足够的结构化数据,供报告、索引和绘图使用。交叉检验图表和报告数字均来源于 JSON,而非手动转录。与数据智能体协调数据划分事宜,与科学传播智能体协调结论声明。', 'output_to_orchestrator_schema': 'Handoff plus: metric_formulae; circular_error_algorithm; metrics_json_schema; per_residue_schema_all_models; test_index_schema; plot_data_requirements; number_source_map mapping report claims to JSON keys; statistical_notes; validation_commands;

issues_for_training_or_report if values are inconsistent.' }, { 'agent_id': 'agent_profile_systems', 'role': '运行时、内存、环境及性能分析方法专家。', 'instruction': '实现并验证 src/profile.py。对所有 model_id 或主要基线对,在不同批次大小下分析前向传播和训练步骤的延迟。捕获环境信息、参数量、设备、线程及性能分析方法。对于内存,若 PyTorch 原生分配未被 tracemalloc 捕获,不应仅依赖 tracemalloc;至少包含一种可靠的度量方式,如 psutil RSS 增量、torch profiler profile_memory、可用时的 CUDA max_memory_allocated,以及参数内存估算。若内存值极小,请解释方法的局限性并提供更有意义的估算值。生成 deliverables/plots/runtime_memory_profile.png 所需数据及 results/metrics.json 的结构化指标。',

原文第 38 页

```
'output_to_orchestrator_schema': 'Handoff plus: profiling_protocol; environment_capture_schema; batch_sizes; runtime_memory_json_schema; memory_methods_and_caveats; parameter_memory_estimates; throughput_calculation; plot_spec; validation_commands; profiler_limitations; recall_triggers for implausible or undocumented memory values.' }, { 'agent_id': 'agent_scicomm_publication', 'role': '会议风格报告、图表叙述及结论证据专家。', 'instruction': '仅在评估与性能分析智能体发布冻结的 ResultsContract 和 ProfilingContract 后,方可起草 research_report.md。撰写一篇相当于 7 至 9 页的论文,包含摘要、引言、相关工作、方法、结果、性能分析、局限性、结论和参考文献各节。每项定量声明必须引用来自 metrics、ablation、training history 或 profiling 的 JSON 键。检查图表标题的方向性是否正确,例如 MAE 越低意味着性能越好,而非越差。避免在小规模 PDB 衍生 x1 预测范围之外过度声明。', 'output_to_orchestrator_schema': 'Handoff plus: report_outline; figure_table_map; claims_to_evidence_matrix; exact_metric_keys_used; caption_consistency_checks; limitations_section_points; citation_plan; unresolved_claims_needing_eval; recall_requests for inconsistent or missing metrics.' }, { 'agent_id': 'agent_artifact_validator', 'role': '独立 CI、制品(artifact)、模式(schema)及可复现性验证专家。', 'instruction': '独立检查最终工作空间。运行或指定 uv run 冒烟检查、导入检查、JSON 解析检查、图表非空检查、每项必要交付物的文件存在性检查,以及 research_report.md、deliverables/index.json、results/metrics.json 和 results/ablation_results.json 之间的一致性检查。该智能体不应参与主要实现的编写;其职责是审计实现并向编排器提交具体问题工单以触发召回。', 'output_to_orchestrator_schema': 'Handoff plus: file_presence_matrix; json_schema_results; command_results with command, exit_code, stdout_summary, stderr_summary; plot_integrity_results; report_json_consistency_results; artifact_registry_diff; issue_tickets with severity and owner; final_blockers.' }, { 'agent_id': 'agent_quality_safety', 'role': '科学严谨性、数据泄露、可复现性及合规性审查专家。', 'instruction': '审计已组装的计划和制品,检查数据泄露、缺失对照、过度声明、错误的等变性表述、薄弱的消融实验、无效的性能分析、不一致的指标及可复现性缺口。使用来自文件和 JSON 输出的具体证据。当某专家需要解释或修复问题时,通过 requests_for_agents 直接沟通。最终批准要求:无阻断性数据泄露、无未更正的报告-指标矛盾、已记录的数据获取过滤条件、有效的圆形指标,以及可复现的 uv 工作流。安全范围仅限于计算层面;避免涉及湿实验室操作内容。', 'output_to_orchestrator_schema': 'Handoff plus: audit_checklist; leakage_assessment; equivariance_assessment; ablation_assessment; profiling_assessment; reproducibility_assessment; overclaiming_assessment; required_recalls; final_recommendation pass/fail/conditional; evidence_paths.' }
```

]

所有非编排器智能体的共享通信契约(contract):

每条响应必须是一个可机器检查的 Handoff 对象,包含以下键:agent_id、phase、scope、inputs_consumed、assumptions、interface_exports、artifact_plan_or_changes、validation_checks、metrics_or_estimates、risks、open_questions、requests_for_agents、recall_recommendations。interface_exports 条目必须命名 schema_or_signature、producer、consumer_agents、version 和 compatibility_notes。artifact_plan_or_changes 条目必须包含 path、status(planned|created|modified|validated)、purpose、producer、consumer_agents 和 validation_status。validation_checks 条目必须包含 check_id、command_or_method、expected、actual、status(pass|fail|blocked)和 blocking_if_fail。requests_for_agents 条目必须包含 target_agent_id、question、required_response_schema 和 urgency。

强制性智能体间交接(handoff):

1. agent_structural_data_rcsb 和 agent_torsion_geometry 必须在模型、训练或评估实现开始前,联合生成 DatasetRecord v1。
 2. agent_geometric_ml 和 agent_baseline_ablation 必须在训练开始前,联合生成 ModelIO 和 AblationGrid v1。
- 原文第 39 页
3. agent_eval_benchmarks 必须在 scicomm 撰写定量声明或 index.json 摘要之前发布 ResultsContract v1。
 4. agent_profile_systems 必须在报告讨论速度或内存之前发布 ProfilingContract v1。

5. agent_artifact_validator 和 agent_quality_safety 独立审查已冻结的制品,并可开具召回工单。

最少编排器-子智能体跳数(hop):

跳 A:携带范围和验收标准的初始委派。跳 B:InterfaceContract v1 发布后的合约确认,每个消费方智能体返回 ack_or_change_request。跳 C:实现完成后的证据审查,验证器和质量审查的结论路由回各归属专项智能体。跳 D:针对每个阻塞问题发送定向 PatchHandoff 召回,包含 issue_ids_closed、changed_files、validation_evidence 和 remaining_risk。编排器可执行额外跳数,直至 final_gate 通过。

召回触发条件:

若 RCSB 查询载荷、缓存清单、侧链完整性规则、分割清单或无泄漏证据缺失,则召回数据或几何模块。若随机旋转/平移不变性测试缺失或等变性(equivariance)表述有误,则召回模型。若 results/ablation_results.json 中少于一个不变基线加两个等变体,且无已批准的偏差说明,则召回消融或训练模块。若中位角度误差、MAE、RMSE、每残基计数或循环度量定义在各文件间缺失或不一致,则召回评估模块。若内存数值小得不合情理且无方法说明,以及缺少参数量或 RSS 估算,则召回性能分析(profiling)模块。若报告正文与 JSON 指标、图表或图题矛盾,则召回 scicomm 模块。若任何必需交付物路径缺失、为空、不可解析,或未列入 deliverables/index.json,则召回验证器。

冲突解决:

若两个智能体存在分歧,编排器创建一条 ConflictRecord,记录各方立场、证据、受影响制品及决策。优先选取科学上更保守的方案,在 research_report.md 中记录局限性,并更新 logs.txt。

工具组 H[2]

Harness(2)

创建一个包含以下候选智能体的智能体团队来解决该问题:

```
{
  "agent_id": "agent_orchestrator",
  "role": "首席科学家-编排器、集成负责人、召回管理器及最终验收门禁。",
```

原文第 40 页

```
"instruction": "全权负责 SE(3) 侧链  $\chi_1$ 
训练方案。运行多轮工作流 ( workflow ), 包含显式合约协商、定向召回、补丁验证和最终门禁。R0: 根据必需交付物和先前失败证据建立基于证据的验收规则
profiler.py, 而任务明确要求 src/profile.py; 声称通过 RCSB Search API 获取了数据, 但无查询/下载清单为证; 报告摘要称吞吐量损耗为
30 倍, 而性能分析 JSON 值暗示约 100 倍。R1: 并行扇出至接口/
路径管理、数据、扭转几何、模型、消融、训练、评估、性能分析、报告、声明审计、验证器和质量等智能体。R2: 编译 InterfaceContract v2 和
RequiredPathManifest v1, 发送给每个消费方征求
ack_or_change_request; 在阻塞性合约变更得到协调之前, 不允许消费方推进实现。R3: 使用已冻结合约委派实现任务。R4: 在昂贵的训练开始前执行早期制
具体检查确切的必需路径、RCSB 来源制品、导入和 uv 命令。R5: 仅在评估/性能分析验证完成后冻结 ResultsContract 和
ProfilingContract。R6: 仅在消费了已冻结的结果/性能分析合约后, 才允许 scicomm
起草定量报告声明。R7: 将报告草稿和索引送交独立声明裁判员、验证器和质量审查员。R8: 针对每个阻塞问题以窄范围 PatchHandoff 召回归属专项智能体,
重测直至关闭或明确豁免。R9: 仅在阻塞问题全部关闭后进行最终集成。在 issue_tracker 中维护严重性、负责人、证据路径或 JSON
指针、请求修复、阻塞条件、状态、召回历史、重测证据及豁免理由 ( 如有 )。在 logs.txt
中记录智能体名册、调用图、所有交接、合约确认、召回决策、命令及未解决风险。优先使用下方任务专项智能体; 除非目标改变, 否则不实例化扩散、分子动
"output_to_orchestrator_schema": "OrchestrationStateV2
对象, 包含以下键: objective ( 目标 ); evidence_grounded_acceptance_criteria ( 基于证据的验收标准 ); canonical_require
ed_paths ( 规范必需路径 ); shared_interface_contract ( 共享接口合约 ); contract_ack_matrix ( 合约确认矩阵 ); artifact_
registry ( 制品注册表 ); data_provenance_registry ( 数据来源注册表 ); issue_tracker ( 问题追踪器 ); conflict_records (
冲突记录 ); delegation_rounds ( 委派轮次 ); recall_plan ( 召回计划 ); patch_retest_log ( 补丁重测日志 ); final_gate_dec
ision ( 最终门禁决策 )。artifact_registry 条目需包含 exact_path、required_by_task
布尔值、producing_agent、consuming_agents、status、size_bytes_or_sha256、validation_status 和
index_json_listed 布尔值。final_gate_decision 必须引用每个剩余 issue_id 及豁免理由。"
},
{
  "agent_id": "agent_interface_contract_steward",
  "role": "模式 ( schema )、精确交付物路径及跨智能体合约管理员。",
  "instruction": "在实现开始前, 将任务交付物转化为规范的 RequiredPathManifest 和 InterfaceContract
v2。将精确文件名视为约束性规定: 要求 src/profile.py; src/profiler.py 仅可作为辅助文件存在, 不能替代前者。确保 deliverables/
index.json 将列出精确的必需路径, 而非仅列出语义相似的路径。为
DatasetRecord、ModelIO、ModelRegistry、AblationGrid、TrainingHistory、ResultsContract、Profil
ingContract、PlotManifest、DataProvenanceManifest、ReportClaims 和 IndexManifest 构建
SchemaRegistry
条目。将每份合约草案路由至所有消费方智能体, 并收集字段级确认或变更请求。在代码编写之前, 将任何缺失生产方、消费方模糊或字段不可测试的情况标记
"output_to_orchestrator_schema": "ContractStewardHandoff 继承
HandoffV2, 必须包含: required_path_manifest ( 含
exact_path、required_by_task、producer_agent、validator_agent、substitute_allowed 默认为 false
```

```

除非明确说明, 以及 expected_index_entry ); schema_registry ( 含
schema_id、version、required_fields、optional_fields、producer、consumer_agents、validation_me
thod 和 compatibility_notes ); contract_ack_matrix ( 含 consumer_agent、schema_id、ack_status
accept|change_request|blocked、requested_changes 和
resolution ); data_provenance_required_artifacts ( 包含 data/rcsb_query.json、data/
rcsb_response_ids.json、data/download_manifest.json、data/data_card.json 和 data/
split_manifest.json ); blocker_candidates ( 含证据和负责人 )。"
},
{
  "agent_id": "agent_structural_data_rcsb",
  "role": "RCSB PDB 数据获取、缓存、整理、来源及分割专项智能体。",

```

原文第 41 页

'instruction': '为 src/extract_sidechain_dataset.py 设计并实现公开 PDB 数据路径。使用分辨率和质量过滤器、实验方法、聚合物/蛋白质过滤器、非废弃条目、max_structures、确定性排序或带种子的采样以及响应 ID 处理,指定并持久化 RCSB Search API POST 端点和请求载荷。使用 files.rcsb.org 下载 URL,并支持缓存、重试、状态记录和跳过结构原因记录。创建可验证的来源工件(provenance artifact),而非仅生成报告性声明。与 agent_torsion_geometry 协调残基完整性问题,与 agent_eval_benchmarks 协调集合划分泄漏(split leakage)问题。数据集必须在 $\chi 1$ 中排除 ALA 和 GLY,保留由侧链原子衍生的标签,并防止侧链坐标进入模型输入。优先采用结构级划分,并附注同源性或序列同一性风险;若完整同源性聚类超出范围,则创建一份有文档记录的保守划分清单及局限性说明。', 'output_to_orchestrator_schema': 'DataRCSBHandoff 继承 HandoffV2,必须导出 DataProvenanceManifest v1 和 DatasetRecordDraft v2。必需工件:data/rcsb_query.json,包含 endpoint、payload、query_hash、timestamp、max_structures、ordering_or_seed 和 filters;data/rcsb_response_ids.json,包含 returned_ids 和 selected_ids;data/download_manifest.json,包含 pdb_id、url、cache_path、status(downloaded|cached|skipped|failed)、retry_count、file_size、sha256_if_available 和 skip_reason;data/data_card.json,包含统计计数和提取统计信息;data/split_manifest.json,包含训练/验证/测试集 PDB ID、残基计数、划分方法、随机种子、无重叠检查以及同源性局限性说明。包含 residue_record_schema、明确排除侧链原子坐标的 input_feature_schema、按文件划分的 implementation_todos_by_file、使用 uv run 的 validation_commands、leakage_checklist,以及向其他智能体(agent)提出的几何标签定义和评估集划分检查请求。', { 'agent_id': 'agent_torsion_geometry', 'role': '蛋白质侧链化学、 χ 角标签、完整性与无泄漏专家。', 'instruction': '为所有 18 种含 $\chi 1$ 的残基类型定义 $\chi 1$ 原子名称,明确完整重侧链原子要求、替代构象(altloc)占有率策略、插入码与链处理方式、杂原子/水分子排除规则、缺失原子处理行为及圆角约定。提供可测试的二面角公式和边界情况检查。在模型、训练或评估使用数据集之前,与 agent_structural_data_rcsb 共同签署 DatasetRecord v2。明确审计侧链原子仅用于计算标签,不得出现在模型输入张量、已保存的特征张量和批次(batch)对象中。', 'output_to_orchestrator_schema': 'GeometryHandoff 继承 HandoffV2,必须包含 chi1_atom_definition_table;complete_sidechain_atom_table;含占有率平局处理行为的 altloc_policy;residue_exclusion_rules;dihedral_formula 和 angle_range_convention;标签字段 sin_chi1、cos_chi1、chi1_degrees;含 accepted_fields、rejected_fields 和 required_patches 的 DatasetRecordAck;含命令或方法的 geometry_validation_tests;命名所检查数据集键的 no_sidechain_input_leakage_checks;以及面向数据、模型、训练和评估智能体的 downstream_schema_updates。', { 'agent_id': 'agent_geometric_ml', 'role': 'SE(3)/E(3) 等变架构、模型 API 与等变性测试专家。', 'instruction': '围绕小型可复现的 GVP 风格 SE(3) 等变网络设计 src/model_se3.py,用于从骨架上下文预测 $\chi 1$ 。明确说明最终 $\chi 1$ 输出对全局 SE(3) 变换是不变的,而内部向量通道是等变的。定义从 DatasetRecord v2 中获取的节点、边、坐标、图、批次、类名和输出模式(schema)。针对不变基线,指定公平的参数量预算。添加针对模型输出旋转和平移不变性的测试,以及在可访问时对向量特征等变性的测试。与 agent_baseline_ablation 协调模型变体,与 agent_training_reproducibility 协调可训练类名、工厂函数和检查点键。', 'output_to_orchestrator_schema': 'ModelMLHandoff 继承 HandoffV2,必须导出 ModelIOContract v2 和 ModelRegistryDraft v1。必需字段: feature_dimension_table;含半径、k、批次行为和坐标来源的 graph_construction_contract;class_and_function_signatures;支持的 model_ids;含 sin/cos 顺序和归一化行为的 output_schema;equivariance_proof_sketch;equivariance_unit_test_plan,以及实现后的 EquivarianceTestEvidence(包含 command、seed、变换描述、tolerance、max_error、status 和 evidence path);parameter_count_estimates;dependencies;以及向其他智能体提出的缺失 DatasetRecord 字段或训练钩子请求。', { 'agent_id': 'agent_baseline_ablation',

原文第 42 页

'role': '不变基线、等变变体及公平对比消融专家。', 'instruction': '定义受控基线与消融网格。最小网格必须包含一个仅使用不变距离的基线,以及至少两行等变配置,例如完整GVP与规模更小或特征消融后的GVP,除非计算资源限制已由编排器明确批准作为偏差。在数据划分、指标实现、优化器、训练轮次预算、早停策略及参数规模说明等方面保持比较公平。与 agent_geometric_ml 共同维护 ModelRegistry v1,确保训练/评估/分析 agent 可直接使用 model_id 而无需猜测。', 'output_to_orchestrator_schema': 'AblationHandoff 继承 HandoffV2,必须包含 baseline_contract;fairness_controls;AblationGrid

v2,其字段包括model_id、variant_description、model_class_or_factory、expected_checkpoint_path、expected_history_path、expected_metrics_keys、train_budget、parameter_budget及equivariant布尔值;ablation_results_json_schema;用于deliverables/plots/ablation_equivariant_vs_invariant.png的comparison_plot_spec;用于验证相同划分和相同循环指标实现的validation_checks;以及当不变变体少于一个或等变变体少于两个时触发的recall_triggers。'}, {'agent_id': 'agent_training_reproducibility', 'role': '训练循环、可复现性、配置、检查点、uv工作流(workflow)及CLI集成专家。', 'instruction': '实现并验证src/train.py及可复现uv工作流所需的任何CLI集成。使用固定随机种子、确定性划分消费、训练/验证/测试分离、检查点保存、training_history JSON文件、设备回退、短时冒烟运行选项及精确命令。与模型和消融agent协调,训练所有必需的model_id。确保NaN、缺失数据、形状不匹配、检查点不兼容、缺失model_id或划分清单缺失等故障能产生可操作的错误信息。若main.py已存在,不得将其保留为无实质内容的hello-world入口点;应将其改造为有用的流水线调度器或在logs.txt和索引元数据中保持所有文档化命令的显式记录。', 'output_to_orchestrator_schema': 'TrainingHandoff继承HandoffV2,必须包含training_config_schema;seed_and_determinism_plan;引用data/split_manifest.json的split_consumption_plan;optimizer_and_schedule;含预期键的checkpoint_schema;training_history_schema;model_id_to_command_map;使用uv run的command_plan_full_and_smoke;含精确检查点/历史路径的produced_artifact_list;runtime_estimates;含command、exit_code、stdout_summary、stderr_summary及证据路径的validation_results;failure_debug_playbook;以及出现不兼容时的requests_for_model_or_data_patches。'}, {'agent_id': 'agent_eval_benchmarks', 'role': '循环指标、基准测试、逐残基分析、划分泄漏审查及图表数据专家。', 'instruction': '实现并验证src/evaluate.py及生成指标的工具体函数。定义循环绝对误差、MAE、中位数误差、RMSE,为每个已评估的model_id提供含样本数的逐残基分解,以及在时间允许时提供可选的自助法(bootstrap)不确定性估计。使用split_manifest验证测试集PDB ID与训练/验证集PDB ID不相交,并记录任何同源性局限。确保results/metrics.json和results/ablation_results.json包含足够的结构化数据供报告、索引和图表使用。交叉验证图表和报告中的数值均来源于JSON而非手动录入。与scicomm协作维护number_source_map。', 'output_to_orchestrator_schema': 'EvalHandoff继承HandoffV2,必须导出ResultsContract v2。必填字段:metric_formulae;circular_error_algorithm;含model_id键、test_mae_degrees、test_median_error_degrees、test_rmse_degrees、n_test_samples、n_params、best_epoch、best_val_loss及per_residue计数的metrics_json_schema;ablation_results_schema;test_index_schema;含训练/验证/测试集PDB重叠状态的split_leakage_result;plot_data_requirements;将报告声明标签映射至JSON指针和计算表达式的number_source_map;statistical_notes;validation_commands;以及当数值不一致时的issues_for_training_or_report。'}, {'agent_id': 'agent_profile_systems', 'role': '运行时、内存、环境、必需的src/profile.py路径及分析方法专家。',

原文第 43 页

'instruction': '严格按照任务要求实现并验证 src/profile.py。仅当 src/profile.py 作为已记录的 CLI/包装器存在且 deliverables/index.json 中列出 src/profile.py 时,才允许使用名为 src/profiler.py 的辅助文件。对所有 model_ids 或主基线对,分批次大小对前向传播和训练步骤的延迟进行剖析(profiling)。捕获环境详情、参数量、设备、线程、同步策略、预热/重复次数及剖析方法论。在内存方面,若 PyTorch 原生分配被遗漏,不可仅依赖 tracemalloc;需包含 psutil RSS、可用时的 torch CUDA max_memory_allocated、torch profiler profile_memory 或已记录的替代方案,以及参数内存估算。计算吞吐量比值(throughput ratio),尤其是在报告批次大小下不变基线(invariant baseline)与 SE3-GVP 之间的比值,以防科学传播(scicomm)在 JSON 实际含义约为 100 倍时声称 30 倍。', 'output_to_orchestrator_schema': 'ProfilingHandoff 继承 HandoffV2,并须导出 ProfilingContract v2。必需产物:src/profile.py 状态(created|modified|validated);results/profiling_results.json;若 report 使用,则在 results/metrics.json 中镜像或引用 profiling 章节。必需字段:profiling_protocol;environment_capture_schema;batch_sizes;runtime_memory_json_schema;memory_methods_and_caveats;存在 CUDA 时的 cuda_memory_fields;rss_fields;parameter_memory_estimates;throughput_calculation;throughput_ratio_table(含 numerator_model、denominator_model、batch_size、json_pointers、ratio 及 narrative_phrase_recommendation);deliverables/plots/runtime_memory_profile.png 的 plot_spec;validation_commands;profiler_limitations;recall_triggers(针对 src/profile.py 缺失、未记录的异常内存或 report/profile 比值不一致)。'}, {'agent_id': 'agent_scicomm_publication', 'role': '会议风格报告、图表叙述及主张-证据专家。', 'instruction': '仅在 eval 和 profiling 智能体(agent)发布冻结的 ResultsContract v2 与 ProfilingContract v2、且编排器(orchestrator)提供 number_source_map 之后,方可起草 research_report.md。撰写一篇等价于 7 至 9 页的论文,包含摘要、引言、相关工作、方法、结果、剖析、局限性、结论及参考文献各章节。每一项定量主张须引用来自 metrics、ablation、训练历史、profiling 或数据溯源产物的 JSON 键或计算表达式。检查图表标题的方向正确性,例如较低的 MAE 意味着性能更优而非更差。速度相关主张须使用 ProfilingContract 中的 throughput_ratio_table,而非手工估算。避免超出小规模 PDB 衍生 χ 1 预测范围的过度声索(overclaiming)。', 'output_to_orchestrator_schema': 'SciCommHandoff 继承 HandoffV2,须包含:report_outline;figure_table_map;claims_to_evidence_matrix(含 claim_text、value、unit、source_path、json_pointer_or_expression、tolerance 及 checked_status);exact_metric_keys_used;caption_consistency_checks;引用

rcsb_query 及下载清单的 data_provenance_claims;limitations_section_points(含 split/homology 注意事项); citation_plan;unresolved_claims_needing_eval_or_profile;以及针对不一致或缺失指标的 recall_requests。'}, {'agent_id': 'agent_claims_consistency_referee', 'role': '独立的定量主张、报告指标及索引一致性裁决者。', 'instruction': '在 scicomm 起草 research_report.md 和 deliverables/index.json 之后,独立地将每一项数值或比较性主张与 results/metrics.json、 results/ablation_results.json、 results/profiling_results.json、 data/data_card.json、 data/split_manifest.json 及 claims_to_evidence_matrix 进行比对。尤其注意此前已观察到的不一致:441151 样本/秒对 4136 样本/秒意味着约 107 倍,而非 30 倍。对陈旧文本、错误比值、无依据的 RCSB/溯源主张,或与 JSON 不符的索引指标,发起问题工单(issue ticket)。本智能体不负责主要实现的撰写;其职责为批审并提供机器可检验的问题工单。', 'output_to_orchestrator_schema': 'ClaimsAuditHandoff 继承 HandoffV2,须包含:numeric_claim_inventory(含 claim_id、 file、 quote、 parsed_value、 unit、 source_json_pointer_or_missing、 recomputed_value、 relative_error_pct 及 status);comparative_claim_checks(含方向 lower_is_better 或 higher_is_better);throughput_ratio_checks;data_claim_checks;index_summary_checks;issue_tickets(含 severity、 owner、 evidence、 requested_fix 及 blocking_if_fail);最终建议 pass|fail|conditional。'}, {'agent_id': 'agent_artifact_validator', 'role': '独立的 CI、精确路径、模式(schema)、溯源及可复现性验证器。',

原文第 44 页

'instruction': '独立检查最终工作区至少两次:在实现开始后设置一个早期冒烟关卡,在补丁完成后设置一个最终关卡。运行或指定以下检查:uv run 冒烟检查(smoke checks)、导入检查、JSON 解析检查、图表非空检查、每个必需可交付成果的精
确文件存在检查、RCSB 溯源检查,以及 research_report.md、 deliverables/index.json、 results/metrics.json、 results/ablation_results.json、 results/profiling_results.json 与数据清单之间的一致性检查。若 src/profile.py 不存在,即使 src/profiler.py 存在,精确路径检查也必须失败。该智能体(agent)不负责编写主要实现;其职责是审计实现并向编排者 (orchestrator)提交具体问题工单以触发召回。', 'output_to_orchestrator_schema': 'ValidatorHandoff 继承 HandoffV2,必须包含:file_presence_matrix(含 exact_path、 exists、 size_bytes、 listed_in_index、 required_by_task 和 status);json_schema_results;command_results(含 command、 exit_code、 stdout_summary、 stderr_summary 和 evidence path);plot_integrity_results;rcsb_provenance_results(检查 query/download/split 清单);report_json_consistency_results;artifact_registry_diff;issue_tickets(含 severity 和 owner);final_blockers;以及补丁后针对 issue_ids 的 retest_results。'}, {'agent_id': 'agent_quality_safety', 'role': '科学严谨性、数据泄露、可复现性、等变性 (equivariance)语言、性能分析及合规性审查员。', 'instruction': '审计已组装的计划和工件,检查数据泄露、缺失的对照实验、过度声明、错误的等变性语言、消融实验不足、无效的性能分析、不一致的指标、无法核实的 RCSB 数据获取、精确可交付成果不匹配以及可复现性缺口。使用来自文件、JSON 输出、日志和验证命令的具体证据。当某专项智能体需要解释或修复某问题时,通过 requests_for_agents 进行沟通。最终审批要求:无阻塞性数据泄露、无未纠正的报告-指标矛盾、已记录的 RCSB 过滤条件和下载清单、有效的循环指标、至少一行不变性(invariant)加两行等变性消融结果(除非豁免)、健全的性能分析注意说明、确切的 src/profile.py 文件,以及可复现的 uv 工作流(workflow)。安全范围仅限计算层面;避免涉及湿实验操作内容。', 'output_to_orchestrator_schema': 'QualityAuditHandoff 继承 HandoffV2,必须包含: audit_checklist;leakage_assessment;data_provenance_assessment;equivariance_assessment;ablation_assessment;profiling_assessment;reproducibility_assessment;overclaiming_assessment;exact_deliverable_path_assessment;required_recalls;final_recommendation(pass|fail|conditional);evidence_paths 和 json_pointers。'}]

每个非编排者智能体的共享通信契约(contract):

每条响应必须是一个机器可检查的 HandoffV2 对象,包含以下键:agent_id、 phase、 scope、 contract_version、 inputs_consumed、 assumptions、 interface_exports、 artifact_plan_or_changes、 validation_checks、 metrics_or_estimates、 evidence_bundle、 risks、 open_questions、 requests_for_agents、 ack_or_change_request、 recall_recommendations。 inputs_consumed 条目必须包含:source_agent_id、 export_name、 version、 artifact_path_or_json_pointer 和 consumed_status。 interface_exports 条目必须包含:schema_id_or_signature、 producer、 consumer_agents、 version、 status(draft|frozen|deprecated)、 required_fields、 optional_fields、 compatibility_notes 和 validation_method。 artifact_plan_or_changes 条目必须包含:path、 exact_required_path(布尔值)、 status(planned|created|modified|validated)、 purpose、 producer、 consumer_agents、 size_or_sha256_if_available 和 validation_status。 validation_checks 条目必须包含: check_id、 command_or_method、 expected、 actual、 exit_code_if_command、 status(pass|fail|blocked)、 blocking_if_fail 和 evidence_path。 metrics_or_estimates 条目必须包含:name、 value、 unit、 source_path、 json_pointer_or_expression 和 uncertainty_or_caveat。 requests_for_agents 条目必须包含:target_agent_id、 question、 required_response_schema、 urgency、 blocking_if_unanswered 和 requested_by_hop。 ack_or_change_request 必须包含:reviewed_schema_ids、

decision(accept|change_request|blocked)、field_level_comments 和 required_resolution。recall_recommendations 必须包含: issue_id、owner_agent_id、narrowed_scope、acceptance_criteria_for_patch 和 retest_method。

强制性智能体间交接(handoff)规则:

1. agent_interface_contract_steward 在任何实现开始之前发布 RequiredPathManifest v1;每个生产智能体和验证智能体必须返回 ContractAck 或 ChangeRequest。
 2. agent_structural_data_rcsb 与 agent_torsion_geometry 在模型、训练或评估实现消费数据之前,联合生成 DatasetRecord v2 和 DataProvenanceManifest v1。
- 原文第 45 页
3. agent_structural_data_rcsb 与 agent_eval_benchmarks 在训练/评估声明冻结之前,共同生成 SplitLeakageReview v1。
 4. agent_geometric_ml 与 agent_baseline_ablation 在训练开始之前,共同生成 ModelRegistry v1、ModelIOContract v2 和 AblationGrid v2;agent_training_reproducibility 必须确认可导入性、检查点键(checkpoint keys)和 model_ids。
 5. agent_eval_benchmarks 在 scicomm 撰写定量精度声明或 index.json 摘要之前,发布 ResultsContract v2。
 6. agent_profile_systems 在 scicomm 撰写速度或内存声明之前,发布 ProfilingContract v2 并验证确切的 src/profile.py。
 7. agent_scicomm_publication 将 ClaimsEvidenceMatrix 发送给 agent_claims_consistency_referee;所有裁判阻塞项被路由至负责该项的专项智能体(specialist)或 scicomm 进行修补。
 8. agent_artifact_validator 与 agent_quality_safety 独立审查冻结的制品(artifact),可开具召回工单;其最终检查在任何补丁轮次之后进行,而不仅限于补丁之前。

编排器-子智能体最少跳数(hop):

跳 A EvidenceRubric 扇出:编排器将任务交付物、先前证据缺陷和验收标准发送给所有智能体。

跳 B DraftContract 返回:专项智能体返回草稿导出物和 requests_for_agents;管家(steward)编译 InterfaceContract v2 和 RequiredPathManifest v1。

跳 C 合约确认:编排器将相关合约(contract)发送给消费方;每个消费方返回 ack_or_change_request。阻塞性变更请求在实现之前立即触发合约协调召回。

跳 D 实现委派:编排器委派具有冻结合约和精确验收检查的文件与制品。

跳 E 早期冒烟验证:验证器在昂贵的训练/分析/报告工作开始之前,检查精确路径、导入项、RCSB 溯源制品和 uv 冒烟命令。

跳 F 结果与分析冻结:训练、评估和分析智能体发布冻结的指标、消融结果、历史记录、profile JSON 和 number_source_map;编排器将其路由至 scicomm。

跳 G 报告与索引起草:scicomm 仅使用冻结合约起草报告/索引,并生成 ClaimsEvidenceMatrix。

跳 H 独立审查:声明裁判、验证器和质量智能体对报告、索引、数据溯源、精确路径和 JSON 一致性进行审计。

跳 I 定向 PatchHandoff 召回:对于每个阻塞项,编排器仅召回其所有者,并附上缩小范围(narrowed_scope)、issue_ids_to_close、required_changed_files 和重测标准。所有者返回 PatchHandoff,包含 issue_ids_closed、changed_files、diff_summary、validation_evidence 和 remaining_risk。

跳 J 重测与最终关卡:验证器或质量智能体重测特定 issue_ids;编排器重复跳 I J,直至无阻塞项残留,或记录附有证据与理由的豁免。

召回触发条件:

若任何精确要求的交付物路径缺失、为空、不可解析、未列于 deliverables/index.json,或被不同名称的文件替代,则召回接口管家或验证器;src/profile.py 缺失始终为阻塞项。若 RCSB 查询载荷、端点、选定 ID、下载 URL、缓存/下载清单、分割清单或 data_card 证据缺失或仅存在于报告中,则召回数据智能体。若 χ^2 原子表、完整性规则、替代位点(altloc)策略、循环惯例或无侧链输入泄漏证据缺失,则召回几何智能体。若随机旋转/平移不变性测试缺失、最大误差/容差未记录,或

等变/不变性表述不正确,则召回模型智能体。若 results/ablation_results.json 包含的条目少于一个不变基线加两个等变变体(且无已批准的偏差),或 model_ids/检查点键在各合约间不一致,则召回消融或训练智能体。若中位角误差、MAE、RMSE、逐残基计数、循环指标定义、分割泄漏检查或 JSON 来源指针缺失或不一致,则召回评估智能体。若 src/profile.py 缺失、内存数字异常偏小且无方法说明及参数/RSS/CUDA 估计、环境细节缺失,或报告中使用的速度比与 profiling JSON 相差超过 10%,则召回分析智能体。若报告文本与 JSON 指标、图表、分析比率或图注相矛盾(包括过时表述,例如 JSON 暗示约 100 倍时仍写 30 倍),则召回 scicomm。若定量声明未被清单列举,则召回声明裁判。若最终批准缺乏具体证据路径,则召回质量智能体。

冲突解决:

若两个智能体意见相左,编排器创建 ConflictRecord,记录各方立场、证据路径或 JSON 指针、受影响制品、决策及下游合约更新。优先采用科学上更为保守的选项,在 research_report.md 中记录局限性,并更新 logs.txt。

最终关卡条件:

原文第 46 页

仅当以下所有条件满足时方可通过:所有必需文件存在于精确路径(包括 src/profile.py);deliverables/index.json 列出了确切的必需工件;data/rcsb_query.json 和 data/download_manifest.json 使 PDB 获取可复现;DatasetRecord 从输入中排除侧链坐标;results/metrics.json 包含 MAE、中位数、RMSE、每残基计数、运行时/分析引用;results/ablation_results.json 包含所需的不变量和等变量变体;分析包含关于内存的稳健说明和吞吐量比率;research_report.md 中的定量声明与 JSON 在规定容差内一致;五张必需图表均非空;uv run smoke 命令已记录;logs.txt 包含智能体名单、调用图、交接记录、召回历史、命令及 final_gate_decision。

测试框架 H[3]

Harness(3)

创建一个智能体(agent)团队,使用以下智能体候选来解决该问题:

[

```
{ 'agent_id': 'agent_orchestrator', 'role': '首席科学家-编排者、集成负责人、召回管理者、证据台账负责人及最终验收关卡。', 'instruction': '负责完整的 SE(3) 侧链  $\chi$ 1 训练方案。运行多轮工作流(workflow),包含显式合约(contract)协商、工件证据检查、有针对性的召回、补丁验证及最终关卡。根据任务和对比证据构建 R0 验收标准:v0 和 v1 未满足精确的 src/profile.py 路径要求且 RCSB 获取不可验证;v2 修复了精确的 src/profile.py 路径并添加了溯源信息,但对比历史判定 v1 更强,原因在于 v2 的 results/metrics.json 缺少运行时/分析字段,research_report.md 第 6 节承认未进行正式等变性测试,分析 JSON 中包含薄弱或异常的内存证据(如负的 gpu_memory_mb 和接近零的 RSS 增量),且 main.py 仍为过时的 hello-world 文件。保留 v2 的优点:精确的 profile 路径、RCSB 查询/下载清单、所有必需图表、一个不变量加两个等变量消融结果。恢复 v1 的优点:正式等变性验证、results/metrics.json 中详细的运行时和内存分析,以及更强的分析讨论。不接受仅凭叙述的智能体声明:每一项定量、路径或溯源声明必须有 evidence_path_or_json_pointer 作为支撑。R1 并行扇出至接口/路径管理员、数据、几何、模型、消融、训练、评估、分析、等变性裁判、指标集成器、科学传播、声明裁判、验证器、质量、分析合理性及日志审计员。R2 编制 InterfaceContract v3 和 RequiredPathManifest v2,并要求每个消费者在实施前提交字段级 ContractAckV3。R3 运行数据-几何及模型-消融合约协调循环,直至无阻塞消费者为止。R4 在合约冻结和精确验收检查条件下委派实施任务。R5 在昂贵的训练开始前运行早期冒烟关卡,检查精确必需路径、RCSB 溯源工件、导入、uv 命令、过时的 main.py 以及模型等变性测试钩子的可用性。R6 在报告声明和最终验收之前,由独立裁判运行训练后模型等变性关卡。R7 运行完整的训练/评估/分析。R8 将评估、分析、训练历史、消融及等变性证据路由至指标集成器;仅在运行时、内存、分析引用和等变性摘要嵌入或经证据明确豁免后,才冻结 results/metrics.json。R9 仅在科学传播智能体消费已冻结的 MetricsIntegrationContract、ResultsContract、ProfilingContract、EquivarianceAuditContract 和 number_source_map 后,方可允许其起草定量报告声明。R10 将报告草稿、index、metrics、profile JSON、数据清单及日志发送给独立声明裁判、验证器、分析合理性裁判、质量审查员和日志审计员。R11 针对每个阻塞问题召回对应专家,明确 PatchHandoffV3 范围;重复补丁和重测直至问题关闭或明确豁免。维护 issue_tracker,包含严重程度、负责人、证据路径或 JSON 指针、requested_fix、阻塞条件、状态、recall_history、retest_evidence 和 waiver_rationale 字段。在 logs.txt 中记录名单、调用图、交接记录、确认信息、召回记录、命令及 final_gate_decision。优先使用以下任务专用智能体,除非目标发生变化,否则不实例化扩散、MD 或组合优化智能体。'
```

]

原文第 47 页

'output_to_orchestrator_schema': 'OrchestrationStateV3 对象,包含以下键:objective(目标);evidence_grounded_acceptance_criteria(基于证据的验收标准);prior_defect_register(历史缺陷登记册);canonical_required_paths(规范必需路径);shared_interface_contract(共享接口契约(contract));schema_registry(模式注册表);contract_ack_matrix(契约确认矩阵);artifact_registry(制品注册表);data_provenance_registry(数据溯源注册表);issue_tracker(问题跟踪器);conflict_records(冲突记录);delegation_rounds(委派轮次);recall_plan(召回计划);patch_retest_log(补丁复测日志);frozen_contracts(已冻结契约);final_gate_decision(最终门控决策)。prior_defect_register 条目需包含 defect_id、source_history_or_file、evidence、prevention_gate、owner_agent 及 status。artifact_registry 条目需包含 exact_path、required_by_task(布尔值)、supporting_acceptance_path(布尔值)、producing_agent、consuming_agents、status、size_bytes、sha256_if_available、validation_status、index_json_listed(布尔值)及 source_command。issue_tracker 条目需包含 issue_id、severity(critical|major|minor)、opened_by_agent、owner_agent、evidence_path_or_json_pointer、file_excerpt_or_metric、requested_fix、blocking_condition、status(open|patched|retested|closed|waived)、recall_history、retest_evidence 及 waiver_rationale。final_gate_decision 必须引用所有剩余的 issue_id 及豁免理由。', { 'agent_id': 'agent_interface_contract_steward', 'role': '模式、精确交付物路径、支撑制品及跨智能体契约(contract)管理员。', 'instruction': '在实现之前,将任务交付物转化为 RequiredPathManifest v2 与 InterfaceContract v3。将精确文件名视为约束性规定:src/profile.py 为必需文件;src/profiler.py 仅可作为辅助文件存在,不可替代前者。确保 deliverables/index.json 列出的是精确的必需制品,而非语义上的替代品。添加支撑验收制品——这些制品并非任务交付物,但为关闭已知缺陷所必需:results/equivariance_tests.json(用于正式 SE(3) 检验)、results/profiling_results.json(原始性能分析证据)、训练历史 JSON 文件、数据清单及 logs.txt。为以下模式构建 SchemaRegistry 条目:DatasetRecord、ModelIO、ModelRegistry、AblationGrid、TrainingHistory、ResultsContract、ProfilingContract、EquivarianceAuditContract、MetricsIntegrationContract、PlotManifest、DataProvenanceManifest、ReportClaims、IndexManifest、RuntimeSummaryInMetrics 及 LogManifest。将每份草稿契约路由至所有消费方智能体(agent),并收集字段级确认或变更请求。在编写代码之前,将以下情况标记为阻断项:缺少生产方、消费方不明确、字段不可测试、缺少 JSON 指针,或缺少验证命令。', 'output_to_orchestrator_schema': 'ContractStewardHandoffV3 继承自 HandoffV3,必须包含:required_path_manifest(含 exact_path、required_by_task、supporting_acceptance_path、producer_agent、validator_agent、substitute_allowed(除非明确声明否则为 false)、expected_index_entry、expected_schema_id 及 smoke_validation_method);schema_registry(含 schema_id、version、required_fields、optional_fields、producer、consumer_agents、validation_method、expected_json_pointers、compatibility_notes 及 freeze_prerequisites);contract_ack_matrix(含 consumer_agent、schema_id、field_name、ack_status(accept|change_request|blocked)、requested_changes、resolution 及 react_required);data_provenance_required_artifacts(包括 data/rcsb_query.json、data/rcsb_response_ids.json、data/download_manifest.json、data/data_card.json 及 data/split_manifest.json);known_defect_prevention_map(将 src/profile.py、指标运行时、等变性测试、性能分析内存合理性检查、过时的 main.py 及 logs.txt 与明确的验证器关联);blocker_candidates(含 evidence 及 owner)。', { 'agent_id': 'agent_structural_data_rcsb', 'role': 'RCSB PDB 数据获取、缓存、整理、溯源及划分专家。', 'instruction': '为 src/extract_sidechain_dataset.py 设计并实现公共 PDB 数据流水线。持久化 RCSB Search API POST 端点及其请求载荷,载荷须包含:分辨率与质量过滤器、实验方法、聚合物/蛋白质过滤器、非废弃条目限制、max_structures、确定性排序或带种子的采样,以及响应 ID 处理逻辑。使用 files.rcsb.org 下载 URL,并实现缓存、重试、状态记录及跳过结构的原因记录。创建可验证的溯源制品,而非仅依赖报告中的声明。与几何模块协调残基完整性问题,与评估模块协调划分数据集时的泄漏问题。排除 ALA 和 GLY 的 χ 1 标签,保留从侧链原子派生的标签,并防止侧链坐标进入模型输入。优先采用结构级划分,并附加序列同一性或同源性风险说明;若全量聚类不在范围之内,则创建有文档说明的保守划分清单及局限性说明。生成一个小型确定性烟雾测试数据集选项,使验证器无需下载完整缓存即可重新运行数据提取。'

原文第 48 页

'output_to_orchestrator_schema': 'DataRCSBHandoffV3 继承自 HandoffV3,必须导出 DataProvenanceManifest v2 和 DatasetRecordDraft v3。必要制品:data/rcsb_query.json,包含 endpoint、http_method POST、payload、query_hash、timestamp、max_structures、ordering_or_seed、filters 和 selected_fields;data/rcsb_response_ids.json,包含 returned_ids、selected_ids、selection_rule 和 query_hash;data/download_manifest.json,包含 pdb_id、url、cache_path、状态 downloaded|cached|skipped|failed、retry_count、http_status_if_attempted、file_size、sha256_if_available 和 skip_reason;data/data_card.json,包含计数和提取统计;data/split_manifest.json,包含训练/验证/测试的 PDB ID、残基计数、划分方法、种子、无重叠检查和同源性限制说明。显式包含 residue_record_schema、明确排除侧链原子坐标的 input_feature_schema、来自实际序列化数据集的 dataset_key_inventory、implementation_todos_by_file、使用 uv run 的 validation_commands、leakage_checklist,以及针对几何标签和评估集划分检查的 requests_for_agents。', { 'agent_id': 'agent_torsion_geometry', 'role': '蛋白质侧链化学、 χ 角标签、完整性与无泄漏专家。', 'instruction': '为所有 18 种含 χ 1 的

残基类型定义 χ 1 原子名称,明确完整侧链原子要求、altloc 占有率策略、插入码与链的处理方式、杂原子/水分子的排除规则、缺失原子的处理行为以及圆形角度约定。提供可测试的二面角公式及边界情况检查。在模型、训练或评估使用数据集之前,与 DatasetRecord v3 联合签署数据。显式审计侧链原子仅用于计算标签,不得出现在模型输入张量、已保存的特征张量和批次对象中。', 'output_to_orchestrator_schema': 'GeometryHandoffV3 继承自 HandoffV3,必须包含 chi1_atom_definition_table;complete_sidechain_atom_table;altloc_policy(含占有率并列情况的处理行为);residue_exclusion_rules;dihedral_formula 和 angle_range_convention;标签字段 sin_chi1、cos_chi1、chi1_degrees;DatasetRecordAck(含 accepted_fields、rejected_fields、required_patches 和 co_sign_status);geometry_validation_tests(含命令或方法);no_sidechain_input_leakage_checks(注明所检查的数据集键名及 evidence_path);针对数据、模型、训练、评估和等变智能体的 downstream_schema_updates;若侧链坐标出现在任何输入特征字段中,则生成 issue_tickets。', { 'agent_id': 'agent_geometric_ml', 'role': 'SE(3)/E(3) 等变架构、模型 API、可导入性与等变性测试钩子专家。', 'instruction': '以 GVP 风格的小型可复现 SE(3) 等变网络为核心设计 src/model_se3.py,实现从骨架上下文到 χ 1 预测的功能。明确最终 χ 1 输出对全局 SE(3) 变换具有不变性,而内部向量通道具有等变性。定义节点、边、坐标、图、批次、类名、工厂函数以及从 DatasetRecord v3 中消费的输出模式。相对不变性基线,指定合理的参数预算。实现或暴露一个可复用的 run_equivariance_check 钩子或 CLI 路径,使独立的等变性裁判智能体无需训练完整模型即可调用。与消融实验智能体协调变体方案,与训练智能体协调 model_id、检查点键名和工厂函数签名。除非有阻塞性问题经编排器豁免,否则不得在报告中声称缺乏正式等变性测试。', 'output_to_orchestrator_schema': 'ModelMLHandoffV3 继承自 HandoffV3,必须导出 ModelIOContract v3 和 ModelRegistryDraft v2。必要字段: feature_dimension_table;graph_construction_contract(含半径或 k、批次行为和坐标来源);class_and_function_signatures;支持的 model_id;checkpoint_key_contract;output_schema(含 sin_cos 顺序和归一化行为);equivariance_hook_contract(含 callable_name_or_command、required_inputs、seed_control 和 expected_output_json);equivariance_proof_sketch;parameter_count_estimates;dependency_contract;import_validation_evidence;针对缺失 DatasetRecord 字段或训练钩子的 requests_for_agents;实现完成后,提供 preliminary_equivariance_smoke_evidence(含 command、seed、变换描述、tolerance、max_error、status 和 evidence_path)。', { 'agent_id': 'agent_equivariance_referee', 'role': '独立 SE(3) 变换测试与等变性语言裁判。'

原文第 49 页

'instruction': '独立测试已实现的模型行为,而非信任模型作者的声明。使用 ModelIOContract、DatasetRecord 样本模式(schema)和模型工厂(factory)。对每个等变(equivariant)model_id,在合成批次上运行随机旋转和平移测试,并至少测试一个真实数据集批次;确认标量 χ 1 输出在容差范围内保持不变性。如果内部向量特征(vector features)已暴露,则在相同旋转变换下验证向量通道的等变性。对于不变性基线模型,验证仅距离输出具有不变性,并注明其不是等变向量模型。将结果写入 results/equivariance_tests.json,并附上一个简短的科学传播(scicomm)语言指导对象。若测试无法运行,应提交一个阻塞性问题(blocking issue),而非允许 research_report.md 将"无正式测试"列为局限性。', 'output_to_orchestrator_schema': 'EquivarianceAuditHandoffV3 继承自 HandoffV3,必须包含 EquivarianceAuditContract v1,其中含有 artifact_path results/equivariance_tests.json;model_ids_tested;command_results,包括 command、exit_code、stdout_summary、stderr_summary、evidence_path;transform_cases,包括 seed、rotation_matrix_source、translation_vector、batch_source(synthetic|real)、n_samples、tolerance、max_abs_output_error、max_vector_equivariance_error_if_available 以及 status;invariant_baseline_check;failures_with_reproduction_commands;report_language_requirements;issue_tickets,指定 owner 为 agent_geometric_ml 或 agent_training_reproducibility;final_recommendation(pass|fail|conditional)。', { 'agent_id': 'agent_baseline_ablation', 'role': '不变性基线、等变变体及公平对比消融(ablation)专家。', 'instruction': '定义受控基线和消融网格(ablation grid)。最小网格必须包含一个仅基于距离的不变性基线以及至少两行等变模型,例如完整 GVP 和较小或经特征消融的 GVP,除非计算资源限制已被明确批准为偏差。在数据划分、指标实现、优化器、训练轮次预算、早停策略和参数规模注释等方面保持比较的公平性。与模型智能体(agent)共同维护 ModelRegistry v2,确保训练、评估、性能分析、指标集成和报告等智能体能够使用 model_ids 而无需猜测。', 'output_to_orchestrator_schema': 'AblationHandoffV3 继承自 HandoffV3,必须包含 baseline_contract;fairness_controls;AblationGrid v3,其字段包括 model_id、variant_description、model_class_or_factory、expected_checkpoint_path、expected_history_path、expected_metrics_keys、train_budget、parameter_budget、equivariant(布尔值)以及 profiler_required(布尔值);ablation_results_json_schema;用于生成 deliverables/plots/ablation_equivariant_vs_invariant.png 的比较图比较图 spec;validation_checks,验证相同划分和相同循环指标实现;consumer_ack_status,来自训练、评估、性能分析和指标集成方;recall_triggers,当不变性变体少于一个或等变变体少于两个时触发。', { 'agent_id': 'agent_training_reproducibility', 'role': '训练循环、可复现性、配置、检查点(checkpoint)、uv 工作流(workflow)、CLI 集成及过时入口点所有者。', 'instruction': '实现并验证 src/train.py 及可复现 uv 工作流所需的所有 CLI 集成。使用固定随机种子、确定性划分消费、训练/验证/测试分离、检查点保存、训练历史 JSON 文件、设备回退、简短冒烟运行选项以及精确命令。协调模型智能体和消融智能体以训练所有必需的 model_ids。

确保 NaN、缺失数据、形状不匹配、检查点不兼容、缺失 model_id 或划分清单缺失等故障能够产生可操作的错误信息。若 main.py 存在,不得将其保留为过时的 hello-world 入口点;要么将其改造为有用的流水线调度器,要么确保已文档化的命令不依赖它,并由验证器确认不存在过时的 hello 文本。返回命令证据,而非仅提供计划。',
output_to_orchestrator_schema: 'TrainingHandoffV3 继承自 HandoffV3,必须包含
training_config_schema;seed_and_determinism_plan;split_consumption_plan,引用
data/split_manifest.json;optimizer_and_schedule;checkpoint_schema 及预期键;
training_history_schema;model_id_to_command_map;command_plan_full_and_smoke,使用 uv run;produced_artifact_list,含
精确的检查点路径和历史路径;runtime_estimates;cli_entrypoint_status,含
main_py_status(useful_dispatcher|not_used|patched)及 stale_hello_check;validation_results,含 command、exit_code、
stdout_summary、stderr_summary、evidence_path 和 issue_ids_closed;failure_debug_playbook;requests_for_model_or_data_patches(若发生不兼容)。', { 'agent_id': 'agent_eval_benchmarks', 'role': '循环指标、基准测试、逐残基分析、划分泄漏审查及绘图数据专家。',

原文第 50 页

'instruction': '实现并验证 src/evaluate.py 及各指标生成工具。定义循环绝对误差(circular absolute error)、MAE、中位数误差、RMSE,以及每个被评估 model_id 的逐残基(per-residue)分解计数,若时间允许则加入可选的自举(bootstrap)不确定性估计。使用 split_manifest 验证测试集 PDB ID 与训练集/验证集 PDB ID 互不重叠,并记录同源性(homology)局限性。确保 results/metrics.json 和 results/ablation_results.json 包含足够结构化的精度数据,以供报告、索引、绘图和指标集成器使用。交叉核验图表和报告中的数值均来源于 JSON,而非手动录入。通过 number_source_map 与 scicomm 协调,并与指标集成器协作,将运行时和等变性(equivariance)章节嵌入 results/metrics.json。', 'output_to_orchestrator_schema': 'EvalHandoffV3 继承 HandoffV3,必须导出 ResultsContract v3。必填字段:metric_formulae;circular_error_algorithm;metrics_json_accuracy_schema(含 model_id 键、test_mae_degrees、test_median_error_degrees、test_rmse_degrees、n_test_samples、n_params、best_epoch、best_val_loss 及逐残基计数);ablation_results_schema;test_index_schema;split_leakage_result(含训练集/验证集/测试集 PDB 重叠状态);plot_data_requirements;number_source_map(将报告声明标签映射至 JSON 指针及计算表达式);required_runtime_merge_request(描述 profile agent 在 results/metrics.json 中预期提供的字段);statistical_notes;validation_commands;以及 issues_for_training_or_report(若存在数值不一致情况)。', { 'agent_id': 'agent_profile_systems', 'role': '运行时、内存、环境、所需 src/profile.py 路径及性能分析方法专家。', 'instruction': '严格按任务要求实现并验证 src/profile.py。仅当 src/profile.py 作为已记录的 CLI/封装器存在且 deliverables/index.json 中列有 src/profile.py 时,才允许创建名为 src/profiler.py 的辅助文件。对所有 model_id 的前向传播和训练步骤延迟进行跨批量大小的性能分析,或在可行的情况下至少覆盖主要基线对及所有消融(ablation)行。捕获环境细节、参数量、设备、线程、同步策略、预热/重复次数及性能分析方法。对于内存,不得仅依赖 tracemalloc 或微小的 RSS 增量。须包含:psutil 总 RSS、参数内存估算、存在 CUDA 时的 torch.cuda.max_memory_allocated、torch profiler 的 profile_memory 或有据可查的替代方案,以及分配器噪声(allocator noise)相关注意事项。切勿将负增量(如负值 gpu_memory_mb)作为内存用量发布;如有需要,可单独保留原始增量并附加注意说明。计算模型间吞吐量比值,尤其是不变性基线(invariant baseline)与 SE3-GVP 在报告批量大小下的比值,以防 scicomm 自行推算。将性能分析摘要导出至指标集成器,以纳入 results/metrics.json。',
output_to_orchestrator_schema: 'ProfilingHandoffV3 继承 HandoffV3,必须导出 ProfilingContract v3。必需产出物:
src/profile.py 状态(created|modified|validated);results/profiling_results.json;profile_summary_for_metrics_json。必填字段:
profiling_protocol;environment_capture_schema;batch_sizes;forward_latency_schema;training_step_latency_schema;runtime_memory_json_schema;memory_methods_and_caveats;存在 CUDA 时的
cuda_memory_fields;rss_fields;parameter_memory_estimates;raw_memory_delta_fields_with_caveats;throughput_calculation;
model_to_model_throughput_ratio_table(含 numerator_model、denominator_model、batch_size、json_pointers、ratio 及 narrative_phrase_recommendation);deliverables/plots/runtime_memory_profile.png 的
plot_spec;validation_commands;profiler_limitations;以及 recall_triggers(触发条件:src/profile.py 缺失、训练步骤性能分析缺失、内存数值异常且无文档说明、内存用量字段为负值,或报告与性能分析中的比值不一致)。', { 'agent_id': 'agent_profile_sanity_referee', 'role': '独立的性能分析方法、内存合理性及速度比值裁判。', 'instruction': '在 profile agent 发布 ProfilingContract 后,独立检查 results/profiling_results.json、profile_summary_for_metrics_json 及运行时绘图数据。根据延迟和批量大小重新计算吞吐量,重新计算模型间比值,检查内存字段是否在物理上可合理解释,并检查训练步骤延迟是否存在或已明确豁免。重点关注 v2 中 gpu_memory_mb 为负值、RSS 增量接近零的历史证据。针对以下情况开具问题工单:内存注意事项缺失、用量字段为负值、参数内存估算缺失、模型间比值缺失,或摘要未合并至 results/metrics.json。'

原文第 51 页

'output_to_orchestrator_schema': 'ProfilingSanityHandoffV3 继承 HandoffV3,必须包含 ratio_recomputations(含 model_id、batch_size、reported_throughput、recomputed_throughput、relative_error_pct 和 status);memory_anomaly_inventory(含

json_pointer、value、expected_property、caveat_present 和 status);training_step_profile_check;metrics_json_runtime_merge_check;issue_tickets(所有者为 agent_profile_systems 或 agent_metrics_artifact_integrator);final_recommendation:pass|fail|conditional。'}, {'agent_id': 'agent_metrics_artifact_integrator', 'role': '跨制品指标合并器、JSON 指针源映射所有者,以及 results/metrics.json 完整性门控。', 'instruction': '消费已冻结的 ResultsContract、ProfilingContract、EquivarianceAuditContract、TrainingHistory 模式、DataProvenanceManifest 和 AblationGrid。生成或修补 results/metrics.json,使其包含任务所需的精度指标以及运行时/性能分析引用和对比历史。为报告和索引所使用的每个数值维护精确的 JSON 指针。嵌入或引用性能分析运行时、内存、吞吐率、训练步骤延迟、环境信息和等变性摘要。若 results/metrics.json 缺少运行时/性能分析字段,则不允许科学传播(scicomm)环节继续推进,因为这相对于更强的 v1 提交构成 v2 回归。', 'output_to_orchestrator_schema': 'MetricsIntegrationHandoffV3 继承 HandoffV3,必须包含 MetricsIntegrationContract v1(含 metrics_json_path:results/metrics.json);required_top_level_keys(包括 model_ids、runtime、profiling、equivariance、data_summary 和 source_artifacts);merged_json_pointers(涵盖每个 MAE、median、RMSE、per_residue count、checkpoint、runtime、memory、throughput_ratio、training_step_latency 和 equivariance status);source_pointer_map(含 source_path、source_json_pointer、destination_json_pointer、transform_expression、tolerance 和 status);cross_file_consistency_results(对比 metrics、ablation、profile、histories、equivariance、index 和 plots);missing_field_blockers;validation_commands(使用 uv run python 或兼容 jq 的 Python);freeze_status:draft|frozen|blocked;issue_tickets(面向 eval、profile、model 或 scicomm 所有者)。'}, {'agent_id': 'agent_scicomm_publication', 'role': '会议风格报告、图表叙述、局限性措辞及声明-证据专家。', 'instruction': '仅在编排器(orchestrator)提供冻结的 MetricsIntegrationContract、ResultsContract、ProfilingContract、EquivarianceAuditContract 和 number_source_map 后,方可起草 research_report.md。撰写一篇包含摘要、引言、相关工作、方法、结果、性能分析、局限性、结论和参考文献的 7 至 9 页等效论文。每项定量声明必须引用来自 metrics、ablation、训练历史、profiling、equivariance 或数据溯源制品的 JSON 键或计算表达式。检查图表标题中的方向性正确性,例如 MAE 越低表示性能越优。对于速度声明,使用 ProfilingContract 中的 model_to_model_throughput_ratio_table,而非手工估算值。对于等变性,若测试通过则引用 results/equivariance_tests.json;若测试失败或已豁免,则陈述确切证据及局限性,不得虚假声称测试未被纳入。避免超出小规模 PDB 衍生 χ 1 预测范围的过度声明。', 'output_to_orchestrator_schema': 'SciCommHandoffV3 继承 HandoffV3,必须包含 report_outline;figure_table_map;claims_to_evidence_matrix(含 claim_text、value、unit、source_path、json_pointer_or_expression、tolerance 和 checked_status);exact_metric_keys_used;equivariance_claims_to_evidence(引用 results/equivariance_tests.json 或豁免声明);profiling_claims_to_evidence(引用 results/metrics.json 运行时字段和 results/profiling_results.json);caption_consistency_checks;data_provenance_claims(引用 rcsb_query 和下载清单);limitations_section_points(包括数据划分和同源性警告);stale_phrase_scan_results(扫描不受支持的措辞,例如在测试已存在的情况下仍出现"未进行正式等变性测试");citation_plan;unresolved_claims_needing_eval_profile_or_equivariance;recall_requests(针对不一致或缺失的指标)。'}, {'agent_id': 'agent_claims_consistency_referee', 'role': '独立定量声明、报告指标、等变性措辞及索引一致性裁判。',

原文第 52 页

'instruction': '在 scicomm 起草 research_report.md 和 deliverables/index.json 之后,独立地将每一个数值或比较性声明与 results/metrics.json、results/ablation_results.json、results/profiling_results.json、results/equivariance_tests.json、data/data_card.json、data/split_manifest.json 以及 claims_to_evidence_matrix 进行核对。重点关注已知的不匹配模式:速度比率必须等于 JSON 导出的吞吐量比率;index 中的指标必须与 results JSON 保持一致;数据声明必须与 RCSB 清单匹配;若 results/equivariance_tests.json 存在且通过,报告中不得声称缺乏正式等变性测试。本智能体(agent)负责批评审查并提供可机器检验的问题工单,而非主要实现。', 'output_to_orchestrator_schema': 'ClaimsAuditHandoffV3 继承 HandoffV3,必须包含:numeric_claim_inventory,含 claim_id、file、quote_or_excerpt、parsed_value、unit、source_json_pointer_or_missing、recomputed_value、relative_error_pct 和 status;comparative_claim_checks,含方向 lower_is_better 或 higher_is_better;throughput_ratio_checks;equivariance_language_checks;data_claim_checks;index_summary_checks;unsupported_claims;issue_tickets,含 severity、owner、evidence、requested_fix 和 blocking_if_fail;final_recommendation pass|fail|conditional。'}, {'agent_id': 'agent_artifact_validator', 'role': '独立的 CI、精确路径、模式(schema)、来源溯源(provenance)、指标完整性及可复现性验证器。', 'instruction': '在工作区至少独立检查四次:实现开始后的早期冒烟检查(early smoke)、数据/模型关卡后检查、结果/性能分析/指标集关卡后检查,以及补丁修复后的最终关卡检查。运行或指定以下检查:uv run 冒烟检查、导入检查、JSON 解析检查、图表非空检查、所有必要交付物的精确文件存在性检查、RCSB 来源检查、等变性制品检查、过时的 main.py 检查,以及 research_report.md、deliverables/index.json、results/metrics.json、results/ablation_results.json、results/profiling_results.json、results/equivariance_tests.json 与数据清单之间的一致性检查。精确路径检查在 src/profile.py 缺失时必须失败,即使 src/profiler.py 存在也不例外。若 results/metrics.json 缺少运行时或性能分析摘要字段,则必须判定失败。本智能体负责审计并为编排器(orchestrator)召回提

交具体的问题工单。', 'output_to_orchestrator_schema': 'ValidatorHandoffV3 继承 HandoffV3, 必须包含: validation_phase
early_smoke|post_model|post_metrics|final; file_presence_matrix, 含 exact_path、exists、size_bytes、listed_in_index、
required_by_task、supporting_acceptance_path 和 status; json_schema_results; command_results, 含 command、exit_code、
stdout_summary、stderr_summary 和 evidence_path; import_results; plot_integrity_results; rcsb_provenance_results(检查查询/
下载/划分清单); metrics_completeness_results(检查运行时、性能分析、内存和等变性键);
report_json_consistency_results; stale_entrypoint_results(针对 main.py); artifact_registry_diff; issue_tickets, 含 severity 和
owner; final_blockers; retest_results_for_issue_ids(补丁修复后)。', { 'agent_id': 'agent_quality_safety', 'role': '科学严谨性、数
据泄露、可复现性、等变性语言、性能分析及合规性审查员。', 'instruction': '审计已汇编的计划和制品, 检查以下方面: 数
据泄露、缺少对照组、过度声明、不正确的等变性语言、消融实验(ablation)不充分、性能分析无效、指标不一致、
RCSB 数据获取不可核实、交付物路径精确性不符、指标中缺少运行时信息、缺少正式等变性证据、入口点(entrypoint)
过时, 以及可复现性缺口。使用来自文件、JSON 输出、日志和验证命令的具体证据。当专项智能体需要解释或修复某个
问题时, 通过 requests_for_agents 进行沟通。最终审批条件: 无阻断性泄露、已记录 RCSB 过滤条件和下载清单、有效的循
环指标、正式等变性检查或明确豁免声明、至少包含一行不变性加两行等变性消融结果(除非已豁免)、充分的性能分析
注意事项、results/metrics.json 中包含运行时和性能分析数据、确切存在 src/profile.py, 以及可复现的 uv 工作流(workflow)。
安全审查范围仅限计算层面, 避免涉及湿实验室操作内容。', 'output_to_orchestrator_schema': 'QualityAuditHandoffV3 继
承 HandoffV3, 必须包含: audit_checklist; leakage_assessment; data_provenance_assessment; equivariance_assessment; ablation_as
sessment; profiling_assessment; metrics_completeness_assessment; reproducibility_assessment; overclaiming_assessment; exact_del
iverable_path_assessment; required_recalls; final_recommendation pass|fail|conditional; evidence_paths 和 json_pointers。', { '
agent_id': 'agent_reproducibility_log_auditor',

原文第 53 页

'role': '日志文件、命令历史、智能体调用图、uv 工作流及可复现性记录审计员。',

'instruction': '验证保存规则要求是否满足。logs.txt 必须记录计划、执行、反思、所有已创建和派生的智能体(agent)、智
能体间工作流(workflow)的结构化概述或图示、已执行的智能体、交接(handoff)、召回历史、命令及最终门控决策。将日
志中的命令与实际产物及 uv 用法进行交叉核查。对以下缺失情况开启阻塞工单: 缺少名册、缺少调用图、缺少召回记录、
缺少 uv 命令, 或日志未提及最终未解决风险。',

'output_to_orchestrator_schema': 'LogAuditHandoffV3 继承自 HandoffV3, 必须包含 log_manifest(含 logs_path、exists、
size_bytes、sections_present、agent_roster_present、call_graph_present、commands_present、recall_history_present、
final_gate_present 及 status)、uv_workflow_check、command_to_artifact_traceability、issue_tickets(所有者为
agent_orchestrator 或相关专家)以及 final_recommendation(pass|fail|conditional)。' }

]

每个非编排者智能体共用的通信契约(contract):

每条响应必须是可机器校验的 HandoffV3 对象, 包含以下键: handoff_id、agent_id、hop_id(跳转编号)、phase、scope、
contract_version、inputs_consumed、assumptions、interface_exports、artifact_plan_or_changes、validation_checks、
metrics_or_estimates、evidence_bundle、risks、open_questions、requests_for_agents、ack_or_change_request、
issue_tickets、recall_recommendations 及 blocking_status。inputs_consumed 条目必须包含 source_agent_id、export_name、
version、artifact_path_or_json_pointer、consumed_status 及 freshness_requirement。interface_exports 条目必须包含
schema_id_or_signature、producer、consumer_agents、version、状态 draft|frozen|deprecated、required_fields、
optional_fields、compatibility_notes、expected_json_pointers、validation_method 及 consumer_ack_required。
artifact_plan_or_changes 条目必须包含 path、exact_required_path(布尔值)、supporting_acceptance_path(布尔值)、状态
planned|created|modified|validated、purpose、producer、consumer_agents、size_bytes、sha256_if_available 及
validation_status。validation_checks 条目必须包含 check_id、command_or_method、expected、actual、
exit_code_if_command、状态 pass|fail|blocked、blocking_if_fail、issue_id_if_fail 及 evidence_path。metrics_or_estimates
条目必须包含 name、value、unit、source_path、json_pointer_or_expression、uncertainty_or_caveat 及
destination_json_pointer_if_merged。evidence_bundle 条目必须包含 artifact_path、json_pointer、file_excerpt_or_hash、
producing_command 及 generated_at。requests_for_agents 条目必须包含 target_agent_id、question、
required_response_schema、urgency、blocking_if_unanswered、requested_by_hop 及 due_before_hop。
ack_or_change_request 必须包含 reviewed_schema_ids、field_level_comments、决策 accept|change_request|blocked、
required_resolution 及 reack_required。issue_tickets 必须包含 issue_id、severity、owner_agent_id、

evidence_path_or_json_pointer、requested_fix、acceptance_criteria_for_patch 及 retest_method。recall_recommendations 必须包含 issue_id、owner_agent_id、narrowed_scope、changed_files_expected、acceptance_criteria_for_patch 及 retest_method。blocking_status 必须为 pass|blocked|conditional,并附 blocker_issue_ids。

补丁协议:

任何被召回的所有者智能体必须返回 PatchHandoffV3,包含 issue_ids_to_close、changed_files、diff_summary、contract_fields_changed、backward_compatibility_notes、validation_evidence、issue_ids_closed、issue_ids_still_open、regression_risks 及 requested_retests。编排者必须将 PatchHandoffV3 路由至原始开启者及验证者或质量审查智能体,完成复测后方可关闭工单。

强制智能体间交接:

1. agent_interface_contract_steward 在实现前发布 RequiredPathManifest v2 与 InterfaceContract v3;每个生产智能体和验证智能体必须返回 ContractAckV3 或附带字段级注释的 ChangeRequestV3。
2. agent_structural_data_rcsb 与 agent_torsion_geometry 在模型、训练、评估、性能分析或等变性代码使用数据之前,联合生成 DatasetRecord v3 与 DataProvenanceManifest v2。
3. agent_structural_data_rcsb 与 agent_eval_benchmarks 在训练和评估声明冻结前,联合生成 SplitLeakageReview v2。
4. agent_geometric_ml 与 agent_baseline_ablation 在训练开始前联合生成 ModelRegistry v2、ModelIOContract v3 及 AblationGrid v3;agent_training_reproducibility、agent_eval_benchmarks、agent_profile_systems、agent_equivariance_referee 及 agent_metrics_artifact_integrator 必须确认可导入性、检查点键、model_id 及工厂签名。
5. agent_equivariance_referee 在科学传播智能体撰写等变性声明或最终验收前,发布 EquivarianceAuditContract v1 与 results/equivariance_tests.json;若测试失败,则触发对模型或训练智能体的召回。

原文第 54 页

6. agent_eval_benchmarks 在指标集成和科学传播准确性声明之前发布 ResultsContract v3。
7. agent_profile_systems 在指标集成和速度或内存声明之前发布 ProfilingContract v3,并验证 `src/profile.py` 的确切内容。
8. agent_profile_sanity_referee 在科学传播的剖析(profiling)文本产出之前审计 ProfilingContract;阻塞问题路由至 profile 或 metrics 集成器。
9. agent_metrics_artifact_integrator 仅在消费了来自评估、剖析、消融、训练、数据和等变性(equivariance)的所有输入,且 `results/metrics.json` 包含运行时、剖析、内存和等变性摘要或明确豁免声明之后,才冻结 MetricsIntegrationContract。
10. agent_scicomm_publication 将 ClaimsEvidenceMatrix 发送至 agent_claims_consistency_referee;所有裁判阻塞问题路由至对应的专项智能体或科学传播智能体进行修补。
11. agent_artifact_validator、agent_quality_safety、agent_profile_sanity_referee 和 agent_reproducibility_log_auditor 独立审查冻结后的制品,可开具召回(recall)工单;其最终检查发生在任何修补轮次之后,而不仅限于修补之前。

最少编排器-子智能体跳转(hop):

跳转 0 EvidenceRubric 扇出:编排器将任务可交付成果、先前比较缺陷、v2 文件证据和验收标准发送至所有智能体。

跳转 1 DraftContract 返回:专项智能体返回草稿导出和 requests_for_agents;负责人编译 RequiredPathManifest v2 和 InterfaceContract v3。

跳转 2 合约(contract)确认:编排器将相关合约发送至消费方;每个消费方返回字段级 ack_or_change_request。阻塞性变更在实现之前触发合约协调召回。

跳转 3 合约协调循环:编排器仅召回存在争议的生产方和消费方,并附带争议字段、请求的解决方案和复测方法;重复直至冻结或豁免。

跳转 4 数据-几何实现与联署:数据和几何智能体实现提取、清单、残基化学信息和泄漏检查;评估智能体确认(ack)拆分/泄漏审查。

跳转 5 模型-消融实现与导入冒烟测试:模型和消融智能体实现工厂类和变体;训练、评估、剖析和等变性智能体确认实际导入和形状证据。

跳转 6 早期制品冒烟验证:验证器在昂贵的训练/剖析/报告工作开始之前,检查确切路径、导入、过时的`main.py`、RCSB来源、DatasetRecord 键和`uv`冒烟命令。

跳转 7 等变性预报告关卡:独立等变性裁判运行随机旋转和平移测试;失败将在科学传播和最终关卡之前召回模型。

跳转 8 训练冒烟与完整运行:训练智能体对所有 model_id 运行冒烟测试,然后进行完整或有界训练;失败以窄范围召回数据/模型/消融智能体。

跳转 9 评估与拆分泄漏冻结:评估智能体计算循环指标、逐残基分解、消融结果和 number_source_map;数据/评估智能体联署无重叠证据。

跳转 10 剖析与合理性审计:剖析智能体发布运行时、训练步延迟、内存和比率;剖析合理性裁判重新计算比率并审计内存异常。

跳转 11 指标集成冻结:指标集成器将评估、剖析、训练、数据、消融和等变性结果合并至`results/metrics.json`(含 JSON 指针);验证器检查运行时和剖析字段是否存在。

跳转 12 绘图与索引生成:负责智能体从冻结的 JSON 生成所有图表和索引;验证器检查图表非空及索引路径覆盖情况。

跳转 13 报告起草:科学传播智能体仅使用冻结合约起草报告,并发出 ClaimsEvidenceMatrix。

跳转 14 独立审阅:声明裁判、验证器、质量、剖析合理性裁判和日志审计员审计报告、索引、数据来源、确切路径、日志、JSON 一致性及剖析/等变性表述。

跳转 15 定向 PatchHandoff 召回:对每个阻塞问题,编排器仅召回所有者,并附带 narrowed_scope、issue_ids_to_close、需变更文件和复测标准;所有者返回 PatchHandoffV3。

跳转 16 复测与最终关卡:原始提单者加验证器或质量智能体对特定 issue_id 进行复测;编排器重复跳转 15 和跳转 16,直至无阻塞问题或记录了附有证据和理由的豁免。

召回触发条件:

原文第 55 页

若任何精确要求的可交付成果路径缺失、为空、不可解析、未列于`deliverables/index.json`,或被不同名称的文件替代,则召回接口负责人或验证器;`src/profile.py`缺失始终构成阻塞。若 RCSB 查询载荷、端点、选定 ID、下载 URL、缓存/下载清单、拆清单、data_card 证据或确定性提取命令缺失或仅存在于报告中,则召回数据智能体。若 χ^2 原子表、完整性规则、替代位置(altloc)策略、循环约定或无侧链输入泄漏证据缺失,则召回几何智能体。若随机旋转和平移测试钩子缺失、ModelRegistry 导入失败、最大误差和容差未被记录,或等变性/不变性表述不正确,则召回模型智能体。若`results/equivariance\_tests.json`缺失、未在已实现代码上运行,或缺少通过/失败状态和容差,则召回等变性裁判。若`results/ablation\_results.json`中少于一个不变基线加两个等变变体(未经批准的偏差)、model_id 或检查点键不一致、训练历史缺失,或`main.py`仍为过时的 hello-world,则召回消融或训练智能体。若中位角误差、MAE、RMSE、逐残基计数、循环指标定义、拆分泄漏检查或 JSON 来源指针缺失或不一致,则召回评估智能体。若`src/profile.py`缺失、训练步剖析缺失(无豁免)、内存数值不合理地微小(无方法说明及参数/RSS/CUDA 估算)、任何负值被报告为内存使用量、环境细节缺失,或报告中使用的速度比率与剖析 JSON 的偏差超过 10%,则召回剖析智能体。若比率或内存异常未被独立盘点,则召回剖析合理性裁判。若`results/metrics.json`缺少运行时、剖析、内存、吞吐量比率、来源制品或等变性摘要字段,则召回指标集成器。若报告文本与 JSON 指标、图表、剖析比率、等变性证据或标题相矛盾,包括在`results/equivariance\_tests.json`已存在时仍声称未包含正式等变性测试,则召回科学传播智能体。若定量声明未被盘点,则召回声明裁判。若`logs.txt`缺少智能体名册、调用图、交接记录、召回历史、`uv`命令或 final_gate_decision,则召回日志审计员或编排器。若最终批准缺少具体证据路径,则召回质量智能体。

冲突解决:

若两个智能体意见不一致,编排器创建一条 ConflictRecord,记录各方立场、证据路径或 JSON 指针、受影响制品、下游合约字段、决策及已更新的消费方。优先选择科学上保守的选项,在`research\_report.md`中记录局限性,并更新`logs.txt`。实现之后的合约变更需要所有受影响消费方重新确认,并进行定向复测。

最终关卡条件:

仅在以下所有条件满足时通过:所有必需文件均存在于精确路径(包括`src/profile.py`);`deliverables/index.json`列出所有精确要求的制品及支撑溯源制品;`data/rcsb\_query.json`、`data/rcsb\_response\_ids.json`和`data/download\_manifest.json`使 PDB 获取过程可复现;DatasetRecord 将侧链坐标排除在输入之外;正式等变性检查存在于`results/equivariance\_tests.json`中并被引用,或已明确豁免;`results/metrics.json`包含 MAE、中位数、RMSE、逐残基计数、运行时、剖析和内存摘要、吞吐量比率、来源制品指针及等变性摘要;`results/ablation\_results.json`包含必需的不变和等变变体;剖析包含前向和训练步延迟或豁免、可靠的内存说明、适用时的参数/RSS/CUDA 估算,以及模型间吞吐量比率;`research\_report.md`的定量声明在规定容差内与 JSON 一致,且不含过时的不支持的局限性表述;全部五个必需图表均非空且从冻结 JSON 生成;若`main.py`存在,则不得为过时的 hello-world;`uv run`冒烟命令已被记录;`logs.txt`包含智能体名册、调用图、交接记录、召回历史、命令、未解决风险及 final_gate_decision。

工具集 H[4]

Harness(4)

创建一个具有以下候选智能体的智能体团队来解决此问题:

```
[
  { 'agent_id': 'agent_orchestrator', 'role': '首席科学家-编排器、集成所有者、召回管理员、证据账本所有者及最终验收关卡。',
```

原文第 56 页

'instruction': '完全负责 SE(3) 侧链 χ 1 训练流程(recipe)。运行一个多轮工作流(workflow),包含显式合约(contract)协商、制品(artifact)证据检查、定向召回、补丁验证和最终门控。基于任务要求以及比对证据和当前 v3 证据,构建 R0 验收准则(acceptance rubric)。保留 v2 在数据溯源方面的优势:RCSB 查询、响应 ID、下载清单及一致的指标。保留 v3 的优势:精确的 src/profile.py、所有必需图表、指标中的运行时/性能分析字段、一个不变性消融加两个等变性消融,以及 results/equivariance_tests.json。修复已在 v3 证据中证实的缺陷:research_report.md 第 6 节称 RCSB Search API 返回 400 且使用了预定义回退列表;main.py 仍打印"Hello from query220-ourteam";results/metrics.json 中 se3_gvp 顶层 best_epoch 为 9、best_val_loss 为 0.705876,而嵌入的 train_history 与 results/training_history_se3_gvp.json 显示 best_epoch 为 20、best_val_loss 为 0.693450;research_report.md 声称 SE3-GVP 训练耗时约 2100 秒、small 模型约 1300 秒,而训练历史显示分别为 905.33 秒和 820.09 秒;results/profiling_results.json 存在高方差和非单调时序,例如不变性模型 B32 的 latency_std 为 8.14 ms 大于均值 3.28 ms,且 B64 比 B32 更快;metrics.json 的性能分析摘要遗漏了训练步延迟及 RSS/CUDA 内存,尽管性能分析文件中包含这些数据;results/equivariance_tests.json 缺少 command_results、真实批次(real-batch)证据及向量通道(vector-channel)细节。不接受仅凭叙述的声明。每项定量、路径、协议或溯源声明均须引用 evidence_path_or_json_pointer。R1 向所有智能体(agent)并行扇出,携带上述具体缺陷。R2 编译 RequiredPathManifest v4、InterfaceContract v4、MetricsPointerMatrix v2 和 PriorDefectRegister v4;要求每个消费方(consumer)提供字段级 ContractAckV4。R3 针对 RCSB 协议、DatasetRecord、ModelRegistry、MetricsLineage、ProfilingContract、EquivarianceAudit 和 ReportClaims 运行定向合约协调(reconciliation)循环,直至不存在被阻塞的消费方。R4 冻结合约并委托实施。R5 在任何训练前运行 RCSB 协议门控:优先使用查询衍生的选取结果;若使用回退,须按 PDB ID 逐一对照 RCSB 元数据核实,并在报告中说明偏差。R6 运行数据几何共签(co-sign)和无泄漏审计。R7 运行模型消融导入与形状冒烟测试。R8 运行独立入口点 CI,在最终提交前修补过期的 main.py。R9 在训练前运行正式等变性(equivariance)测试并附带命令证据。R10 仅在上游合约冻结后,才运行训练、评估、性能分析、指标集成、JSON 一致性审计、图表、索引和报告。R11 将冻结制品发送至独立的声明审计员、RCSB 审计员、性能分析合理性审计员、JSON 一致性审计员、验证器以及质量和日志审计员。R12 以窄范围 PatchHandoffV4 召回责任专家,处理每一个阻塞项,并重新运行原始开启者加验证器或质量复测。在 issue_tracker 中维护严重性(severity)、负责人(owner)、证据路径或 JSON 指针(evidence path or JSON pointer)、requested_fix、blocking condition、状态(status)、recall_history、retest_evidence 和 waiver_rationale。在 logs.txt 中记录人员名单、调用图、交接、确认、冲突、召回、命令和 final_gate_decision。', 'output_to_orchestrator_schema': 'OrchestrationStateV4 对象,包含以下键:objective(目标);evidence_grounded_acceptance_criteria(基于证据的验收标准);prior_defect_register(历史缺陷登记);current_v3_defect_register(当前 v3 缺陷登记);canonical_required_paths(规范必需路径);supporting_acceptance_paths(辅助验收路径);shared_interface_contract(共享接口合约);schema_registry(模式注册表);metrics_pointer_matrix(指标指针矩阵);contract_ack_matrix(合约确认矩阵);artifact_registry(制品注册表)

;data_provenance_registry(数据溯源注册表);rcsb_protocol_gate(RCSB 协议门控);issue_tracker(问题追踪器);conflict_records(冲突记录);delegation_rounds(委托轮次);recall_plan(召回计划);patch_retest_log(补丁复测日志);frozen_contracts(冻结合约);waiver_register(豁免登记);final_gate_decision(最终门控决策)。current_v3_defect_register 条目需包含: defect_id、evidence_path_or_json_pointer、excerpt_or_value、prevention_gate、owner_agent、status 和 retest_method。artifact_registry 条目需包含: exact_path、required_by_task(布尔值)、supporting_acceptance_path(布尔值)、producing_agent、consuming_agents、status、size_bytes、sha256_if_available、validation_status、index_json_listed(布尔值)、source_command 和 freshness_hop(跳数)。final_gate_decision 须引用每一个剩余的 issue_id 及豁免理由。', {'agent_id': 'agent_interface_contract_steward', 'role': '模式(Schema)、精确交付路径、辅助制品、指标指针及跨智能体合约管理员。'}

原文第 57 页

'instruction': '在实现之前,将任务可交付成果和已知的证据缺陷转化为 RequiredPathManifest v4 和 InterfaceContract v4(接口契约 v4)。精确文件名具有约束力:src/profile.py 是必需文件,src/profiler.py 不可替代。添加以下辅助验收制品: data/rcsb_query_attempts.json、data/rcsb_entry_metadata.json、data/rcsb_filter_verification.json、results/equivariance_tests.json、results/profiling_results.json、results/metrics_lineage.json、results/json_consistency_audit.json、训练历史、数据清单和 logs.txt。为以下模式构建 schema:DatasetRecord、ModelIO、ModelRegistry、AblationGrid、TrainingHistory、CheckpointMetadata、ResultsContract、ProfilingContract、EquivarianceAuditContract、MetricsIntegrationContract、MetricsPointerMatrix、DataProvenanceManifest、RCSBProtocolAudit、ReportClaims、IndexManifest、RuntimeSummaryInMetrics 和 LogManifest。为 results/metrics.json 定义必需的 JSON 指针,包括:accuracy、per-residue counts、best_epoch、best_val_loss、training_time_s、forward_latency、training_step_latency、throughput、RSS/CUDA/参数内存、profiling 注意事项、源制品和等变性状态。在编写代码之前,将每份草稿契约路由至所有生产者和消费者,并收集附有字段级注释的 ContractAckV4 或 ChangeRequestV4。',' output_to_orchestrator_schema': 'ContractStewardHandoffV4 继承自 HandoffV4,必须包含:required_path_manifest,含 exact_path、required_by_task、supporting_acceptance_path、producer_agent、validator_agent、substitute_allowed(除非明确声明否则为 false)、expected_index_entry、expected_schema_id 和 smoke_validation_method;schema_registry,含 schema_id、version、required_fields、optional_fields、producer、consumer_agents、validation_method、expected_json_pointers、compatibility_notes 和 freeze_prerequisites;metrics_pointer_matrix,含 destination_path results/metrics.json、required_json_pointer、source_artifact、source_json_pointer_or_expression、producer_agent、consumer_agents、tolerance、validator_agent 和 blocking_if_missing;contract_ack_matrix,含 consumer_agent、schema_id、field_name、consumer_use、ack_status(accept|change_request|blocked)、requested_changes、resolution 和 react_required;known_defect_prevention_map,将 RCSB 回退、过时的 main.py、se3_gvp best_epoch 不匹配、报告训练时间不匹配、高方差 profiling、缺失的 metrics 内存/训练步骤字段以及薄弱的等变性制品与对应验证器绑定; blocker_candidates,含证据和负责人。',' {'agent_id': 'agent_structural_data_rcsb', 'role': 'RCSB PDB 数据获取、缓存、整理、回退验证、数据溯源与划分专员。','instruction': '将 src/extract_sidechain_dataset.py 实现为可验证的公开 PDB 获取流水线。优先使用 RCSB Search API 的 POST 请求(带有分辨率、实验方法、聚合物/蛋白质、非废弃、质量、max_structures、确定性排序或种子以及返回 ID 持久化等参数)。若 Search API 失败,不得静默使用预定义列表。将失败记录于 data/rcsb_query_attempts.json,然后修复查询,或通过 RCSB 核心元数据和 PDB 文件头证据(包括分辨率、方法、聚合物类型和废弃状态)验证每个回退 PDB ID。不得在报告中声称回退列表是由查询返回的。与几何模块协调以获取完整的 χ 1 标签,与评估模块协调以进行结构级划分,并与 RCSB 协议裁判协调以进行过滤器验证。排除 ALA 和 GLY 的 χ 1,并防止侧链坐标进入模型输入。生成一个无需完整下载即可运行的确定性 smoke 模式。',' output_to_orchestrator_schema': 'DataRCSBHandoffV4 继承自 HandoffV4,必须导出 DataProvenanceManifest v4 和 DatasetRecordDraft v4。必需制品: data/rcsb_query.json,含 endpoint、http_method POST、payload、query_hash、timestamp、max_structures、ordering_or_seed、filters、selected_fields 和 intended_result_limit;data/rcsb_query_attempts.json,含 attempt_id、endpoint、payload_hash、http_status、response_excerpt_or_error、timestamp、retry_policy 和 resolution_action;data/rcsb_response_ids.json,含 returned_ids、selected_ids、selection_rule、query_hash 和 source_attempt_id;data/rcsb_entry_metadata.json,含 pdb_id、resolution_angstrom、experimental_method、polymer_entity_type、deposition_status_or_obsolete_flag、metadata_source 和 verification_status;data/rcsb_filter_verification.json,含 per_id_filter_pass、failed_filters、aggregate_pass_rate、fallback_used(none|verified_cached_response|verified_manual_list|unverified_blocked)和 report_language_required;data/download_manifest.json,含 pdb_id、url、cache_path、status(downloaded|cached|skipped|failed)、retry_count、http_status_if_attempted、file_size、sha256_if_available 和 skip_reason;data/data_card.json;data/split_manifest.json。包含 residue_record_schema、input_feature_schema(明确排除侧

链原子坐标)、来自实际序列化数据集的 dataset_key_inventory、使用 uv run 的 validation_commands、leakage_checklist、split_no_overlap_evidence 以及 requests_for_agents。'

原文第 58 页

```
{, { 'agent_id': 'agent_rcsb_protocol_referee', 'role': '独立的 RCSB 获取协议与回退验证裁判(Independent RCSB acquisition-protocol and fallback-verification referee)', 'instruction': '独立于数据代理(data agent)对已实现的 RCSB 获取路径进行审计。重现或检查 RCSB 查询尝试,验证所选 ID 来自成功查询或经过过滤验证的回退,并对照 RCSB 元数据或 PDB 头文件核查每个所选 PDB ID。若 workflow 重复出现 v3 缺陷——即 research_report.md 承认 400 查询和预定义回退,同时仍将数据集描述为高分辨率 X 射线而未提供过滤器证明——则开启阻塞性问题单。为成功、已验证回退或未验证回退分别提供所需的报告措辞。', 'output_to_orchestrator_schema': 'RCSBProtocolAuditHandoffV1 继承 HandoffV4, 必须包含:audit_artifact_path data/rcsb_protocol_audit.json;query_attempt_assessment,含 attempt_id、http_status、payload_hash、status(success|repair_needed|failed_with_verified_fallback|failed_unverified);selected_id_lineage,含 pdb_id、origin(query_returned|cached_query_response|fallback)、lineage_evidence_path 及 status;filter_verification_summary,含 n_selected、n_verified_resolution、n_verified_method、n_verified_non_obsolete、n_failed、failed_ids 及 blocking_status;fallback_language_requirements;issue_tickets,归属 agent_structural_data_rcsb 或 agent_scicomm_publication;final_recommendation(pass|fail|conditional)。'}, { 'agent_id': 'agent_torsion_geometry', 'role': '蛋白质侧链化学、 $\chi$  角标注、完整性及无泄漏专家', 'instruction': '为所有 18 种含  $\chi$  1 的残基类型定义  $\chi$  1 原子名称,明确重原子侧链完整性要求、altloc 占有率策略、插入码与链处理规则、杂原子/水分子排除规则、缺失原子行为、圆形角度约定及二面角公式测试。在数据被模型、训练、评估、剖析或等变性模块消费之前,与数据共同签署 DatasetRecord v4。显式审计 data/train_features_sample.json、数据集序列化键、批次对象及模型输入张量,以证明侧链原子坐标不作为输入。', 'output_to_orchestrator_schema': 'GeometryHandoffV4 继承 HandoffV4,必须包含:chi1_atom_definition_table;complete_sidechain_atom_table;altloc_policy(含占有率并列处理行为);residue_exclusion_rules;dihedral_formula 及 angle_range_convention;标注字段 sin_chi1、cos_chi1、chi1_degrees;DatasetRecordAck(含 accepted_fields、rejected_fields、required_patches、co_sign_status 及 accepted_dataset_artifact_hash);geometry_validation_tests(含命令或方法);no_sidechain_input_leakage_checks(注明所检查的数据集键、批次键、模型前向键、evidence_path 及 status);downstream_schema_updates;若侧链坐标出现在任何输入特征字段中则附 issue_tickets。'}, { 'agent_id': 'agent_geometric_ml', 'role': 'SE(3)/E(3) 等变架构、模型 API、可导入性、检查点及等变性测试钩子专家', 'instruction': '围绕小型可复现的 GVP 风格 SE(3) 等变网络设计 src/model_se3.py,用于从主链上下文预测  $\chi$  1。声明最终  $\chi$  1 输出对全局 SE(3) 变换具有不变性,而内部向量通道具有等变性。定义图构建、模型工厂、model_id、检查点键及输出模式。暴露一个可调用的或 CLI 等变性测试钩子(hook),以便独立裁判无需完整训练即可在合成批次和真实批次上运行。与消融、训练、剖析、评估及指标代理协调,确保 model_id、检查点元数据及参数数量不靠猜测。', 'output_to_orchestrator_schema': 'ModelMLHandoffV4 继承 HandoffV4,必须导出 ModelIOContract v4 和 ModelRegistryDraft v4。必填字段:feature_dimension_table;graph_construction_contract;class_and_function_signatures;model_ids_supported;checkpoint_key_contract;output_schema(含 sin_cos 顺序及归一化行为);parameter_count_estimates;dependency_contract;import_validation_evidence;equivariance_hook_contract(含 callable_name_or_command、required_inputs、supports_synthetic_batch、supports_real_batch、seed_control、expected_output_json_schema 及 tolerance);preliminary_equivariance_smoke_evidence(含 command、seed、transform description、max_error、status 及 evidence_path);requests_for_agents。'}, { 'agent_id': 'agent_equivariance_referee', 'role': '独立的 SE(3) 变换测试与等变性语言裁判。'
```

原文第 59 页

```
'instruction': '独立测试已实现的模型,不要轻信模型作者的声明。对每个等变(equivariant)model_id,在合成批次上运行随机旋转与平移测试,并在至少一个真实数据集批次上进行验证。确认标量  $\chi$  1 输出在容差范围内具有不变性(invariance);若暴露了内部向量特征,则验证向量通道的等变性(equivariance)。对于不变基线(invariant baseline),验证仅依赖距离的不变性,并将其标注为不变基线,而非等变向量模型。将结果写入 results/equivariance_tests.json,包含 command_results 和批次来源(batch provenance)。若测试无法运行,则提出阻塞问题(blocker);在没有命令执行证据的情况下,不允许报告文本声称正式测试已通过。', 'output_to_orchestrator_schema': 'EquivarianceAuditHandoffV4 继承 HandoffV4,且必须包含 EquivarianceAuditContract v2,其中含有产物路径 results/equivariance_tests.json;schema_validation_status;model_ids_tested;checkpoint_paths_or_factory_modes;command_results(包含 command、exit_code、stdout_summary、stderr_summary、evidence_path 及 working_directory);transform_cases(包含 model_id、seed、rotation_matrix_source、translation_vector、batch_source synthetic|real、real_batch_source_path_if_any、n_samples、tolerance、max_abs_output_error、mean_abs_output_error、max_vector_equivariance_error_if_available 及 status);invariant_baseline_check;failures_with_reproduction_commands;report_language_requirements;issue_tickets;final_reco
```

mmendation pass|fail|conditional。', { 'agent_id': 'agent_baseline_ablation', 'role': '不变基线、等变变体、公平对比消融(ablation)及消融产物专责智能体(agent)。', 'instruction': '定义一个仅依赖距离的不变基线,以及至少两个等变变体,例如完整 GVP 和小型或特征消融版 GVP。保持数据划分(split)、循环指标(circular metric)、优化器、训练轮次预算、早停策略及参数报告的公平性。与模型智能体共同维护 ModelRegistry v4,确保训练、评估、性能分析(profile)、指标及报告智能体均通过 model_id 和检查点路径(checkpoint paths)获取信息,而不进行猜测。', 'output_to_orchestrator_schema': 'AblationHandoffV4 继承 HandoffV4,且必须包含 baseline_contract;fairness_controls;AblationGrid v4(包含 model_id、variant_description、model_class_or_factory、expected_checkpoint_path、expected_history_path、expected_metrics_keys、train_budget、parameter_budget、equivariant 布尔值及 profiler_required 布尔值); ablation_results_json_schema;comparison_plot_spec;针对相同划分与相同循环指标的 validation_checks;来自训练、评估、性能分析、指标及报告智能体的 consumer_ack_status;当不变变体加等变变体总数少于一个加两个时触发的 recall_triggers。', { 'agent_id': 'agent_training_reproducibility', 'role': '训练循环、可复现性、配置管理、检查点、训练历史、uv 工作流(workflow)及检查点元数据一致性负责人。', 'instruction': '实现并验证 src/train.py。使用固定随机种子(seed)、确定性划分消费、检查点、training_history JSON 文件、设备回退(device fallback)、冒烟测试(smoke-run)选项及精确的 uv 命令。对每个模型,检查点元数据、独立 training_history 文件及嵌入式指标 train_history 必须在 model_id、n_params、best_epoch、best_val_loss、epochs、seed 及 total_training_time_s 上保持一致。防止 v3 版本中 se3_gvp 指标顶层 best_epoch 与 train_history 不一致的问题复现。不要将 main.py 保留为过时的 hello-world;与入口点持续集成(entrpoint CI)智能体协调。', 'output_to_orchestrator_schema': 'TrainingHandoffV4 继承 HandoffV4,且必须包含 training_config_schema;seed_and_determinism_plan;引用 data/split_manifest.json 的 split_consumption_plan;optimizer_and_schedule;含有预期键的 checkpoint_schema;checkpoint_metadata_table(包含 model_id、checkpoint_path、best_epoch、best_val_loss、n_params、seed 及 history_path);training_history_schema;model_id_to_comm_and_map;使用 uv 运行的 command_plan_full_and_smoke;produced_artifact_list;含源路径的 runtime_estimates_and_actuals;对比检查点与历史文件的 consistency_assertions;向 agent_cli_entrpoint_ci 发出的 cli_entrpoint_status 请求;含有 command、exit_code、stdout_summary、stderr_summary、evidence_path 及 issue_ids_closed 的 validation_results;failure_debug_playbook;requests_for_model_or_data_patches。', { 'agent_id': 'agent_cli_entrpoint_ci', 'role': '过时 main.py、CLI 可发现性、uv 冒烟命令及可复现性入口点哨兵(sentinel)智能体。'

原文第 60 页

'instruction': '独立检查并执行仓库入口点。若 main.py 存在,其中不得含有过时的 hello-world 文本,且必须能够分派到已文档化的流水线命令,或清晰地打印 train/evaluate/profile/extract 的帮助信息。运行类 grep 检查以搜索 Hello from query220,并执行 uv run python main.py --help 或等效命令。若过时文本仍然存在,则提交一个阻塞性问题(blocking issue),因为当前 v3 证据显示 main.py 打印 Hello from query220-ourteam,尽管 v3 设计要求此行为应被阻止。', 'output_to_orchestrator_schema': 'EntrypointCIHandoffV1 继承自 HandoffV4,必须包含:main_py_status(取值 useful_dispatcher|documented_not_used|stale_hello|missing_ok);stale_text_scan(含 command_or_method、matches、evidence_path 和 status);cli_smoke_results(含 command、exit_code、stdout_summary、stderr_summary 和 status);documented_command_crosscheck(含 logs_path、commands_found、commands_execute_status 和 missing_commands);issue_tickets(所有者为 agent_training_reproducibility 或 agent_orchestrator);final_recommendation(取值 pass|fail|conditional)。', { 'agent_id': 'agent_eval_benchmarks', 'role': '循环度量(circular metric)、基准测试、逐残基分析、划分泄漏(split leakage)审查及度量溯源专家。', 'instruction': '实现并验证 src/evaluate.py。定义循环绝对误差、MAE、中位数、RMSE、带计数的逐残基分解,以及划分泄漏检查。确保 results/metrics.json 和 results/ablation_results.json 可从冻结的历史记录、检查点(checkpoint)、预测结果和划分清单(split manifest)中生成。创建 results/metrics_lineage.json,包含精确的来源指针和表达式。检测并提交阻塞问题,针对与历史记录、检查点元数据或消融 JSON 不一致的顶层度量值。经过此关卡后,v3 中 se3_gvp best_epoch 的不匹配问题必须不再可能出现。', 'output_to_orchestrator_schema': 'EvalHandoffV4 继承自 HandoffV4,必须导出 ResultsContract v4。必填字段:metric_formulae;circular_error_algorithm;metrics_json_accuracy_schema(含 model_id 键、test_mae_degrees、test_median_error_degrees、test_rmse_degrees、n_test_samples、n_params、best_epoch、best_val_loss、total_training_time_s 及 per_residue 计数); ablation_results_schema;test_index_schema;split_leakage_result;per_residue_count_sum_check;predictions_length_check;metric_lineage_map(含 source_path、source_json_pointer_or_expression、destination_json_pointer、tolerance 和 status);required_runtime_merge_request(描述 metrics 中预期的 profile 字段);validation_commands;issues_for_training_profile_or_report。', { 'agent_id': 'agent_profile_systems', 'role': '运行时、内存、环境、必要的 src/profile.py 路径、性能分析方法论及度量分析载荷(profiling payload)专家。', 'instruction': '精确实现并验证 src/profile.py。对所有 model_id 或已批准的子集进行前向推理和训练步骤的延迟分析(profile),捕获环境信息、参数量、

设备、线程、同步机制、预热/重复次数及内存。在均值不稳定时不得发布未加说明的时序均值;需计算变异系数(CV)、单调性告警,报告中位数和 p95,并在均值不稳定时使用中位数计算比率。包含 psutil RSS、参数内存、CUDA max_memory_allocated(当 CUDA 存在时)、torch profiler 内存或有文档说明的替代方案,以及分配器注意事项。禁止发布负内存使用值。导出 profile_summary_for_metrics_json,其中应包含 training_step_latency 及 RSS/CUDA/参数内存指针,而不仅限于前向延迟。', 'output_to_orchestrator_schema': 'ProfilingHandoffV4 继承自 HandoffV4,必须导出 ProfilingContract v4。必要产出物:src/profile.py 状态(created|modified|validated);results/profiling_results.json;profile_summary_for_metrics_json。必填字段:profiling_protocol;environment_capture_schema;batch_sizes;forward_latency_schema(含 mean、median、p95、std、cv 和 stability_status);training_step_latency_schema(含相同统计量);runtime_memory_json_schema;cuda_memory_fields(当 CUDA 存在时);rss_fields;parameter_memory_estimates;raw_memory_delta_fields_with_caveats;profiling_quality_flags(针对 high_cv、nonmonotonic_batch_scaling、tiny_rss_delta 和 missing_cuda);throughput_calculation;model_to_model_throughput_ratio_table(含 numerator_model、denominator_model、batch_size、statistic_used(mean|median)、json_pointers、ratio 和 narrative_phrase_recommendation);metrics_merge_payload(含 runtime、profiling、memory、throughput ratios 和 training_step_latency 的必要目标指针);plot_spec;validation_commands;recall_triggers。', {'agent_id': 'agent_profile_sanity_referee', 'role': '独立的性能分析方法、内存合理性、时序稳定性及速度比率裁判。',

原文第 61 页

'instruction': '在 profile 智能体(profile agent)发布 ProfilingContract 后,独立检查 results/profiling_results.json 及运行时绘图数据。重新计算吞吐量和比率,检查高方差(high-variance)和非单调(nonmonotonic)计时,验证内存字段在物理上是否可解释,并验证训练步延迟和内存字段是否已合并至 results/metrics.json。对缺失的注意事项、负使用量字段、缺失的参数数量/RSS/CUDA 估计、缺失的比率或报告/profile 比率不匹配问题,开启阻塞工单(blocker)。v3 高方差 B32 不变计时和非单调 B64/B32 计时必须重新运行或加注说明(caveat)。', 'output_to_orchestrator_schema': 'ProfilingSanityHandoffV4 继承 HandoffV4,必须包含:ratio_recomputations(含 model_id、batch_size、statistic_used、reported_throughput、recomputed_throughput、relative_error_pct 和 status);timing_anomaly_inventory(含 model_id、batch_size、json_pointer、mean、std、cv、median、monotonicity_context、caveat_present 和 status);memory_anomaly_inventory(含 json_pointer、value、expected_property、caveat_present 和 status);training_step_profile_check;metrics_json_runtime_memory_merge_check;issue_tickets;以及 final_recommendation,取值为 pass|fail|conditional。', {'agent_id': 'agent_metrics_artifact_integrator', 'role': '跨制品指标合并器、JSON 指针源映射所有者,以及 results/metrics.json 完整性门控。', 'instruction': '消费(consume)已冻结的 ResultsContract、ProfilingContract、EquivarianceAuditContract、TrainingHistory 模式、CheckpointMetadata、DataProvenanceManifest、RCSBProtocolAudit 和 AblationGrid。生成或修补(patch)results/metrics.json,使其包含准确率、运行时、profiling、内存、吞吐量比率、训练步延迟、数据摘要、源制品、等变性(equivariance)摘要和 consistency_checks。若任意顶层模型的 best_epoch 或 best_val_loss 与 training_history 或 checkpoint 元数据不一致,则不得冻结。若 metrics 缺少运行时/profiling 字段、内存字段、训练步延迟或源指针,则不允许 scicomm 流程继续推进。', 'output_to_orchestrator_schema': 'MetricsIntegrationHandoffV4 继承 HandoffV4,必须包含:MetricsIntegrationContract v2,其中 metrics_json_path 为 results/metrics.json;required_top_level_keys 包括 model_ids、models、runtime、profiling、memory、throughput_ratios、equivariance、data_summary、source_artifacts 和 consistency_checks;required_pointer_status_matrix,包含 MetricsPointerMatrix 中每个指针的 source_path、source_json_pointer_or_expression、destination_json_pointer、value、tolerance 和 status;merged_json_pointers,涵盖每个 MAE、median、RMSE、per_residue count、checkpoint、best_epoch、best_val_loss、total_training_time_s、forward latency、training_step_latency、memory、throughput_ratio 和 equivariance status;cross_file_consistency_results,比对 metrics、ablation、profile、histories、checkpoints、equivariance、data、index 和 plots;missing_field_blockers;freeze_status,取值为 draft|frozen|blocked;以及 issue_tickets。', {'agent_id': 'agent_json_consistency_referee', 'role': '独立的 JSON 血缘(lineage)、指标一致性和跨制品数值取证裁判。', 'instruction': '独立比对 results/metrics.json、results/ablation_results.json、训练历史文件、checkpoint 元数据、results/profiling_results.json、results/equivariance_tests.json、数据清单(data manifests)、plots 元数据(如存在)以及报告声明。从每个训练历史中重新计算 best_epoch 和 best_val_loss,重新计算消融改进量(ablation improvements),重新计算总训练时间,并验证所需指针是否存在。针对精确的 v3 不匹配模式开启阻塞工单:即 se3_gvp 顶层 best_epoch 或 best_val_loss 与嵌入的 train_history 或 results/training_history_se3_gvp.json 不一致。', 'output_to_orchestrator_schema': 'JSONConsistencyAuditHandoffV1 继承 HandoffV4,必须包含:artifact_path 为 results/json_consistency_audit.json;model_history_consistency(含 model_id、metrics_best_epoch_pointer、history_best_epoch_pointer、checkpoint_best_epoch_pointer、values、status 和 evidence_path);training_time_consistency(含 model_id、metrics_pointer、history_pointer、report_claim_if_available、values、tolerance 和 status);ablation_consistency(含 source_pointers、recomputed_values 和 status);profiling_merge_consistency;equivariance_schema_consistency;data_summary_consistency;issue_tickets(含 severity 和

owners);以及 final_recommendation,取值为 pass|fail|conditional。'}, {'agent_id': 'agent_scicomm_publication', 'role': '会议风格报告、图表叙述、局限性措辞及声明-证据专家。'},

原文第 62 页

'instruction': '仅在 MetricsIntegrationContract、ResultsContract、ProfilingContract、RCSBProtocolAudit、EquivarianceAuditContract、JSONConsistencyAudit 和 number_source_map 全部冻结后,方可起草 research_report.md。每项定量或比较性声明均须引用 JSON 指针或计算表达式。使用经验证的 RCSB 措辞:若使用了回退(fallback),须如实说明,不得声称 ID 由失败的查询返回。训练时间须来自历史记录或指标指针,而非性能估算值。避免 CPU/GPU 矛盾。速度声明须使用性能比率表及相应注意事项。均变性(equivariance)相关内容须引用 results/equivariance_tests.json 及命令证据,或明确说明豁免原因。', 'output_to_orchestrator_schema': 'SciCommHandoffV4 继承自 HandoffV4,必须包含: report_outline;figure_table_map;claims_to_evidence_matrix(含 claim_id、claim_text、parsed_value、unit、source_path、json_pointer_or_expression、tolerance、checked_status 及 dependent_gate);exact_metric_keys_used;rscsb_claims_to_evidence(引用查询尝试及过滤器验证);equivariance_claims_to_evidence;profiling_claims_to_evidence;training_time_claims_to_evidence;caption_consistency_checks;data_provenance_claims;limitations_section_points;stale_phrase_scan_results(针对不受支持的短语);contradiction_scan_results(针对 CPU 与 CUDA、回退与查询返回、以及正式测试语言之间的矛盾);unresolved_claims_needing_eval_profile_or_equivariance;recall_requests。'}, {'agent_id': 'agent_claims_consistency_referee', 'role': '独立的定量声明、报告指标、RCSB 语言、性能分析、均变性及索引一致性裁判。', 'instruction': '在 scicomm 起草 research_report.md 和 deliverables/index.json 后,解析每一项数值、近似、比较及来源声明。与 results/metrics.json、消融实验、性能分析、均变性测试、RCSB 查询尝试、过滤器验证、data_card、split_manifest 及 claims_to_evidence_matrix 进行比对。重点核查:报告中的训练时间声明是否与历史记录不符、速度比率是否未从性能比率表导出、是否存在 CPU/GPU 矛盾,以及是否声称由失败的 RCSB 查询产生了所选数据集。本智能体(agent)负责批评并提交机器可核查的问题工单,不承担主要实现工作。', 'output_to_orchestrator_schema': 'ClaimsAuditHandoffV4 继承自 HandoffV4,必须包含: numeric_claim_inventory(含 claim_id、file、quote_or_excerpt、parsed_value、unit、source_json_pointer_or_missing、recomputed_value、tolerance、relative_error_pct 及 status);approximate_claim_checks;comparative_claim_checks(含方向 lower_is_better 或 higher_is_better);throughput_ratio_checks;training_time_claim_checks;rscsb_language_checks;equivariance_language_checks;data_claim_checks;index_summary_checks;unsupported_claims;contradiction_checks;issue_tickets(含 severity、owner、evidence、requested_fix 及 blocking_if_fail);final_recommendation pass|fail|conditional。'}, {'agent_id': 'agent_artifact_validator', 'role': '独立的 CI、精确路径、模式(schema)、来源、指标完整性、入口点及可复现性验证器。', 'instruction': '至少对工作区执行六次检查:早期冒烟测试、RCSB 协议门控、数据/模型门控后、结果/性能/指标集成门控后、报告批评后,以及补丁修复后的最终门控。运行 uv run 冒烟检查、导入检查、JSON 解析检查、精确文件存在性检查、图表非空检查、RCSB 来源检查、均变性工件模式检查、陈旧 main.py 检查、指标指针矩阵检查及报告 JSON 一致性检查。若 src/profile.py 缺失,精确路径检查失败。若 results/metrics.json 缺少 runtime、profiling、memory、training_step_latency、source 指针或均变性摘要,则视为不合格。若 main.py 包含"Hello from query220",则视为不合格。', 'output_to_orchestrator_schema': 'ValidatorHandoffV4 继承自 HandoffV4,必须包含: validation_phase(early_smoke|rscsb_protocol|post_model|post_metrics|post_report|final);file_presence_matrix(含 exact_path、exists、size_bytes、listed_in_index、required_by_task、supporting_acceptance_path 及 status);json_schema_results;metrics_pointer_matrix_results;command_results(含 command、exit_code、stdout_summary、stderr_summary 及 evidence_path);import_results;plot_integrity_results;rscsb_provenance_results;equivariance_artifact_results;metrics_completeness_results;report_json_consistency_results;stale_entrypoint_results;artifact_registry_diff;issue_tickets;final_blockers;retest_results_for_issue_ids。'}, {'agent_id': 'agent_quality_safety', 'role': '科学严谨性、数据泄露、可复现性、均变性语言、性能分析、来源溯源及合规性审查员。'},

原文第 63 页

'instruction': '审查已组装的计划和工件,检查数据泄露、缺失控制、过度声明、错误的均变性语言、薄弱的消融实验、无效的性能分析、不一致的指标、无法核实或使用回退的 RCSB 数据获取、与预期交付物的精确路径不匹配、指标中缺少 runtime/memory 信息、陈旧的入口点,以及可复现性缺口。须使用来自文件、JSON 输出、日志及验证命令的具体证据。最终批准要求:无阻塞性泄露;RCSB 过滤器已记录且经过验证,或显式验证的回退已记录;循环指标有效;形式化均变性检查具备命令证据或豁免声明;至少包含一个不变(invariant)变体及两个等变(equivariant)变体(除非已豁免);稳健的性能分析注意事项;results/metrics.json 中包含 runtime/profiling/memory/training-step;src/profile.py 路径精确;main.py 无陈旧内容;uv 工作流可复现。', 'output_to_orchestrator_schema': 'QualityAuditHandoffV4 继承自 HandoffV4,必须包含: audit_checklist;leakage_assessment;data_provenance_assessment;rscsb_fallback_assessment;equivariance_assessment;ablation_assessment;profiling_assessment;metrics_completeness_assessment;metric_consistency_assessment;reproducibility_assessment;overclaiming_assessment;exact_deliverable_path_assessment;required_recalls;final_recommendation

```
pass|fail|conditional;evidence_paths 及 json_pointers.' }, { 'agent_id': 'agent_reproducibility_log_auditor', 'role': 'logs.txt、命令历史、智能体调用图、uv 工作流、召回历史及可复现性记录审计员。', 'instruction': '验证 logs.txt 是否记录了以下内容: 计划、执行、反思、所有已创建和派生的智能体、智能体间的工作流、已执行的智能体、交接(handoff)、合同确认、召回历史、补丁重测、命令、未解决风险及最终门控决策。将日志中的命令与工作及 uv 使用情况进行交叉核查。若缺少智能体名单、调用图、召回记录、uv 命令、RCSB 门控说明、JSON 一致性门控说明或最终未解决风险,则开具阻塞工单。', 'output_to_orchestrator_schema': 'LogAuditHandoffV4 继承自 HandoffV4,必须包含:log_manifest(含 logs_path、exists、size_bytes、sections_present、agent_roster_present、call_graph_present、contract_ack_present、commands_present、recall_history_present、patch_retest_present、final_gate_present、unresolved_risks_present 及 status);uv_workflow_check;command_to_artifact_traceability;issue_tickets;final_recommendation pass|fail|conditional.' } ]
```

所有非编排者(orchestrator)智能体共用的通信合约(contract):

原文第 64 页

每条响应必须是一个可机器校验的 HandoffV4 对象,包含以下键:handoff_id、agent_id、hop_id、phase、scope、contract_version、inputs_consumed、assumptions、interface_exports、artifact_plan_or_changes、validation_checks、metrics_or_estimates、evidence_bundle、downstream_impact、self_audit_against_prior_defects、risks、open_questions、requests_for_agents、ack_or_change_request、issue_tickets、recall_recommendations 以及 blocking_status。inputs_consumed 条目必须包含:source_agent_id、export_name、version、artifact_path_or_json_pointer、consumed_status 以及 freshness_requirement。interface_exports 条目必须包含:schema_id_or_signature、producer、consumer_agents、version、状态 draft|frozen|deprecated、required_fields、optional_fields、compatibility_notes、expected_json_pointers、validation_method、consumer_ack_required 以及 breaking_change_if_modified。artifact_plan_or_changes 条目必须包含: path、exact_required_path(布尔值)、supporting_acceptance_path(布尔值)、状态 planned|created|modified|validated、purpose、producer、consumer_agents、size_bytes、sha256_if_available、validation_status 以及 source_command。validation_checks 条目必须包含:check_id、command_or_method、expected、actual、exit_code_if_command、状态 pass|fail|blocked、blocking_if_fail、issue_id_if_fail 以及 evidence_path。metrics_or_estimates 条目必须包含:name、value、unit、source_path、json_pointer_or_expression、uncertainty_or_caveat 以及 destination_json_pointer_if_merged。evidence_bundle 条目必须包含:artifact_path、json_pointer、file_excerpt_or_hash、producing_command、exit_code_if_command、generated_at 以及 freshness_hop。downstream_impact 条目必须包含:consumer_agent_id、consumed_field、downstream_artifact 以及 reack_required_if_changed。self_audit_against_prior_defects 必须包含以下各行: src/profile.py 路径、RCSB 回退/溯源、运行时/性能分析指标、等变性正式证据、过时的 main.py、best_epoch 一致性、报告训练时一致性、性能分析异常以及日志完整性。requests_for_agents 条目必须包含:target_agent_id、question、required_response_schema、urgency、blocking_if_unanswered、requested_by_hop 以及 due_before_hop。ack_or_change_request 必须包含:reviewed_schema_ids、field_level_comments、决定 accept|change_request|blocked、required_resolution 以及 reack_required。issue_tickets 必须包含:issue_id、严重程度 critical|major|minor、owner_agent_id、evidence_path_or_json_pointer、file_excerpt_or_metric、requested_fix、acceptance_criteria_for_patch、retest_method 以及 history_trigger_if_any。recall_recommendations 必须包含:issue_id、owner_agent_id、narrowed_scope、changed_files_expected、acceptance_criteria_for_patch 以及 retest_method。blocking_status 必须为 pass|blocked|conditional,并附带 blocker_issue_ids。

契约(contract)确认协议:

在所有生产者、消费者和验证者均返回 ContractAckV4 或 ChangeRequestV4 之前,编排器(orchestrator)不得冻结任何共享契约。ContractAckV4 需包含:schema_id、version、agent_id、field_name、consumer_use、accepted_type_or_constraint、validation_method、确认状态 accept|change_request|blocked、requested_change、blocking_reason 以及 reack_required。契约在实现后若发生变更,须向所有受影响的消费者重新确认,并进行针对性复测。

补丁(patch)协议:

任何被召回的所有者必须返回 PatchHandoffV4,包含:issue_ids_to_close、changed_files、diff_summary、contract_fields_changed、backward_compatibility_notes、validation_evidence、metrics_or_pointer_changes、issue_ids_closed、issue_ids_still_open、regression_risks 以及 requested_retests。编排器必须将 PatchHandoffV4 路由至原始开启者以及验证者或质量方,复测通过后方可关闭问题。若某补丁改变了指标、性能分析、数据或报告中的声明,指标集

成器、JSON 一致性裁决器、声明裁决器和日志审计器必须重新执行各自受影响的检查。

强制智能体间交接(handoff):

1. 接口管理员在实现前发布 RequiredPathManifest v4、InterfaceContract v4 以及 MetricsPointerMatrix v2;每个生产者和验证智能体须返回字段级 ContractAckV4 或 ChangeRequestV4。
2. 数据智能体发布 DataProvenanceManifest v4、RCSB 查询尝试记录以及过滤器验证结果;RCSB 协议裁决器须在训练开始前及任何报告数据声明冻结前通过或有条件通过。
3. 数据智能体与几何智能体联合产出 DatasetRecord v4 以及无侧链输入泄漏证据,模型、训练、评估、性能分析或等变性代码方可消费数据。
4. 数据智能体与评估智能体联合产出 SplitLeakageReview v3,方可冻结训练和评估声明。
5. 模型智能体与消融(ablation)智能体联合产出 ModelRegistry v4、ModelIOContract v4 以及 AblationGrid v4;训练、评估、性能分析、等变性和指标智能体必须确认可导入性、工厂签名、model_ids 以及检查点键。
6. 训练智能体产出 TrainingHistory 和 CheckpointMetadata;评估和指标智能体必须在指标冻结前确认 epoch 选择字段。
7. 入口点 CI 在最终版本前审计 main.py;过时的 hello-world 内容属于阻塞性问题。

原文第 65 页

8. 等变性裁判在科学传播撰写等变性声明之前,发布 EquivarianceAuditContract v2 及附带命令证据的 results/equivariance_tests.json。
9. 评估(Eval)代理在指标集成和报告精度声明之前,发布 ResultsContract v4 及 metrics_lineage。
10. 性能分析(Profile)代理在指标集成和速度或内存声明之前,发布 ProfilingContract v4 并验证精确的 src/profile.py。
11. 性能分析完整性裁判在科学传播撰写性能分析文本之前,审计 ProfilingContract 及指标运行时-内存合并结果。
12. 指标集成器仅在消耗来自评估、训练、性能分析、消融、数据、RCSB 和等变性的全部输入,且 results/metrics.json 包含所有必要指针后,才冻结 MetricsIntegrationContract。
13. JSON 一致性裁判在图表、索引摘要和报告被视为最终版本之前,审计已冻结的指标。
14. 科学传播代理向声明裁判发送 ClaimsEvidenceMatrix;所有声明阻断项路由至负责的专项代理或科学传播代理进行修复。
15. 验证器、质量、RCSB 裁判、性能分析完整性裁判、JSON 一致性裁判、声明裁判、入口点 CI 和日志审计器在每轮补丁后均进行审查,而非仅在补丁之前执行。

最少编排器-子代理跳数(hop):

跳 0 EvidenceRubric 扇出:向所有代理发送任务交付物、两两历史记录及当前 v3 文件证据。

跳 1 并行缺陷预检:各专项代理返回 self_audit_against_prior_defects 及 requests_for_agents。

跳 2 起草合约:管理员编译 RequiredPathManifest、InterfaceContract、MetricsPointerMatrix 及模式注册表。

跳 3 合约确认:相关消费方返回 ContractAckV4 或附字段级注释的 ChangeRequestV4。

跳 4 合约协调循环:编排器仅召回存在争议的生产方和消费方,针对争议字段和重测方法进行处理,直至冻结或显式豁免。

跳 5 RCSB 实现:数据代理实现查询/下载/缓存/元数据/划分,并返回 DataRCSBHandoffV4。

跳 6 RCSB 协议裁判门控:独立裁判审计查询状态、回退策略和过滤器验证;失败则召回数据代理或科学传播代理。

跳 7 数据-几何联署:数据代理和几何代理实现特征提取、残基化学、无泄漏检查及划分审查;评估代理确认划分结果。

跳 8 模型-消融实现:模型代理和消融代理实现工厂类及变体;训练、评估、性能分析和等变性代理确认导入与形状证据。

跳 9 早期 CI 冒烟测试:验证器和入口点 CI 在执行耗时工作之前,检查精确路径、导入、main.py、uv 命令、RCSB 制品、DatasetRecord 键及等变性钩子。

跳 10 等变性预报告门控:独立裁判运行合成批次和真实批次的变换测试;失败则召回模型代理或训练代理。

跳 11 训练冒烟测试:训练代理对所有 model_id 运行有界冒烟测试,并返回 history/checkpoint 元数据一致性结果。

跳 12 完整或有界训练:训练代理按批准预算运行并冻结 history/checkpoints。

跳 13 评估与划分泄漏冻结:评估代理计算循环指标、逐残基细分、消融结果、预测值及 metrics_lineage。

跳 14 性能分析与完整性审计:性能分析代理发布运行时、内存、训练步延迟及比率;性能分析完整性裁判重新执行比率和异常检查。

跳 15 指标集成:指标集成器将精度、训练、数据、RCSB、性能分析、消融和等变性结果合并至 results/metrics.json,并附带所有必要指针。

跳 16 JSON 一致性审计:独立裁判重新计算 best_epoch、best_val_loss、训练时长、消融提升幅度及必要指针状态。

跳 17 图表生成:从冻结的 JSON 重新生成图表,并验证图表数据。

跳 18 索引生成:索引列出所有必需和辅助制品;验证器检查覆盖情况。

跳 19 报告起草:科学传播代理仅依据冻结合约起草报告,并输出 ClaimsEvidenceMatrix。

跳 20 独立批评:声明裁判、验证器、RCSB 裁判、性能分析完整性裁判、JSON 一致性裁判、质量裁判、入口点 CI 和日志审计器对报告、索引、指标、性能分析、等变性、数据、日志及路径进行审计。

跳 21 定向 PatchHandoff 召回:对于每个阻断项,编排器仅召回负责方,并附上 narrowed_scope、issue_ids_to_close、预期变更文件及重测标准。

跳 22 重测:原始发现方联合验证器或质量裁判对特定 issue_ids 进行重测;受影响的下流代理重新确认变更后的合约。

原文第 66 页

跳转节点(Hop)23 最终关卡:重复 Hop 21 和 Hop 22,直至无阻塞项,或阻塞项已记录豁免并附有证据及保守措辞的报告。

召回触发条件:

若有任何必需交付物路径缺失、为空、无法解析、未列入 deliverables/index.json,或被同名不符的文件替代,则召回接口管理员(interface steward)或验证员(validator);src/profile.py 缺失始终为阻塞项。若 RCSB 查询载荷、端点、已选 ID、查询尝试记录、元数据验证、回退策略、下载清单、分割清单、data_card 证据或确定性提取命令缺失或仅出现在报告中,则召回数据智能体(data agent)。若查询/回退过滤器验证缺失或未经独立审计,则召回 RCSB 裁判员(RCSB referee)。若 χ 1 原子表、完整性规则、替代位点(altloc)策略、环形约定或无侧链输入泄漏证据缺失,则召回几何智能体(geometry agent)。若变换测试钩子、ModelRegistry 导入、工厂签名、检查点键,或等变性/不变性表述有误,则召回模型智能体(model agent)。若 results/equivariance_tests.json 缺少 command_results、合成与真实批次案例、容差、通过/失败状态或证据路径,则召回等变性裁判员(equivariance referee)。若消融实验少于一个不变变体加两个等变变体且无豁免,model_ids 或检查点键不一致,训练历史缺失,best_epoch 或 best_val_loss 在各制品间不一致,或 main.py 仍为陈旧版本,则召回消融(ablation)或训练智能体。若 main.py 在补丁后未重新检查,则召回入口点 CI 智能体(entrypoint CI agent)。若中位角误差、MAE、RMSE、每残基计数、分割泄漏检查、预测长度或 metric_lineage 指针缺失或不一致,则召回评估智能体(eval agent)。若 src/profile.py 缺失、训练步骤性能分析缺失且无豁免、内存数值不合理且无注意事项、负值被报告为使用量、环境详情缺失、时间变异系数(CV)偏高且无重跑/注意事项,或速度比率与 profile JSON 相差超过 10%,则召回 profile 智能体。若比率、高 CV 时间或内存异常未经独立盘点,则召回 profile 健全性检查员(profile sanity agent)。若 results/metrics.json 缺少 runtime、profiling、memory、training_step_latency、throughput_ratio、source_artifact、consistency_checks 或等变性汇总字段,则召回指标集成员(metrics integrator)。若 best_epoch、best_val_loss、training_time、消融、profile 或数据一致性未经审计,则召回 JSON 一致性裁判员(JSON consistency referee)。若报告正文与 JSON 指标、训练历史、RCSB 协议、profile 比率、等变性证据或图表标题相矛盾,则召回科学传播智能体(scicomm agent)。若定量或溯源声明未经盘点,则召回声明裁判员(claims referee)。若 logs.txt 缺少智能体名单、调用图、合约确认(contract acks)、交接记录、召回历史、uv 命令、补丁重测或 final_gate_decision,则召回日志审计员(log auditor)或编排器(orchestrator)。若最终批准缺少具体证据路径,则召回质量

智能体(quality agent)。

冲突解决:

若两个智能体意见不一致,编排器创建一条 ConflictRecord,记录各方立场、证据路径或 JSON 指针、受影响制品、下游合约字段、决策、更新的消费方及重新确认要求。优先选择科学保守的选项:不声明未验证的 RCSB 过滤器,不隐藏回退情况,不在无注意事项的情况下报告不稳定的 profiling 比率,不发布在各制品间不一致的指标。将决策记录于 logs.txt,将局限性记录于 research_report.md。

最终关卡条件:

仅在满足以下所有条件时方可通过:所有必需文件均位于精确路径(包括 src/profile.py);deliverables/index.json 列出了确切的必需制品及其溯源支持制品;RCSB 数据获取可通过成功的查询衍生 ID 复现,或通过带有每 ID 过滤器证据及正确报告措辞的显式验证回退复现;DatasetRecord 从输入中排除侧链坐标;results/equivariance_tests.json 中存在正式等变性检查,包含 command_results、合成与真实批次证据、model_ids、容差以及通过/失败状态,或附有明确豁免;results/metrics.json 包含 MAE、中位数、RMSE、每残基数、与训练历史/检查点一致的 best_epoch 和 best_val_loss、total_training_time_s、runtime、profiling、RSS/CUDA/参数内存摘要、training_step_latency、吞吐量比率、来源制品指针、consistency_checks 及等变性摘要;results/ablation_results.json 包含必需的不变变体和等变变体并与指标一致;profiling 包含前向与训练步骤延迟、稳健的内存注意事项、稳定性标志、参数/RSS/CUDA 估计(适用时)以及模型间比率;research_report.md 中的定量声明在容差范围内与 JSON 一致,训练时间声明与历史记录一致,RCSB 措辞与协议审计一致,速度声明与 profile 比率一致,且不存在陈旧的无依据局限性表述;所有五个必需图表均非空且由冻结 JSON 生成;若 main.py 存在则不应为陈旧的 hello-world;uv run 冒烟命令已记录;logs.txt 包含智能体名单、调用图、合约确认、交接记录、召回历史、补丁重测、命令、未解决风险及 final_gate_decision。

原文第 67 页

C. 测试框架(Harness)优化器提示词(prompt)

测试框架改进:系统提示词

你是自主编码智能体的首席提示词工程师。

你已使用编码智能体(如 Claude Codex)解决了以下查询。你的任务是仅改进查询提示词中的"多智能体设计"模块,以提升查询交付物的质量(参见下方"当前多智能体设计")。优先考虑多智能体系统能够做好(或已具备)而单智能体系统无法做到(或尚未具备)的测试框架特性,例如并行专家分工、跨智能体批评、智能体间通信以及迭代协调。

重点改进以下两点:

为每个子智能体的"输出至编排器"模式(schema)制定更强的智能体间通信合约(contract);

设计更多编排器-子智能体跳转节点(hop),以显式迭代式召回/重新委派取代单次单向执行。

所有改动必须基于提交制品和对比历史中的具体证据。

仅返回有效 JSON。

测试框架改进:用户提示词

1. 以下查询已使用 Claude 编码智能体(Claude coding agent)解决

任务(任务描述) # 当前多智能体设计(v0)(多智能体设计描述) #
保存规则 所有 Python 工作流均使用 uv——用 uv run 运行代码,用
uv add 安装依赖,用 uvx 使用工具。编写历史日志:在推理日志中于
logs.txt 文件里记录计划、执行(或工具执行)和反思。在 logs.txt 中记
录所有已创建和衍生的智能体,描述智能体间的工作流(例如以结构化
大纲或图表形式),并追踪各智能体的执行情况及其在流程中的角色。

→ → → → → →

2. 来自 Claude Code 的当前提交代码仓库(仅作为证据)

当前迭代的工作区(multi-agent-design-v0)路径: `...` ##
目录树(示例) (树形描述) ## 程序化预检(启发式证据) 来源于任务文
本(预期路径)与本工作区的对比;不能替代对交付物模块的直接阅读。

核心文件(内容) ### 从任务文本中提取的图表路径(内容) # 文件摘录(大小受限;二进制文件已省略)
文件:'research_report.md'(研究报告文件) ### 文件:'deliverables/index.json'(index.json
文件) ### 文件:'results/model_comparison.json'(model_comparison.json 文件) ###
文件:'requirements.txt'

原文第 68 页

(requirmentx.txt 文件)### 文件:'run_backtest.py' [已截断](代码文件)# results/ 下的完整 JSON 产物 ### 文件:
'results/ablation_log.json'(内容)### 文件:'results/model_comparison.json'(内容)

3. 两两对比历史摘要

对比:multi-agent-design-v0 与 multi-agent-design-v1

简称:v0 vs v1

胜者:平局

评判模型:gpt-5.5

评判时间(UTC):2026-05-24T10:00:05.337848+00:00

仓库 A:...

仓库 B:...

评判理由(逐字摘录):

两份提交在所提供的证据中无法区分:二者指向相同的工作区路径,且目录树、research_report.md 摘录、
deliverables/index.json、results/metrics.json、results/model_comparison.json、requirements.txt 以及 run_pipeline.py 摘录
均完全一致。两份提交在实质上均覆盖了所要求的产物清单:research_report.md 包含所要求的会议论文格式章节和
Markdown 图片链接;deliverables/index.json 列出了报告、图表、代码入口点、指标路径和依赖项;deliverables/plots/ 下存
在全部九张必需的 PNG 图片;Python 代码组织于 src/ 目录下,run_pipeline.py 作为唯一入口点;requirements.txt 已锁定版
本;results/metrics.json、results/model_comparison.json 和 results/ablation_log.json 均存在。

然而,两份提交存在相同的量化质量问题。报告与数值产物之间存在内部不一致:摘要将 5 日 GBM IC 值 0.039 称为统计显
著,但 results/metrics.json -> predictive_metrics.gbm.5d 中 p=0.122;指标元数据显示主模型(primary_model)为

elasticnet,trading_metrics.primary_model 为 elasticnet_h5_wf,而报告中却将 GBM 称为主模型。结论部分称 ElasticNet 在 5 日预测上优于 Ridge/GBM,但 results/metrics.json 显示 ElasticNet 5 日 IC=-0.0458,而 Ridge 5 日 IC=0.0896,GBM 5 日 IC=0.0391。

原文第 69 页

两份提交在实质上也均未能完成所要求的 2020 年前后稳定性分析:results/metrics.json -> subperiod_metrics.pre_2020 为空,statistical_tests.stability_scores 为 NaN/null,model_comparison.json 中 stability_score 全部为 null。results/metrics.json 摘录还包含非标准的 JSON NaN 标记,这是一项可重复性/清单质量缺陷。从实证角度看,两份提交均使用了合理的时间切分和成本设置,但报告中的代理冲击(proxy shock)构造使用了事件后 3-5 天的价格变动,摘录内容并不能证明这些特征在用于预测 1-20 日重叠收益之前已被安全地做了滞后处理。由于所有优点和缺点完全相同,两份提交不分高下。

下一版本的可行改进方向:

报告与数值产物之间存在内部不一致。

两份提交均未能完成 2020 年前后的稳定性分析。

指标 JSON 包含非标准 NaN 标记。

代理冲击构造中存在潜在的特征泄露。

两份提交均未能证明自身的优越性。

将此历史记录作为约束条件:保留获胜特质,修复所列缺陷。

3b. 两两对比历史增量清单(需修复的重复问题)

优先修复以下重复问题(计数越高,在历史中出现越频繁):

[1x] 报告与指标不一致

[1x] 稳定性/鲁棒性差距

[1x] null/nan 质量缺陷

[1x] 可重复性差距

[1x] 验证/测试薄弱

4. 指令

将多智能体(multi-agent)设计从 v1 改进至 v2,通过构建相对于单智能体执行的真正多智能体优势,提升查询可交付成果的质量。保留原始意图和所要求的可交付成果,但需针对证据揭示的弱点进行改进。

显式强化"输出给协调器(orchestrator)"的契约(contract),使下游智能体能够消费结构化输出。

显式增加协调器与子智能体之间的反馈循环(feedback loop):允许协调器以更窄的后续范围和更新的验收标准重新调用已调用过的子智能体。最重要的是,优先推进能够创造真正多智能体优势的改进(例如,专家并行、跨智能体验证、智能体间通信以及冲突解决循环)。

反馈质量要求:

具体且有证据支撑。

引用所提供证据中的具体文件/指标/历史信号。

原文第 70 页

避免泛泛而谈(例如"更好的协调");解释机制和预期效果。

不得更改查询中非设计部分。

输出模式(schema):

```
{
  "improved_multi_agent_design": "<仅替换设计块的完整文本>",
```



```

"changes_from_previous": ["<具体改动 1>", "..."],
"why_it_should_improve": ["<机制性理由 1>", "..."],
"evidence_used": ["<文件/指标/历史引用 1>", "..."],
"expected_impact": ["<预期质量/可靠性/可重复性提升 1>", "..."],
"verification_checks": ["<如何在下一次迭代中验证该改动已生效>", "..."]
}

```

improved_multi_agent_design 应以如下内容开头:

"Create an agent team with the following agent candidates to solve this problem:"

D. 任务示例

本附录提供了我们评估基准中所用 30 个任务领域的代表性示例。我们设有三个领域(机器人学、药学和定量研究),每个领域包含 10 个任务。

原文第 71 页

D.1. 机器学习研究任务(机器人学)

机器人学研究任务

任务。 为一个小型视觉-语言模型(vision-language model)创建可重复的微调实验,用于机器人相关指令分类(例如,动作类型、物体类别、空间关系)。使用公开数据集,如与动作对齐的 EPIC-KITCHENS 字幕(或从 Something-Something V2 中整理的子集)。评估准确率、校准性以及同义改写的鲁棒性。

数据集获取。 EPIC-KITCHENS——按以下方式注册并下载:<https://epic-kitchens.github.io/>

可交付成果。

1. research_report.md,以会议论文格式(相当于 6–9 页)描述微调协议和鲁棒性评估。

2. deliverables/index.json。

3. deliverables/plots/

accuracy_by_class.png

confusion_matrix.png

calibration_reliability_diagram.png

paraphrase_robustness_drop.png

learning_curves.png

4. 可重复代码,包括数据集准备、微调、评估、同义改写生成(公开改写模型或基于规则的模板)以及 requirements.txt。

5. results/metrics.json,包含:

Top-1 准确率

宏平均 F1(Macro F1)

期望校准误差(ECE)

同义改写鲁棒性下降幅度

每类别指标

6. results/model_comparison.json,对冻结、微调和部分微调变体(例如 LoRA)进行比较,并附对应超参数。

原文第 72 页

D.2. 机器学习研究任务(定量研究)

定量研究任务

任务。研究来自维基百科页面浏览量的替代数据能否改善大盘股的短期收益预测。

研究范围。道琼斯 30 指数成分股(来自维基百科)及 SPY 作为基准。

数据来源。

Wikimedia REST API,用于获取每日维基百科页面浏览量

Yahoo Finance,用于获取价格和交易量

FRED,用于可选的宏观经济控制变量

特征构建。构建以下特征:

异常页面浏览量激增(相对于滞后基准的 z-score)

注意力动量(attention momentum)

星期效应调整

与财报日期的交互项

使用公开的财报日来源(例如,抓取纳斯达克财报日历)。若抓取不稳定,记录备用方案。

建模与评估。训练一个无泄露的模型,预测次日和次周收益及/或波动率。使用净化时间序列交叉验证(purged time-series cross-validation)进行评估。在考虑交易成本的情况下,对市场中性多空注意力策略进行回测。加入因果式安慰剂测试(causal-style placebo test),将页面浏览量向后平移 +7 天,并验证预测信号消失。

可交付成果。

1. research_report.md,以会议论文格式(相当于 7-10 页)。
2. deliverables/index.json。
3. deliverables/plots/,包含实验可视化图表和回测图表。
4. 可重复的 Python 代码,用于数据收集、特征工程、建模和评估。
5. results/metrics.json,包含:

IC / RankIC

预测 $\$R^2$

策略夏普比率(Sharpe ratio)

Beta

最大回撤(Maximum drawdown)

换手率(Turnover)

交易成本

安慰剂测试指标

6. results/feature_mapping.json,记录页面到股票代码的映射及置信度分数;results/model_comparison.json,对基线模型与注意力增强模型进行比较。

原文第 73 页

D.3. 机器学习研究任务(药学)

药学研究任务

任务. 设计一个实验,通过将蛋白质序列嵌入(ESM-2)与来自蛋白质数据库(PDB)的结构衍生图特征相结合,评估多模态预训练方法,以在公开数据集(例如 Swiss-Prot 注释酶)上预测酶委员会(EC)编号。比较仅序列、仅结构和融合表示三种方案。

数据来源。

UniProt / Swiss-Prot 注释

通过公开 SIFTS 数据库获取的 PDB 映射

成功标准。证明多模态融合(multimodal fusion)是否能提升已知结构蛋白质的预测性能。

交付物。

1. research_report.md,以会议论文风格撰写(相当于 8 - 10 页),包含融合架构和数据集划分策略。
2. deliverables/index.json。
3. deliverables/plots/,包含可视化和评估图表。
4. 可复现的 Python 代码,涵盖数据处理、表示学习、多模态融合、训练和评估。
5. results/metrics.json,包含:

宏平均与微平均 F1

跨 EC 第 1 - 4 层的层次准确率

覆盖率统计

6. results/ablation_results.json,比较融合变体(拼接、注意力、门控)的配置及评估指标。

E. 评估器提示词(Prompt)

我们提供大语言模型评估器 `$L\_text{eval}$` 的系统提示词和用户提示词,用于对两个输出进行两两比较(pairwise comparison):

两两比较:系统提示词

你是一名{资深量化研究员},正在比较同一任务的两份已完成提交。{以量化视角进行评判:经验严谨性、样本外(OOS)测试与数据泄露规范、不确定性与基线设置、可复现性,以及叙述与数值产出之间的一致性。}你作风严格、务实、以证据为导向,契合对两名初中级研究员交付物进行内部评审的精神。

规则:

Task 块是唯一规定必须产出内容的合约(contract)。工作区各节仅作为证据,不得凭空捏造文件内容。

任务在本仓库目录(multi_agent_design/*/queries.json)中的编写方式:通常为一份文档,先给出任务说明,然后包含"数据来源"、"数据获取"、"成功标准"等节,以及几乎必有的 Deliverables 块(有时写作单数 Deliverable,或行内写法 Deliverables: (1) ...)。以其中明确要求的所有产出物(以及其上方叙述中同等明确的要求)作为核查清单,而非以文件夹名称或假设推断。

优先引用具体出处(路径、JSON 键、报告章节),而非泛泛称赞。

当某份提交在交付物、严谨性、可复现性和任务契合度方面整体上明显更优时,选择 A 或 B。仅当两份提交整体上确实旗鼓相当(而非仅仅各有侧重)时,才使用平局(tie)。

仅返回符合用户消息中指定键的合法 JSON(无 Markdown 围栏,JSON 外无正文)。

原文第 74 页

两两比较:用户提示词

Task

评估机器人规划成功概率预测器的校准性能。使用公开仿真环境(例如 Robosuite),生成一批难度各异的执行尝试数据集;训练一个模型,从状态/任务特征预测成功概率;评估校准误差(ECE)、区分能力(AUROC)以及决策效用(是否执行还是重新规划)。环境获取:Robosuite — <https://github.com/ARISE-Initiative/robosuite>(`pip install robosuite`;MuJoCo)。

交付物:

1. research_report.md,以会议论文风格撰写(相当于 6–9 页),聚焦于校准与决策制定。
2. deliverables/index.json。
3. deliverables/plots/:reliability_diagram.png、ece_vs_temperature.png、roc_curve.png、decision_utility_curve.png、calibration_by_difficulty_bin.png。
4. 可复现代码:数据生成、特征提取、模型训练、校准方法(温度缩放/等渗回归)、评估;requirements.txt。
5. results/metrics.json:auroc、average_precision、ece、brier_score、expected_utility_gain。
6. results/model_comparison.json,比较未校准与已校准变体,附每个区间的指标。

标签(仅供定位参考)

提交 A 对应运行类型:'base'

提交 B 对应运行类型:'defaultTeam'

提交 A 的工作区(仅作参考) 路径:'...' ## 目录树(代表性示例)

```
query318/
|-- data/
|   |-- episodes.parquet [.parquet binary/large]
|-- deliverables/
|   |-- plots/
|       |-- calibration_by_difficulty_bin.png [.png binary/large]
|       |-- decision_utility_curve.png [.png binary/large]
|       |-- ece_vs_temperature.png [.png binary/large]
|       |-- reliability_diagram.png [.png binary/large]
|       |-- roc_curve.png [.png binary/large]
|   |-- index.json
|-- logs/
|-- models/
|   |-- trained.pkl [.pkl binary/large]
|-- results/
|   |-- metrics.json
|   |-- model_comparison.json
|-- src/
|   |-- __init__.py
|   |-- data_generation.py
|   |-- evaluation.py
|   |-- model_training.py
|   |-- plotting.py
|   |-- run_pipeline.py
|-- .gitignore
|-- .python-version
|-- logs.txt
|-- main.py
|-- pyproject.toml
|-- requirements.txt
```

原文第 75 页

```
|-- research_report.md
|-- uv.lock
```

程序化预检(仅启发式证据)

基于任务文本(预期路径)与本工作区的对比推导;不能替代对 Deliverables 块的逐条阅读。

核心文件

'deliverables/index.json':已找到

'requirements.txt':已找到

'research_report.md':已找到

'results/metrics.json':已找到

从任务文本中提取的图表路径(子串匹配)

提及数量:5

(通过)'deliverables/plots/calibration_by_difficulty_bin.png'

(通过)'deliverables/plots/decision_utility_curve.png'

(通过)'deliverables/plots/ece_vs_temperature.png'

(通过)'deliverables/plots/reliability_diagram.png'

(通过)'deliverables/plots/roc_curve.png'

文件摘录(大小受限;二进制文件略)

文件:'research_report.md'

(内容)

文件:'deliverables/index.json'

```
{
  "project": "Calibration of Robot Plan Success Probability Predictors",
  "environment": "Robosuite 1.5.1 \u2014 Lift task (Panda robot, MuJoCo)",
  "n_episodes": 800,
  "n_difficulty_levels": 10,
  "models": [
    "logistic",
    "xgb_uncal",
    "xgb_temp",
    "xgb_iso",
    "xgb_platt"
  ],
  "files": {
    "research_report": "research_report.md",
    "plots": {
      "reliability_diagram": "deliverables/plots/reliability_diagram.png",
      "ece_vs_temperature": "deliverables/plots/ece_vs_temperature.png",
      "roc_curve": "deliverables/plots/roc_curve.png",
      "decision_utility_curve": "deliverables/plots/decision_utility_curve.png",
      "calibration_by_difficulty_bin": "deliverables/plots/calibration_by_difficulty_bin.png"
    },
    "results": {
      "metrics": "results/metrics.json",
      "model_comparison": "results/model_comparison.json"
    },
    "source": {
      "data_generation": "src/data_generation.py",
      "model_training": "src/model_training.py",
      "evaluation": "src/evaluation.py",
      "plotting": "src/plotting.py",
      "pipeline": "src/run_pipeline.py"
    }
  }
}
```

原文第 76 页

```
},
"requirements": "requirements.txt"
},
"key_results": {
  "best_model": "logistic",
  "auROC": 0.9320388349514563,
  "average_precision": 0.9663487666977535,
  "ece": 0.07296066373719252,
  "brier_score": 0.100785777593245,
  "expected_utility_gain": 0.120000000000000005,
  "optimal_threshold": 0.43,
  "temperature": 1.0476145522946596,
  "n_test_episodes": 160,
  "overall_success_rate": 0.6425,
  "all_models_ece": {
    "logistic": 0.07296066373719252,
    "xgb_uncal": 0.10315536023117601,
    "xgb_temp": 0.09393513752147556,
    "xgb_iso": 0.09425514140166341,
    "xgb_platt": 0.09693039900938674
  },
  "all_models_auROC": {
    "logistic": 0.9320388349514563,
    "xgb_uncal": 0.9192641798671436,
    "xgb_temp": 0.9192641798671436,
    "xgb_iso": 0.884857775506728,
    "xgb_platt": 0.9192641798671436
  }
}
}
```

文件:'results/model_comparison.json'

```
{
  "logistic": {
    "auroc": 0.9320388349514563,
    "average_precision": 0.9663487666977535,
    "ece": 0.07296066373719252,
    "brier_score": 0.100785777593245,
    "expected_utility_gain": 0.12000000000000005,
    "optimal_threshold": 0.43,
    "optimal_utility": 0.40750000000000003,
    "always_execute_utility": 0.2875,
    "per_bin_accuracy": [
      0.0555555559694767,
      0.222222238779068,
      0.1428571492433548,
      0.20000000298023224,
      0.4000000059604645,
      0.1666666716337204,
      0.5,
      0.6666666865348816,
      0.6000000238418579,
      0.0,
      0.8333333134651184,
      0.5555555820465088,
      1.0,

```

原文第 77 页

0.75,

1.0

```
], "per_bin_confidence": [ 0.03278137258815277, 0.09498844606580734, 0.15948443452447084, 0.2324303381767705,
0.2939246998914943, 0.37446054887181796, 0.43948446741954095, 0.4909236747175559, 0.5797088355919958,
0.6369048148423019, 0.7079080166483102, 0.7718871406484689, 0.8356235518263792, 0.897439985598854,
```

0.9841492082585949

```
], "per_bin_counts": [ 18, 9, 7, 5, 10, 6, 2, 6, 5, 1, 6, 9, 8, 4, ], "calibration_by_difficulty": [ { "difficulty": "easy", "count": 30, "
success_rate": 1.0, "mean_predicted_prob": 0.9977654482832153, "ece": 0.0022345517167846607 }, { "difficulty": "extreme", "
count": 30, "success_rate": 0.10000000149011612, "mean_predicted_prob": 0.0646247711265978, "ece": 0.06829405712638813 }, { "
difficulty": "hard", "count": 31, "success_rate": 0.774193525314331, "mean_predicted_prob": 0.7680816782422626,
```

原文第 78 页

```
"ece": 0.21086340848418922 }, { "difficulty": "medium", "count": 34, "success_rate": 1.0, "mean_predicted_prob":
0.9721348788251063, "ece": 0.027865121174893703 }, { "difficulty": "very_hard", "count": 35, "success_rate":
0.34285715222358704, "mean_predicted_prob": 0.34255369386431916, "ece": 0.12196064549503632 }
]
```

]

```
}, "xgb_uncal": { "auroc": 0.9192641798671436, "average_precision": 0.9602022705286992, "ece": 0.10315536023117601, "
brier_score": 0.11811486631631851, "expected_utility_gain": 0.10000000000000003, "optimal_threshold": 0.395, "
optimal_utility": 0.3875, "always_execute_utility": 0.2875, "per_bin_accuracy": [ 0.125, 0.4285714328289032,
0.111111119389534, 0.20000000298023224, 0.625, 0.0, 0.75, NaN, 0.6666666865348816, NaN, 0.6666666865348816, 0.75, 0.5,
0.6666666865348816,
```

0.9740259647369385

```
], "per_bin_confidence": [ 0.021986672654747963, 0.09743238985538483, 0.16554240882396698, 0.24125663936138153,
0.30319130420684814, 0.3695129454135895, 0.4141604006290436, NaN, 0.557410717010498, NaN, 0.696160078048706,
0.7705077528953552,
```

原文第 79 页

0.8378811478614807, 0.8917625546455383,

0.991079568862915

```
], "per_bin_counts": [ 24, 7, 9, 5, 8, 7, 4, 0, 3, 0, 3, 4, 6, 3, ], "calibration_by_difficulty": [ { "difficulty": "easy", "count": 30, "success_rate": 1.0, "mean_predicted_prob": 0.9960089921951294, "ece": 0.0039910078048706055 }, { "difficulty": "extreme", "count": 30, "success_rate": 0.10000000149011612, "mean_predicted_prob": 0.05119968578219414, "ece": 0.11872586409250896 }, { "difficulty": "hard", "count": 31, "success_rate": 0.774193525314331, "mean_predicted_prob": 0.7874037027359009, "ece": 0.19866928410145546 }, { "difficulty": "medium", "count": 34, "success_rate": 1.0, "mean_predicted_prob": 0.9958076477050781, "ece": 0.004192352294921875 }, { "difficulty": "very_hard", "count": 35, "success_rate": 0.34285715222358704, "mean_predicted_prob": 0.33570966124534607, "ece": 0.30034031367727687 }
```

原文第 80 页

```
]
}, "xgb_temp": { "auroc": 0.9192641798671436, "average_precision": 0.9602022705286992, "ece": 0.09393513752147556, "brier_score": 0.11716009676456451, "expected_utility_gain": 0.10000000000000003, "optimal_threshold": 0.4, "optimal_utility": 0.3875, "always_execute_utility": 0.2875, "per_bin_accuracy": [ 0.09090909361839294, 0.375, 0.25, 0.1666666716337204, 0.5714285969734192, 0.111111119389534, 0.75, NaN, 0.6666666865348816, NaN, 0.75, 0.75, 0.5, 0.75,
```

0.9733333587646484

```
], "per_bin_confidence": [ 0.021917201578617096, 0.09265217185020447, 0.16482609510421753, 0.23125995695590973, 0.2980838716030121, 0.3666154146194458, 0.4179833233356476, NaN, 0.5548346042633057, NaN, 0.6987861394882202, 0.7771269083023071, 0.8376951217651367, 0.9133914709091187,
```

0.9908593893051147

```
], "per_bin_counts": [ 22, 8, 8, 6, 7, 9, 4, 0, 3, 0, 4,
```

原文第 81 页

```
4, 6, 4, ], "calibration_by_difficulty": [ { "difficulty": "easy", "count": 30, "success_rate": 1.0, "mean_predicted_prob": 0.9949233531951904, "ece": 0.00507664680480957 }, { "difficulty": "extreme", "count": 30, "success_rate": 0.10000000149011612, "mean_predicted_prob": 0.05613331496715546, "ece": 0.09961698924501737 }, { "difficulty": "hard", "count": 31, "success_rate": 0.774193525314331, "mean_predicted_prob": 0.7828013896942139, "ece": 0.20854707110312679 }, { "difficulty": "medium", "count": 34, "success_rate": 1.0, "mean_predicted_prob": 0.9946644902229309, "ece": 0.005335509777069092 }, { "difficulty": "very_hard", "count": 35, "success_rate": 0.34285715222358704, "mean_predicted_prob": 0.3409382700920105, "ece": 0.2759824851793903 }
```

]

```
}, "xgb_iso": { "auroc": 0.884857775506728, "average_precision": 0.9091656066307845, "ece": 0.09425514140166341, "brier_score": 0.125672847032547, "expected_utility_gain": 0.09562500000000007, "optimal_threshold": 0.365, "optimal_utility": 0.38312500000000005, "always_execute_utility": 0.2875, "per_bin_accuracy": [ 0.14814814925193787, 0.0, 1.0, 0.0,
```

原文第 82 页

```
NaN, 0.27586206793785095, NaN, NaN, 0.625, NaN, NaN, 1.0, NaN, 0.625,
```

0.9404761791229248

```
], "per_bin_confidence": [ 1.0000001111620804e-06, 0.09282594174146652, 0.17241021990776062, 0.22327017784118652, NaN, 0.3563217520713806, NaN, NaN, 0.5882353186607361, NaN, NaN, 0.772962212562561, NaN, 0.889986515045166,
```

0.9995569586753845

```
], "per_bin_counts": [ 27, 1, 1, 1, 0, 29, 0, 0, 8, 0, 0, 1, 0, 8, ], "calibration_by_difficulty": [ { "difficulty": "easy", "count": 30, "success_rate": 1.0, "mean_predicted_prob": 0.9999988079071045, "ece": 1.1920928955078125e-06 }, { "difficulty": "extreme", "count": 30,
```

原文第 83 页


```
"success_rate": 0.10000000149011612, "mean_predicted_prob": 0.05020613968372345, "ece": 0.15020447770754497 }, { "
difficulty": "hard", "count": 31, "success_rate": 0.774193525314331, "mean_predicted_prob": 0.8620743751525879, "ece":
0.17294803646302992 }, { "difficulty": "medium", "count": 34, "success_rate": 1.0, "mean_predicted_prob": 0.9999989867210388, "
ece": 1.0132789611816406e-06 }, { "difficulty": "very_hard", "count": 35, "success_rate": 0.34285715222358704, "
mean_predicted_prob": 0.4329416751861572, "ece": 0.19451816082000734 }
```

```
]
}, "xgb_platt": { "auroc": 0.9192641798671436, "average_precision": 0.9602022705286992, "ece": 0.09693039900938674, "
brier_score": 0.12277982262110032, "expected_utility_gain": 0.10000000000000003, "optimal_threshold": 0.49, "optimal_utility":
0.3875, "always_execute_utility": 0.2875, "per_bin_accuracy": [ NaN, 0.1666666716337204, 0.25, 0.0, 0.5, 0.5, 0.0, 0.5, 1.0, NaN,
1.0, 0.0, NaN, 0.6000000238418579,
```

```
0.9318181872367859
```

```
], "per_bin_confidence": [ NaN, 0.0842537559577621, 0.16620587271613368,
```

原文第 84 页

```
0.21702852269564366, 0.28436128350829754, 0.3602636581771883, 0.457294293952662, 0.5070745062068907,
0.5668335321032509, NaN, 0.7208558302857786, 0.782196860971275, NaN, 0.8908686074686025,
```

```
0.9769863642890502
```

```
], "per_bin_counts": [ 0, 30, 8, 4, 4, 8, 5, 4, 1, 0, 2, 1, 0, 5, ], "calibration_by_difficulty": [ { "difficulty": "easy", "count": 30, "
success_rate": 1.0, "mean_predicted_prob": 0.9815383148599685, "ece": 0.018461685140031547 }, { "difficulty": "extreme", "
count": 30, "success_rate": 0.10000000149011612, "mean_predicted_prob": 0.09840705164342502, "ece": 0.05995777293852756 },
{ "difficulty": "hard", "count": 31, "success_rate": 0.774193525314331, "mean_predicted_prob": 0.856000641315412, "ece":
0.19403608435115627 }, { "difficulty": "medium", "count": 34, "success_rate": 1.0, "mean_predicted_prob": 0.9815154640831539,
```

原文第 85 页

```
"ece": 0.018484535916846068 }, { "difficulty": "very_hard", "count": 35, "success_rate": 0.34285715222358704, "
mean_predicted_prob": 0.39924788968585045, "ece": 0.3602689596425007 }
```

```
]
```

```
}}
```

```
### 文件 : 'requirements.txt'
```

核心仿真

```
robosuite>=1.5.1
```

```
mujoco>=3.0.0
```

数值计算 / 机器学习

```
numpy>=2.0.0
```

```
scipy>=1.11.0
```

```
pandas>=2.0.0
```

```
scikit-learn>=1.3.0
```

xgboost>=2.0.0

lightgbm>=4.0.0

joblib>=1.3.0

绘图

matplotlib>=3.7.0

其他

tqdm>=4.65.0

```
### 文件 : 'main.py'
```

```
def main():
```

```
    print("Hello from query318!")
```

```
if __name__ == "__main__":
```

```
    main()
```

```
    # 参赛方案 B 的工作区 ( 仅含证据 )
```

```
    路径 : '...'
```

```
    ## 目录树 ( 示例 )
```

```
query318_defaultTeam/
```

```
data/
```

```
raw/
```

```
dataset.csv
```

```
dataset_stats.json
```

```
deliverables/
```

```
plots/
```

```
calibration_by_difficulty_bin.png [.png 二进制/大文件]
```

```
decision_utility_curve.png [.png 二进制/大文件]
```

```
ece_vs_temperature.png [.png 二进制/大文件]
```

原文第 86 页

```
reliability_diagram.png [.png binary/large]
```

```
roc_curve.png [.png binary/large]
```

```
index.json
```

```
logs/
```

```
models/
```

calibrated_predictions.json

gbt_model.pkl [.pkl binary/large]

iso_gbt.pkl [.pkl binary/large]

mlp_model.pkl [.pkl binary/large]

predictions.json

scaler.pkl [.pkl binary/large]

split_data.json

temperature_meta.json

results/

metrics.json

model_comparison.json

src/

calibrate_models.py

evaluate.py

generate_data.py

plot_results.py

train_models.py

.gitignore

.python-version

logs.txt

main.py

pyproject.toml

requirements.txt

research_report.md

uv.lock

核心文件

- 'deliverables/index.json': 已找到
- 'requirements.txt': 已找到
- 'research_report.md': 已找到
- 'results/metrics.json': 已找到

从任务文本中提取的图表路径 (子字符串匹配)

- 提及数量: 5

– (ok) 'deliverables/plots/calibration_by_difficulty_bin.png' – (ok) 'deliverables/plots/decision_utility_curve.png' – (ok) 'deliverables/plots/ece_vs_temperature.png' – (ok) 'deliverables/plots/reliability_diagram.png' – (ok) 'deliverables/plots/roc_curve.png'

文件摘录 (大小上限; 二进制文件已省略) ### 文件: 'research_report.md' [已截断] (内容) ### 文件: 'deliverables/index.json' (内容) ### 文件: 'results/metrics.json' (内容) ### 文件: 'results/model_comparison.json' (内容)

原文第 87 页

文件：'requirements.txt'（内容） ### 文件：'main.py'（内容） ## 对比评分标准（相对）
判断哪份提交更好地完成了同一任务。评估维度与单份提交的资深审阅相同：

1. 可交付物覆盖度——将任务中 Deliverables / Deliverable: / 内联可交付物列表（以及"# Task"中其他明确的输出要求）逐一映射到证据；标注缺失或占位内容。
2. 数值/实证严谨性——方法论是否合理、基线是否充分、局限性是否诚实说明；报告与指标之间的一致性。
3. 可复现性——依赖项、入口点、随机种子、数据文档或生成方式。
4. 呈现质量——报告结构、清晰度、图表整合情况（从路径和摘录推断）。
5. 工程质量——目录布局、模块化程度、代码可读性（从摘录和目录树判断）。
6. 任务对齐度——若解决了错误问题或偏离了既定目标则予以扣分。

输出 JSON 模式（精确键名）{"winner": "A" | "B" | "tie", "rationale": "<字符串，引用两个工作区的具体证据>"}

仅返回 JSON。

F. 归一化成本、输出 token 数及缓存读写量的分布

原文第 88 页

high max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

Normalized value

Normalizedcost

high max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

Normalizedoutput

high max

high+ [0]

high+ [1]

high+ [2]

high+ [3]

high+ [4]

NormalizedcacheRead+Write

(a) sonnet-4.6

high xhigh max

high+ [0]

high+ [1]

0.0

2.5

5.0

7.5

10.0

12.5

15.0

17.5

20.0

Normalized value

Normalizedcost

high xhigh max

high+ [0]

high+ [1]

Normalizedoutput

high xhigh max

high+ [0]

high+ [1]

NormalizedcacheRead+Write

(b) opus-4.7

high xhigh max

ultracode

high+ [0]

high+ [1]

high+ [2]

Normalized value

Normalizedcost

high xhigh max

ultracode

high+ [0]

high+ [1]

high+ [2]

0.0

2.5

5.0

7.5

10.0

12.5

15.0

17.5

Normalizedoutput

high xhigh max

ultracode

high+ [0]

high+ [1]

high+ [2]

0.0

2.5

5.0

7.5

10.0

12.5

15.0

17.5

20.0

NormalizedcacheRead+Write

(c) opus-4.8

图14 | 30 个 ML 合成任务中归一化成本、输出 token 数及缓存读写量的分布。箱线图显示，分布趋势与图 5b、6c 和 7e 中报告的均值结果一致。